Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

Mdig functions

Synopsis

The functions prefixed with Mdig make up the Digitizer module. The Digitizer Control module allows you to manipulate and control the digitizer section of an imaging board. It also allows you to set the state required for either simple or complex acquisition and allows acquisition functions to be sent to the imaging board.

Functions

MdigAlloc
MdigControl
MdigControlFeature
MdigFocus
MdigFree
MdigGetHookInfo
MdigGrab
MdigGrabContinuous
MdigGrabWait
MdigHalt
MdigHookFunction
MdigInquire
MdigInquireFeature
MdigProcess

MdigAlloc

Synopsis

Allocate a digitizer.

Syntax

```
AIL_ID MdigAlloc(
    AIL_ID SystemId,               //in
    AIL_INT DigNum,                //in
    AIL_CONST_TEXT_PTR DataFormat, //in
    AIL_INT64 InitFlag,            //in
    AIL_ID *DigIdPtr               //out
)
```

Description

This function allocates a digitizer on the specified system so that it can be used by subsequent Aurora Imaging Library digitizer functions. A digitizer on the target system must be allocated to acquire data from a camera. Its device number specifies the acquisition path to allocate for the digitizer. Use MdigGrab(), MdigGrabContinuous(), or MdigProcess() to grab.

Typically, each digitizer only uses a single acquisition path. However, in some cases a digitizer might use several acquisition paths depending on the input format of the camera. Its digitizer configuration format (DCF) establishes the number of acquisition paths used.

Multiple digitizers can be allocated on the same system. If the system supports the possibility of independent acquisition paths, the digitizers' device number together with their DCF establishes if they represent dependent or independent acquisition path(s) on the system. If independent, grabs from these digitizers occur simultaneously. If dependent, grabs from these digitizers occur consecutively.

If multiple cameras of the same type are connected to the same acquisition path(s), instead of allocating multiple digitizers, you can allocate a single digitizer in the required format and then use MdigControl() with M_CHANNEL to switch between the cameras. M_CHANNEL selects the active input channel of the digitizer. The switch occurs upon the next call to MdigGrab(), MdigGrabContinuous(), or MdigProcess() with the digitizer. The default input channel is determined by the selected DCF (generally, M_CH0).

It is faster to allocate all the required digitizers first, perform the required grabs, and then free all the digitizers rather than allocate, grab, and then free each digitizer in succession.

Upon execution of this function, Aurora Imaging Library ensures that the digitizer is present before allocating it and generates an error if it is not. Note that the corresponding hardware won't necessarily be programmed at the time of allocation.

After allocating the digitizer, you should check if the operation was successful, using MappGetError() or by verifying that the digitizer identifier returned is not **M_NULL** (or *nullptr* if M_UNIQUE_ID was specified).

When the digitizer is no longer required, release it using MdigFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the digitizer's lifetime and you must not manually free it.

Parameters

SystemId INQ

Specifies the identifier of the system on which to allocate the digitizer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.						a									
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

DigNum INQ

Specifies the number or string to identify the device to allocate for the digitizer.

This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the device number

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default device number. This value is set during Aurora Imaging Library installation and using the Aurora Imaging Configurator utility. Note that the end-user can always change this value.	c		e	f	g	a		i		j	k	l	m	n	b
M_GC_CAMERA_ID(AIL_TEXT_PTR CameraIdentifierString)	Specifies the camera (or device) to allocate for the digitizer.			e	f	g									n	
	Parameters															
	CameraIdentifierString Sets the camera identifier.			e	f	g									n	
	System specific															
	When the InitFlag is set to M_GC_DEVICE_IP_ADDRESS, specify the camera's IP address. When the InitFlag is set to M_GC_DEVICE_NAME, specify the camera's name. This macro is not available with any other InitFlag.			e	f	g										
	Note that this macro can only be used when the InitFlag is set to M_GC_DEVICE_NAME.														n	
M_DEV*n*	Specifies which acquisition path on your frame grabber, or which camera by rank when dealing with a grabberless interface camera (for example, a GigE Vision or USB3 Vision camera), to allocate for the digitizer. You can inquire the total number of possible independent acquisition paths on a system using MsysInquire() with M_DIGITIZER_NUM.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	The value of *n* must be from 0 to 31. This number represents the index of the simulated digitizer.						a									
	The value of *n* must be from 0 to 31. This number represents the camera's index number. Note that this value can only be used when the InitFlag is set M_DEV_NUMBER.			e	f	g									n	b
	The value of *n* must be 0 for Zebra Iris GTX.								i							
	The value of *n* must be from 0 to 3. If you are allocating a digitizer for a camera that uses multiple connections, specify the index of the connector on your board used for the main connection. The other connections used for the camera are associated with the digitizer automatically.												l	m		
	When dealing with the SF version of your Zebra hardware, the value of *n* must be 0. When dealing with the DB version or DF version of your Zebra hardware, the value of *n* must be either 0 or 1. When dealing with a QB version of your Zebra hardware, the value of *n* must be 0 to 3.										j	k				

DataFormat INQ

Specifies the name of the digitizer configuration format (DCF) appropriate for your camera. Depending on the target system, different DCFs are supported.

One of the following values can be specified when allocating a digitizer for either a monochrome or a color camera, or a 3D sensor:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For monochrome or color cameras, or 3D sensors																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

AIL_TEXT("M_DEFAULT") 1	Specifies the default digitizer format. This value is set during Aurora Imaging Library installation and using the Aurora Imaging Configurator utility. Note that the end-user can always change this value.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies the data format is set by the digitizer allocated as the multicast controller. Note that this value should only be used when allocating a multicast worker (using M_GC_MULTICAST_WORKER). For more information about multicasting and configuring your Zebra GigE Vision digitizer, refer to the Zebra GigE Vision driver chapter in the Hardware-specific Notes.			e	f	g										b
AIL_TEXT("*DCF File name*") 1	Specifies the path and file name of the DCF (for example: "*C:\mydirectory\myfile*"). See the installed Drivers directory (for example, *C:\Program Files\Aurora Imaging Library\11\Drivers* for the current list of supported formats. When the path of the directory from which to retrieve the file is on a remote computer (under Distributed Aurora Imaging Library), precede the path with "*remote:///*" (for example, "*remote:///C:\mydirectory*").	c		e	f	g			i		j	k	l	m	n	b
	System specific															
	Use the Intellicam program to create your own DCF file.			e	f	g					j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

One of the following values can be specified when allocating a simulated digitizer:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For simulated digitizers																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
AIL_TEXT("*Image path[@n]*") 1	Specifies the path of the directory from which to retrieve image files (or an AVI file) when allocating a simulated digitizer (for example, "*C:\mydirectory*"), where *n* is the frame rate; the information in brackets is optional (for example, "*C:\mydirectory\@30*"). For a list of image file formats supported, refer to MbufImport(). Note that, the X- and Y-size of each image in the path should be the same; otherwise, the X- and Y-size is taken from the first image and all subsequent images are either cropped to these dimensions, or expanded without zooming or stretching the original image, accordingly. In the latter case, portions of the previous image(s) might still be visible at the borders if the new image is smaller than the previous image. When the path of the directory from which to retrieve the image files (or AVI file) is on a remote computer (under Distributed Aurora Imaging Library), precede the path with "*remote:///*" (for example, "*remote:///C:\mydirectory*").						a									
AIL_TEXT("M_2D_SIMULATOR") 1	Specifies to use Aurora Imaging Library's internal 2D pattern generator with the simulated digitizer. This generates a series of images of an arrow, which can be used to simulate acquisition without the need to create sample images. The generated images are 640x480, in the 8-bit RGB color format.						a									
AIL_TEXT("M_3D_SIMULATOR") 1	Specifies to use Aurora Imaging Library's internal 3D pattern generator with the simulated digitizer. This generates a meshed point cloud of an arrow, which can be used to simulate acquisition without the need to create sample point clouds.						a									
AIL_TEXT("*SDCF File name*") 1	Specifies the path and file name of the simulated DCF for use with a simulated digitizer (for example: "*C:\mydirectory\myfile*"). When the path of the directory from which to retrieve the SDCF file is on a remote computer (under Distributed Aurora Imaging Library), precede the path with "*remote:///*" (for example, "*remote:///C:\mydirectory*").						a									

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

InitFlag INQ

Specifies the type of initialization to perform on the digitizer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of initialization																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	The default value is M_DEV_NUMBER.			e	f	g									n	b
	The default value is M_EMULATED.						a									
	Specifies that grabbing is permitted with the allocated digitizer.	c														
M_DEV_NUMBER +	Specifies to allocate the digitizer for the camera with the specified device number. To specify the device number, use the DigNum paramater (M_DEVn).			e	f	g									n	b
M_EMULATED	Specifies to allocate the digitizer as a simulated digitizer.						a									
M_GC_DEVICE_IP_ADDRESS +	Specifies to allocate the digitizer for the camera with the specified IP address. To specify the IP address, use M_GC_CAMERA_ID().			e	f	g										
M_GC_DEVICE_NAME +	Specifies to allocate the digitizer for the camera with the specified device's user name. To specify the device's user name, use M_GC_CAMERA_ID().			e	f	g									n	
M_MINIMAL	Specifies that grabbing is not permitted with the allocated digitizer. When performing a grab, using either MdigGrab() or MdigProcess(), an error is generated. To receive a callback when a camera is connected or disconnected, use MdigHookFunction() with M_CAMERA_PRESENT. Up to 8 digitizers from M_DEV0 to M_DEV7 can be allocated simultaneously with this type of initialization.	c														

Combination values that can be used alone or as combination values for M_DEV_NUMBER; M_GC_DEVICE_IP_ADDRESS; M_GC_DEVICE_NAME.

You can use one or more of the following values in combination with each other or with the above-mentioned values, to specify further details of the allocation of your camera.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying further details																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_GC_MANIFEST_ENTRY(AIL_INT EntryNum)	Specifies which camera's device description file (XML) to download from the camera's manifest table. Note that, by default the Aurora Imaging Library driver will always try to download a GenICam schema 1.1 compliant XML file. If no such file is present, then a GenICam schema 1.0 file is used. If manifest tables are not supported in the camera, then the single camera's device description file is downloaded. To skip this download, use M_GC_XML_DOWNLOAD_SKIP.			e	f	g									n	
	Parameters															
	EntryNum Specifies which XML to download from the camera's manifest table. This can be a value from 0 to 15.			e	f	g									n	
M_GC_PACKET_SIZE_NEGOTIATION_SKIP	Specifies that the packet size negotiation task is skipped. Instead, Aurora Imaging Library uses the GigE Vision digitizer's packet size register. If the packet size is too small or if the packet size is not supported by your digitizer, acquisition reliability will suffer. To force the packet size to a certain value, either store it in the DCF or use MdigControl() with M_GC_PACKET_SIZE.				f	g										

M_GC_XML_DOWNLOAD_SKIP	Specifies that the camera's device description file (XML) should not be downloaded from the camera associated with the digitizer. The cached version, present in the Aurora Imaging Library XML repository is used instead. If the camera's device description file is not present in the Aurora Imaging Library XML repository or if the camera's device description file's name and/or size differs from the file in the Aurora Imaging Library XML repository, the camera's device description file will be downloaded and this flag is ignored. Note that M_GC_XML_DOWNLOAD_SKIP and M_GC_XML_FORCE_DOWNLOAD are mutually exclusive, and cannot be used in combination with each other.				f	g								n	
M_GC_XML_FORCE_DOWNLOAD	Specifies that the camera's device description file (XML) should be downloaded from the camera associated with the digitizer. The cached version, present in the Aurora Imaging Library XML repository is overwritten. Note that M_GC_XML_DOWNLOAD_SKIP and M_GC_XML_FORCE_DOWNLOAD are mutually exclusive, and cannot be used in combination with each other.				f	g								n	

Combination values that can be used alone or as combination values for M_DEV_NUMBER; M_GC_DEVICE_IP_ADDRESS; M_GC_DEVICE_NAME.

You can use one of the following values on its own, or add it to the above-mentioned values, to specify whether the allocated digitizer will be a controller, monitor, or worker digitizer in a multicast controller-worker relationship.

For information about using IP multicast and configuring your Zebra GigE Vision digitizer, refer to the Using IP multicast section of the Zebra GigE Vision driver chapter in the Hardware-specific Notes.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the multicasting controller-worker relationship																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_GC_MULTICAST_CONTROLLER	Specifies that the allocated digitizer will be the controller digitizer in a multicasting controller-worker or monitor relationship. Assigning the allocated digitizer as a controller digitizer does not limit the digitizer in any way. In addition, as a multicast controller digitizer, it is responsible to program the camera's destination stream and message channels (if supported) with an administratively scoped IP multicast address.					g										
M_GC_MULTICAST_MONITOR	Specifies that the allocated digitizer will be a special type of worker digitizer (a monitor digitizer) in a multicast controller-worker relationship. Assigning the allocated digitizer as a monitor digitizer limits its capabilities, since a monitor digitizer can only receive images and events (GenICam messages) from the camera and request packet resends, as necessary. In the controller-worker relationship, only the multicast controller digitizer has control. Therefore, the monitor digitizer's DCF that you pass **MdigAlloc**() should only contain X-size, Y-size, and pixel format information. Image acquisition is only performed when the controller digitizer (on another computer) initiates a grab and the GigE Vision camera streams the data. If the controller has not initiated the image acquisition, then when using MdigGrab(), MdigGrabContinuous(), or MdigProcess() with the monitor digitizer, the monitor digitizer will wait until the controller digitizer initiates the image acquisition. Note that a monitor digitizer cannot be allocated if the GigE Vision camera is in **chunk mode**.					g										

M_GC_MULTICAST_WORKER	Specifies that the allocated digitizer will be a worker digitizer in a multicast controller-worker relationship. Assigning the allocated digitizer as a worker digitizer limits its capabilities, since a worker digitizer can only read from the camera (using MdigInquireFeature()), receive images and events (GenICam messages) from the camera, and request packet resends. In the controller-worker relationship, only the multicast controller digitizer has control. Therefore, the worker digitizer's DCF that you pass to **MdigAlloc**() is not used. For best effect, set the DataFormat parameter to M_NULL. Image acquisition is only performed when the controller digitizer (on another computer) initiates a grab and the GigE Vision camera streams the data. If the controller has not initiated the image acquisition, then when using MdigGrab(), MdigGrabContinuous(), or MdigProcess() with the worker digitizer, the worker digitizer will wait until the controller digitizer initiates the image acquisition.					g									

DigIdPtr

Specifies the address of the variable in which to write the digitizer identifier or specifies the data type that the function should use to return the digitizer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated digitizer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated digitizer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_DIG_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the digitizer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated digitizer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the digitizer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_DIG_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Remark

- If you are creating a DLL that includes a call to **MdigAlloc**(), ensure that the call is not made from the `DllMain()` function, because **MdigAlloc**() might load a required DLL and you cannot load a DLL from `DllMain()`. If necessary, call **MdigAlloc**() from an initialization function in your DLL instead.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MdigControl

Synopsis

Control a digitizer setting.

Syntax

```
void MdigControl(
    AIL_ID DigId,           //out
    AIL_INT64 ControlType,  //in
    AIL_DOUBLE ControlValue //in
)
```

Description

This function allows you to control various digitizer settings.

> Note that, when a control type has only one supported control value on a given system, it is not documented in this function because it cannot be changed to another value. Instead it can only be inquired.

To inquire the current value of a particular digitizer setting, use MdigInquire().

If you change the setting of a control type of a digitizer currently being used in a grab, the change does not affect the current grab. It does, however, affect the very next grab.

[Zebra Radient eV-CL; Zebra Rapixo CL; Zebra Rapixo CoF; Zebra Rapixo CXP]

Note that, to change this default behavior and have the change affect the digitizer even if it is currently being used in a grab, use M_COMMAND_QUEUE_MODE.

You can also interactively control and test most of the digitizer settings and obtain results in real-time, using Feature Browser; to access this interface, use Aurora Imaging Intellicam or **MdigControl**() with M_GC_FEATURE_BROWSER. If your camera is connected to a CoaXPress (CXP) communication standard-compliant frame grabber (such as Zebra Rapixo CXP), you can save digitizer (and camera) settings selected in Intellicam's Feature Browser to the DCF for your camera.

[Zebra GenTL driver; Zebra GigE Vision driver; Zebra USB3 Vision driver; Video4Linux2]

Typically, the supported control types affect the GenICam-compliant camera (or device) associated with the digitizer. These control types assume your camera supports the associated GenICam standard feature naming convention (SFNC) feature. If the device does not support the associated GenICam SFNC feature, an error is generated. In case of error, use MdigControlFeature() with the device-specific feature names and values specified by the device manufacturer.

Note that there are several control types that are available on a Host system, even without any additional Zebra imaging boards; these are to control a simulated digitizer (allocated using MdigAlloc() with M_EMULATED). To see the control types available to control a simulated digitizer, select the Host system as your board type in this Help file. For more information on simulated digitizers, refer to the Simulated digitizer section of Chapter 27: Grabbing with your digitizer.

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **MdigControl()** internally calls the *AIL_INT64* version of this function (MdigControlInt64()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (MdigControlDouble()).

Parameters

DigId

Specifies the identifier of the digitizer.

ControlType

Specifies the digitizer setting to control.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the value to assign to the digitizer setting.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

- For the general digitizer settings
- For the general reference settings
- For controlling the input gain and shading correction
- For routing I/O signals and setting their mode
- For setting the state of specified user-bits in a static-user-output register
- For controlling the settings to grab using a trigger
- For controlling frame bursts
- For controlling the settings of a timer
- For connection testing
- For controlling the camera's exposure
- For controlling the settings of a quadrature decoder with inputs from a rotary or linear encoder
- For controlling a data latch

The following control types allow you to control various digitizer settings.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the general digitizer settings																	
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																
M_BAYER_COEFFICIENTS_ID	Loads the internal white balance coefficients buffer with the values in the specified buffer. To have your digitizer automatically calculate these coefficients and load them into the internal buffer, use the M_WHITE_BALANCE control type with M_CALCULATE. Note that the internal buffer only has an effect if the M_BAYER_CONVERSION and M_WHITE_BALANCE control types are enabled. If you change the coefficient values in the specified buffer, you must call **MdigControl**() again with the updated buffer to load the new values into the internal white balance coefficients buffer. INQ																
	System specific																
	Refer to your Zebra imaging board's hardware manual for more details.										j	k	l	m		
	Note that this control type is used to perform Bayer conversion if performed by the Host. To control the Bayer conversion on the camera, use MdigControlFeature(); refer to your camera's documentation for details regarding the features to set.			e	f	g									n	b
	Note that this control type is only available when using color versions of Zebra Iris GTX (such as 2000C).								i							
M_DEFAULT	Specifies to use the default coefficients that were automatically set, following a call to MdigAlloc(). Note this is the same as using M_WHITE_BALANCE with M_DISABLE.			e	f	g			i						n	b
Array buffer identifier	Specifies the identifier of an M_ARRAY buffer containing the white balance coefficients. The buffer must be allocated as a single-band 32-bit floating-point buffer with a 3x1 dimension. For more information, see White balancing your Bayer images.			e	f	g			i		j	k	l	m	n	b
M_BAYER_CONVERSION	Sets whether to perform a Bayer color conversion, using your digitizer. This control type can only be used when grabbing from a camera that has a Bayer color filter, otherwise an error will be generated. For a more conventional technique of controlling Bayer conversion, use MbufBayer() and see White balancing your Bayer images. INQ			e	f	g			i		j	k	l	m	n	b

		System specific															
		Refer to your Zebra imaging board's hardware manual for more details.									j	k	l	m			
		Note that this control type is used to perform Bayer conversion if performed by the Host. To control the Bayer conversion on the camera, use MdigControlFeature(); refer to your camera's documentation for details regarding the features to set.			e	f	g								n	b
		Note that this control type is only available when using color versions of Zebra Iris GTX (such as 2000C).								i							
	M_DEFAULT	Same as M_ENABLE.			e	f	g			i		j	k	l	m	n	b
	M_DISABLE	Specifies to not perform Bayer conversion on the grabbed image.			e	f	g			i		j	k	l	m	n	b
	M_ENABLE	Specifies to perform Bayer conversion on the grabbed image. Note that to white balance an image, M_WHITE_BALANCE must also be set to M_ENABLE.			e	f	g			i		j	k	l	m	n	b
M_CHANNEL		Sets the channel on which the digitizer is to acquire data, as well as sets the synchronization channel to the default for the selected data channel. Note that, typically, this control type is only available when the specified digitizer uses acquisition paths with several multiplexed data inputs. This means that they have several data input channels but can only grab from one channel at a time. INQ	c														
	M_DEFAULT	Same as M_CH0.	c														
	M_CH0	Specifies channel 0 as the channel on which the digitizer is to receive input data. For Y/C input, this corresponds to VID_IN2 (Green of camera 0) and VID_IN3 (Blue of camera 0). For CVBS input, this corresponds to VID_IN2 (Green of camera 0).	c														
	M_CH1	Specifies channel 1 as the channel on which the digitizer is to receive input data. For Y/C input, this corresponds to VID_IN6 (Luminance of camera 1) and VID_IN5 (Chrominance of camera 1). For CVBS input, this corresponds to VID_IN5 (Chrominance of camera 1).	c														
	M_CH2	Specifies channel 2 as the channel on which the digitizer is to receive input data. For CVBS input, this corresponds to VID_IN6 (Luminance of camera 2).	c														
M_COMMAND_QUEUE_MODE		Sets whether changes to digitizer settings affect the digitizer immediately, even if it is currently being used in a grab, or the change will wait until the current grab is complete. INQ				f						j	k	l	m		
	M_DEFAULT	Same as M_QUEUED.				f						j	k	l	m		
	M_IMMEDIATE	Specifies that, if you change the setting of a control type of a digitizer currently being used in a grab, the change affects the current grab. Note that, this can cause the grabbed image to become corrupt.				f						j	k	l	m		
	M_QUEUED	Specifies that, if you change the setting of a control type of a digitizer currently being used in a grab, the change does not affect the current grab. Instead, the changes are queued and applied before the very next grab.				f						j	k	l	m		

M_CORRUPTED_FRAME_ERROR	Sets whether an error is generated when a corrupted or incomplete frame is grabbed. INQ	c		e	f	g			i					n	b
	System specific														
	Note that whether the grabbed image is corrupt can always be inquired using MdigGetHookInfo() with M_CORRUPTED_FRAME from the user-defined function hooked to the end of a grab using MdigProcess().			e	f	g								n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g			i					n	b
M_DISABLE	Specifies not to generate an error when grabbing a corrupted frame.	c		e	f	g			i					n	b
M_ENABLE	Specifies to generate an error when grabbing a corrupted frame.	c		e	f	g			i					n	b
M_DIGITIZER_INTERNAL_BUFFERS_NUM	Sets the number of internal grab buffers allocated and used when Aurora Imaging Library cannot grab directly into the specified buffers (for example, when the grab buffer's format is not compatible with the camera's current pixel format or when the image sent from a GenICam-compliant camera has additional information that needs to be stripped out to make the grabbed image compatible with Aurora Imaging Library buffers, such as when GenICam "chunk mode" is enabled). INQ			e		g								n	b
M_DEFAULT	Specifies the default value; the default value is 5.			e		g								n	b
Value >= 0	Specifies the number of internal grab buffers.			e		g								n	b
M_FIX_PATTERN_NOISE_CORRECTION	Sets the type of fixed pattern noise correction to apply when acquiring images. Note that, acquisition must be stopped prior to changing this control type. INQ								i						
M_DEFAULT	Same as M_PREPROCESSING.								i						
M_PREPROCESSING	Specifies a correction for dark signal non-uniformity and for photo response non-uniformity, applied to the grabbed images. Aurora Imaging Library calculates the correction factor(s) when the control type is called.								i						
M_SENSOR	Specifies a correction for dark signal non-uniformity, applied to the grabbed images. Aurora Imaging Library adjusts the correction factor for each image.								i						
M_FOCUS +	Sets the lens focus capabilities. Note that this control type requires that your Zebra Iris GTX use an I^2C (inter-integrated circuit) controlled lens. INQ								i						
Min. focus <= *Value* <= *Max. focus*	Specifies a value between the minimum and maximum values supported by the camera. Use MdigInquire() to determine these values.								i						
M_FOCUS_PERSISTENCE	Sets whether to store a focus position for the I_2C (inter-integrated circuit) controlled lens. This stored focus position is written to the I_2C (inter-integrated circuit) controlled lens when the smart camera is allocated, using MdigAlloc(). If there is no I_2C (inter-integrated circuit) controlled lens attached, the value is ignored. INQ								i						
M_DEFAULT	Same as M_DISABLE.								i						
M_DISABLE	Specifies to not store the focus position.								i						

	M_ENABLE	Specifies to store the focus position.								i						
M_FOCUS_PERSISTENT_VALUE		Sets the focus position to store for the I_2C (inter-integrated circuit) controlled lens. If there is no I_2C (inter-integrated circuit) controlled lens attached, the value is ignored. INQ								i						
	0 <= *Value* <= 1023	Specifies to store the focus position.								i						
M_GC_CLPROTOCOL		Sets whether the GenICam CLProtocol module is enabled. The GenICam CLProtocol module allows Camera Link cameras to be accessed using GenICam-specific constants (including M_GC_FEATURE_BROWSER), as well as using MdigControlFeature() and MdigInquireFeature(). Even when the GenICam CLProtocol module is enabled, you should follow the Aurora Imaging Library documentation specific to your board; if a control type requires the CLProtocol module to be enabled, it will be noted. Note that this control type should be enabled only if using a GenCP-compliant camera or after the required third-party, vendor-supplied, standard-compliant CLProtocol library for the Camera Link camera is installed on your computer and M_GC_CLPROTOCOL_DEVICE_ID is set. INQ										j	k			
	M_DEFAULT	Same as M_DISABLE.										j	k			
	M_DISABLE	Specifies that the GenICam CLProtocol module is disabled.										j	k			
	M_ENABLE	Specifies that the GenICam CLProtocol module is enabled.										j	k			
M_GC_CLPROTOCOL_DEVICE_ID		Identifies the Camera Link camera connected to your digitizer, as well as the GenICam CLProtocol library for the camera. When allocating a digitizer for a camera, the digitizer identifies that the connected device is a camera; M_GC_CLPROTOCOL_DEVICE_ID provides the digitizer with the details required to identify a specific Camera Link camera and its corresponding GenICam CLProtocol library. Note that this control type should be used only if using a GenCP-compliant camera or after the required third-party vendor-supplied, standard compliant, CLProtocol library for the Camera Link camera is installed on your computer. To enumerate the complete list of camera identifiers installed on your computer, use MdigInquire() with M_GC_CLPROTOCOL_DEVICE_ID + *n*. For an example, see **CLProtocol.cpp**. Even after you have specified the library to use, you must enable the CLProtocol module (with M_GC_CLPROTOCOL set to M_ENABLE) before you can use an Aurora Imaging Library GenICam constant (including M_GC_FEATURE_BROWSER), or use MdigControlFeature(), or MdigInquireFeature(). INQ										j	k			
	AIL_TEXT("*CameraLinkCameraIdentifier*") 1	Specifies the identifier string of your Camera Link camera and its corresponding GenICam CLProtocol library. The specified string should contain the tokens separated by the hash ("#") sign instead of back slashes (for example, "LibraryDirectory#LibraryFileName#Manufacturer#Family#Model#Version). Each of these tokens must follow the naming convention for C variables. The specified string must contain sufficient data to uniquely identify the Camera Link camera and its GenICam CLProtocol library. > Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*CameraLinkCameraIdentifier*")).										j	k			

AIL_TEXT("M_DEFAULT") 1	Specifies to use the default identifier string of your Camera Link camera and its corresponding GenICam CLProtocol library, specified using the Aurora Imaging Configurator utility. The specified string must contain sufficient data to uniquely identify the Camera Link camera and its GenICam CLProtocol library. > Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("M_DEFAULT")).									j	k				
M_GC_FEATURE_BROWSER +	[This is only applicable to Windows] Sets whether to open or close a window that allows you to view and edit the GenICam-compliant camera configuration information interactively. This window is referred to as the Feature Browser.	c		e	f	g				j	k	l	m	n	
	System specific														
	Note that, this control type is only available if using a GenCP-compatible camera, or if you first install the third-party, vendor-supplied, standard compliant CLProtocol library for the Camera Link camera connected to your digitizer.									j	k				
	The Zebra GenTL Consumer (library) allows you to edit the GenICam-compliant stream, device, or remote device configuration information.			e											
M_DEFAULT	Same as M_OPEN.	c		e	f	g				j	k	l	m	n	
M_CLOSE	Closes Feature Browser.	c		e	f	g				j	k	l	m	n	
M_OPEN +	Opens Feature Browser.	c		e	f	g				j	k	l	m	n	
	System specific														
	You must specify a combination value from the following table: • To set whether the Feature Browser should be synchronous or asynchronous	c		e	f	g						l	m	n	
M_GC_FEATURE_POLLING	Sets whether Feature Browser will periodically poll camera features that can automatically change on the camera, for updates (for example, the camera's temperature). The camera's device description file (XML) defines these feature as pollable. To learn whether a camera feature is pollable, refer to your camera's documentation. If this control type is enabled, these features will be polled periodically and Feature Browser (if open either through Aurora Imaging Intellicam or launched using M_GC_FEATURE_BROWSER) will be updated automatically with this information. The polling period is determined by Aurora Imaging Library and throttled with respect to available bandwidth. If this control type is disabled, Feature Browser will display the initial value of the pollable features and not update them until the user clicks upon their corresponding display field in Feature Browser. Note that MdigInquire() and MdigInquireFeature() always return information that comes from the camera at the time of the inquire, regardless of whether feature polling is enabled or not. Note that this control type does not enable execution-polling completion of executable commands. To enable execution-polling completion, use MsysControl() with M_GC_FEATURE_EXECUTE_POLLING_MODE. INQ			e	f	g				j	k	l	m	n	
	System specific														
	Note that, this control type is only available if using a GenCP-compatible camera, or if you first install the third-party, vendor-supplied, standard compliant CLProtocol library for the Camera Link camera connected to your digitizer.									j	k				
M_DEFAULT	Same as M_DISABLE.			e	f	g				j	k	l	m	n	

	M_DISABLE	Specifies that automatically changing camera features will not be polled.			e	f	g					j	k	l	m	n
	M_ENABLE	Specifies that automatically changing camera features will be polled.			e	f	g					j	k	l	m	n
M_GC_FRAME_MAX_RETRIES		Sets the maximum number of times packets should be re-sent before flagging their associated frame as corrupt. Each packet can be re-sent the number of times specified using M_GC_PACKET_MAX_RETRIES. When the total number of packet re-sends is equal to M_GC_FRAME_MAX_RETRIES, the frame is flagged as corrupt (for example, if each packet can be re-sent 3 times and the frame's maximum retries is set to 30, a total of 30 packets associated with the frame can be re-sent before the frame is flagged as corrupt). To establish the number of frames corrupted, use MdigInquire() with M_GC_TOTAL_FRAMES_MISSED, or Aurora Imaging Capture Works. INQ					g									
	M_DEFAULT	Specifies the default value; the default value is 30.					g									
	Value >= M_GC_PACKET_MAX_RETRIES	Specifies the maximum number of times to retry sending the packets of a frame. Note that this value should always be greater than or equal to M_GC_PACKET_MAX_RETRIES.					g									
M_GC_FRAME_TIMEOUT		Sets the maximum amount of time to wait for the remaining packets of a frame, after receiving the trailer packet. If the missing packets are not returned within this time period, the corresponding frame is flagged as corrupt. To establish the number of corrupt frames, use MdigInquire() with M_GC_TOTAL_FRAMES_MISSED, or Aurora Imaging Capture Works. INQ					g									
	M_DEFAULT	Specifies the default value; the default value is 100 msec.					g									
	Value >= 0	Specifies the maximum time to wait, in msec.					g									
M_GC_HEARTBEAT_STATE		Sets whether the heartbeat mechanism is used to keep the GigE Vision-compliant camera active. The heartbeat mechanism is used to inform your camera that communication between your Aurora Imaging Library application and the camera is still open. If your application experiences a fatal error while the heartbeat mechanism is disabled, you will be unable to reconnect to your camera until you manually restart it. You should therefore typically disable the heartbeat mechanism only when required for debugging your application. If you experience timeouts when your application is running under normal circumstances, you should extend the heartbeat timeout period using M_GC_HEARTBEAT_TIMEOUT instead. INQ				f	g									
	M_DEFAULT	Same as M_ENABLE.				f	g									
	M_DISABLE	Specifies that the heartbeat mechanism is disabled. The camera always assumes communication will continue and remains active indefinitely.				f	g									
	M_ENABLE	Specifies that the heartbeat mechanism is enabled. If, after the heartbeat timeout value expires, the camera assumes there will be no further communication and shuts down.				f	g									
M_GC_HEARTBEAT_TIMEOUT		Sets the amount of time that your GigE Vision-compliant camera will wait after the last communication before shutting down. INQ				f	g									
	M_DEFAULT	Specifies the default value; the default value is 5000 msec.				f	g									
	Value >= 0	Specifies the amount of time, in msec.				f	g									

M_GC_INTER_PACKET_DELAY	Sets the delay between packets sent by your camera when transmitting a stream of image packets. The unit used is a timestamp tick. If you know the inter-packet delay in sec, use the following calculation to convert from timestamp ticks to sec: (1/*m*)**n* , where *m* is the frequency of the timestamp ticks (to inquire this value, use MdigInquire() with M_GC_COUNTER_TICK_FREQUENCY), and *n* is your inter-packet delay value. INQ				f	g								
Value >= 0	Specifies the delay, in timestamp ticks.				f	g								
M_GC_LOCAL_MESSAGE_PORT	Sets the UDP port number used for the message channel on the computer connected to your GigE Vision-compliant camera. By default, this port number is set by Aurora Imaging Library. This control type should be changed only if there is a multicast conflict. Note that, with a multicast conflict, the multicast IP address and/or the UDP port assignment can be in conflict. If the problem persists after changing the message port, you can try changing the default multicast IP address, using M_GC_MESSAGE_CHANNEL_MULTICAST_ADDRESS_STRING. This control type is only available when dealing with a multicast controller or monitor (allocated using MdigAlloc() with M_GC_MULTICAST_CONTROLLER or M_GC_MULTICAST_MONITOR). For more information on using a multicast IP address, refer to the Using IP multicast section of the Zebra GigE Vision driver chapter in the Hardware-specific Notes. INQ				f	g								
0 <= *Value* <=65535	Specifies the port number.				f	g								
M_GC_LOCAL_STREAM_PORT	Sets the UDP port number used for the GVSP (GigE Vision Stream Protocol) channel on the computer connected to your GigE Vision-compliant camera. By default, this port number is set by Aurora Imaging Library within the reserved range of multicast port numbers. This control type should be changed only if there is a multicast conflict. Note that, with a multicast conflict, the multicast IP address and/or the UDP port assignment can be in conflict. If the problem persists after changing the stream port, you can try changing the default multicast IP address, using M_GC_STREAM_CHANNEL_MULTICAST_ADDRESS_STRING. This control type is only available when dealing with a multicast controller or monitor (allocated using MdigAlloc() with M_GC_MULTICAST_CONTROLLER or M_GC_MULTICAST_MONITOR). For more information on using a multicast IP address, refer to the Using IP multicast section of the Zebra GigE Vision driver chapter in the Hardware-specific Notes. INQ				f	g								
0 <= *Value* <=65535	Specifies the port number.				f	g								
M_GC_MAX_NBR_PACKETS_OUT_OF_ORDER	Sets the maximum number of packets that can be received out-of-order, before the associated frame is marked as corrupt. This is useful when using link-aggregated cameras, where packets are typically received out-of-order. To establish the number of packets currently received out-of-order, use MdigInquire() with M_GC_MAX_NBR_PACKETS_OUT_OF_ORDER, or Aurora Imaging Capture Works. INQ					g								
M_DEFAULT	Specifies the default value; the default value is 0.					g								
Value >= 0	Specifies the maximum number of out-of-order packets to receive.					g								

M_GC_MESSAGE_CHANNEL_MULTICAST_ADDRESS_STRING	Sets the IP address used for the multicast message channel of your GigE Vision-compliant camera. This IP address is set by Aurora Imaging Library within the reserved range of multicast IP addresses (239.0.0.0 to 239.255.255.255). This control type should be changed only if there is a multicast conflict. Note that, with a multicast conflict, the multicast IP address and/or the UDP port assignment can be in conflict. If the problem persists after changing the multicast IP address, you can try changing the default UDP port assignment, using M_GC_LOCAL_MESSAGE_PORT. This control type is only available when dealing with a multicast controller or monitor (allocated using MdigAlloc() with M_GC_MULTICAST_CONTROLLER or M_GC_MULTICAST_MONITOR). For more information on using a multicast IP address, refer to the Using IP multicast section of the Zebra GigE Vision driver chapter in the Hardware-specific Notes. INQ					g								
AIL_TEXT("*239.n.n.n*") 1	Specifies the multicast IP address of the camera, in dotted decimal notation where each dotted decimal value (*n*) is a number between 0 and 255. Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*239.n.n.n*")).					g								
M_GC_PACKET_MAX_RETRIES	Sets the maximum number of times each packet can be re-sent. If the maximum number of times a packet can be re-sent is reached, the packet is flagged as corrupted. INQ					g								
M_DEFAULT	Specifies the default value; the default value is 3.					g								
Value >= M_GC_FRAME_MAX_RETRIES	Specifies the maximum number of times to resend a given packet of a frame. Note that this value should always be less than M_GC_FRAME_MAX_RETRIES.					g								
M_GC_PACKET_RESEND	Sets whether to request packets be re-sent from your GigE Vision-compliant camera, if the packets are not received properly (for example, when the packets are received out-of-order, or a packet timeout occurs). The request is issued up to a maximum number of times for each packet, set using M_GC_PACKET_MAX_RETRIES. INQ					g								
M_DISABLE	Specifies that packets should never be re-sent.					g								
M_ENABLE	Specifies that packets should be re-sent as required. This is the default value.					g								
M_GC_PACKET_SIZE	Sets the packet size used by the GigE Vision-compliant camera when streaming data to the Host. By default, Zebra's GigE Vision driver negotiates the largest possible packet size with the GigE Vision-compliant camera. This control type should only be changed if the negotiated size is causing errors in acquisition (such as image corruption). INQ				f	g								
Value >= 0	Specifies the packet size, in bytes. Typical packet sizes range from 512 bytes to 1.5 Kbytes. Jumbo packets, if enabled on your network and compatible with both your GigE Vision device and your firewall, are up to 9 Kbytes in size.				f	g								
M_GC_PACKET_TIMEOUT	Sets the maximum amount of time to wait before flagging a packet as dropped. INQ					g								
M_DEFAULT	Specifies the default value; the default value is 10 msec.					g								
Value >= 0	Specifies the maximum time to wait, in msec.					g								

M_GC_PIXEL_FORMAT	Sets the pixel format that the digitizer should use to create internal buffers to receive images from the camera, if the digitizer is a multicast monitor digitizer. The pixel format cannot be inquired from the GigE Vision-compliant camera. It must therefore be set using this control type or a DCF. To set this value to the same pixel format as received image data, use MdigInquire() with M_SOURCE_DATA_FORMAT; in this case, you must first grab into an arbitrary image buffer and then issue the inquire. INQ			e	f	g									b
M_DEFAULT	Same as specified in the DCF.			e	f	g									b
Value	Specifies the data depth and raw color format to use to create the internal buffers. The GenICam predefined list of pixel formats are in the *PFNC.h* header file in the installed aildyn folder (for example, *C:\Program Files\Aurora Imaging Library\11\include\classic\aildyn*).			e	f	g									b
M_GC_PIXEL_FORMAT_SWITCHING	Sets whether to allow the camera's pixel format to change automatically to match the current grab buffer, when supported by the GenICam-compliant camera. INQ			e		g								n	b
M_DEFAULT	Same as M_ENABLE.			e		g								n	b
M_DISABLE	Specifies to disable automatic pixel format switching.			e		g								n	b
M_ENABLE	Specifies to enable automatic pixel format switching.			e		g								n	b
M_GC_STATISTICS_RESET	Resets the acquisition statistics. For a list of GenICam-compliant statistics available in Aurora Imaging Library, refer to MdigInquire() with M_GC_TOTAL_..., or Aurora Imaging Capture Works.				f	g								n	
M_DEFAULT	Implements the default behavior.				f	g								n	
M_GC_STREAM_CHANNEL_MULTICAST_ADDRESS_STRING	Sets the IP address used for the multicast stream channel of your GigE Vision-compliant camera. By default, this address is set by Aurora Imaging Library within a limited range of multicast addresses (239.0.0.0 to 239.255.255.255). This control type should be changed only if there is a multicast conflict. Note that, with a multicast conflict, the multicast IP address and/or the UDP port assignment can be in conflict. If the problem persists after changing the multicast IP address, you can try changing the default UDP port assignment, using M_GC_LOCAL_STREAM_PORT. This control type is only available when dealing with a multicast controller or monitor (allocated using MdigAlloc() with M_GC_MULTICAST_CONTROLLER or M_GC_MULTICAST_MONITOR). For more information on using a multicast IP address, refer to the Using IP multicast section of the Zebra GigE Vision driver chapter in the Hardware-specific Notes. INQ					g									
AIL_TEXT("*2nn.n.n.n*") 1	Specifies the multicast IP address of the camera, in dotted decimal notation where each dotted decimal value (*n*) is a number between 0 and 255. While Aurora Imaging Library assigns an address within a limited range of multicast IP addresses (239.0.0.0 to 239.255.255.255), the full range is available (224.0.0.0 to 239.255.255.255). You can determine the size of this string by calling MdigInquire() with M_GC_STREAM_CHANNEL_MULTICAST_ADDRESS_STRING + M_STRING_SIZE. > Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*2nn.n.n.n*")).					g									

M_GC_STREAMING_MODE	Sets the camera's image stream activation mechanism. Typically, this control type is used when performing a monoshot grab in a loop. Since the camera cannot tell when the next grab might arrive, you can either manually control the image stream (starting it before the first grab and stopping it after the last one), or allow Aurora Imaging Library to handle it automatically (using M_GC_STREAMING_STOP_CHECK_PERIOD and M_GC_STREAMING_STOP_DELAY to control the delay between the grab and when the image stream is stopped). Reducing the time during which images are streamed to the Host can reduce bandwidth requirements. INQ			e	f	g							l	m	n	
M_DEFAULT	Same as M_AUTOMATIC.			e	f	g							l	m	n	
M_AUTOMATIC	Specifies that the image stream is started and stopped automatically. Use M_GC_STREAMING_STOP_CHECK_PERIOD and M_GC_STREAMING_STOP_DELAY to control the delay between the grab and when the image stream is stopped.			e	f	g							l	m	n	
M_MANUAL	Specifies that the image stream is started and stopped manually. Use M_GC_STREAMING_START and M_GC_STREAMING_STOP.			e	f	g							l	m	n	
M_GC_STREAMING_START	Starts the camera's image stream. Note that this control type is only available when M_GC_STREAMING_MODE is set to M_MANUAL.			e	f	g							l	m	n	
M_DEFAULT	Implements the default behavior.			e	f	g							l	m	n	
M_GC_STREAMING_STOP	Stops the camera's image stream. Note that this control type is only available when M_GC_STREAMING_MODE is set to M_MANUAL.			e	f	g							l	m	n	
M_DEFAULT	Implements the default behavior.			e	f	g							l	m	n	
M_GC_STREAMING_STOP_CHECK_PERIOD	Sets the maximum amount of time to wait before Aurora Imaging Library checks to see whether a grab is pending. If there is no grab pending after this delay, a second check is made after a second delay (set using M_GC_STREAMING_STOP_DELAY). If there is still no grab pending, then Aurora Imaging Library stops the digitizer from streaming images. Note that this control type is only available when M_GC_STREAMING_MODE is set to M_AUTOMATIC. INQ			e	f	g							l	m	n	
M_DEFAULT	Specifies the default value; the default value is 1000 msec.			e	f	g							l	m	n	
Value >= 0	Specifies the amount of time, in msec.			e	f	g							l	m	n	
M_GC_STREAMING_STOP_DELAY	Sets the amount of time to wait before stopping the image stream, if no grab is pending. Note that this delay occurs after the first check for pending grabs (set using M_GC_STREAMING_STOP_CHECK_PERIOD). Note that this control type is only available when M_GC_STREAMING_MODE is set to M_AUTOMATIC. INQ			e	f	g							l	m	n	
M_DEFAULT	Specifies the default value; the default value is 1000 msec.			e	f	g							l	m	n	
Value >= 0	Specifies the amount of time, in msec.			e	f	g							l	m	n	

M_GC_UPDATE_MULTICAST_INFO	Updates the multicast information on the controller, worker, or monitor digitizer. The multicast information includes the IP address and port number used for the multicast stream and message channels of your GigE Vision-compliant camera. When used with the multicast controller digitizer (allocated using MdigAlloc() with M_GC_MULTICAST_CONTROLLER), this control type updates the controller digitizer's multicast information after you manually change it on the camera (using M_GC_STREAM_CHANNEL_MULTICAST_ADDRESS_STRING, M_GC_LOCAL_STREAM_PORT, M_GC_MESSAGE_CHANNEL_MULTICAST_ADDRESS_STRING, and/or M_GC_LOCAL_MESSAGE_PORT). When used with a multicast worker digitizer (allocated using MdigAlloc() with M_GC_MULTICAST_WORKER), this control type is used to update the local copy of the worker digitizer's multicast information after the multicast controller digitizer disconnects with the camera and reconnects using a different multicast IP address and/or port number. When used with a multicast monitor digitizer (allocated using MdigAlloc() with M_GC_MULTICAST_MONITOR), this control type is used to update the monitor digitizer's multicast information after you change the multicast stream channel address and port address manually for the digitizer. To verify that the multicast controller is connected, use MdigInquire() with M_GC_MULTICAST_CONTROLLER_CONNECTED. To detect when the image stream stops, check the number of images grabbed with MdigProcess(), using MdigInquire() with M_PROCESS_FRAME_COUNT before and after a wait period. For more information on multicast IP addresses, refer to the Using IP multicast section of the Zebra GigE Vision driver chapter in the Hardware-specific Notes.					g										
M_DEFAULT	Implements the default behavior.					g										
M_GC_USER_NAME	Sets the camera's name. INQ			e	f	g									n	
AIL_TEXT("*CameraName*") 1	Specifies the name of the camera. Note that this string can be 16 characters long (including the null-terminating character) and must use the UTF-8 character set. > Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*CameraName*")).			e	f	g									n	
M_GENTL_ANNOUNCE_BUFFER	Announces the specified buffer to the GenTL producer, before starting the acquisition with MdigGrab(). Note that it is not necessary to announce buffers when using MdigGrab(), MdigGrabContinuous(), or MdigProcess(). If you manually announce the buffer with M_GENTL_ANNOUNCE_BUFFER, you need to revoke the buffer after acquisition has stopped with M_GENTL_REVOKE_BUFFER.			e												
Grab image buffer identifier	Specifies the identifier of a destination image buffer with an M_IMAGE + M_GRAB attribute.			e												
M_GENTL_REVOKE_BUFFER	Revokes a previously announced grab image buffer from the GenTL producer. Note that acquisition must be stopped before a buffer can be revoked.			e												
Grab image buffer identifier	Specifies the identifier of a destination image buffer with an M_IMAGE + M_GRAB attribute.			e												
M_GRAB_ABORT	Immediately stops a grab in progress and aborts queued grabs. This is useful for canceling a triggered grab that is waiting for a trigger.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Note that, for this control type to function on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									

	M_DEFAULT	Implements the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB_DIRECTION_X		Sets the horizontal grab direction. INQ	c		e	f	g			i		j	k	l	m	n	
	M_DEFAULT	Same as M_FORWARD.	c		e	f	g			i		j	k	l	m	n	
	M_FORWARD	Specifies to grab from left to right, in the horizontal direction.	c		e	f	g			i		j	k	l	m	n	
	M_REVERSE	Specifies to grab from right to left, in the horizontal direction. Effectively, this flips the grabbed image horizontally.	c		e	f	g			i		j	k	l	m	n	
M_GRAB_DIRECTION_Y		Sets the vertical grab direction. INQ	c		e	f	g			i		j	k	l	m	n	
	M_DEFAULT	Same as M_FORWARD.	c		e	f	g			i		j	k	l	m	n	
	M_FORWARD	Specifies to grab from top to bottom, in the vertical direction.	c		e	f	g			i		j	k	l	m	n	
	M_REVERSE	Specifies to grab from bottom to top, in the vertical direction. Effectively, this flips the grabbed image vertically.	c		e	f	g			i		j	k	l	m	n	
M_GRAB_FIELD_NUM		Sets the number of fields to grab when grabbing data with MdigGrab(). This control type should only be used with interlaced cameras. When using progressive cameras, set this control type to 1. INQ	c														
	M_DEFAULT	Specifies the default value. For interlaced cameras, the default is 2. For progressive cameras, the default is 1.	c														
	1	Specifies to grab one field. The grabbed field is written to sequential rows of the grab buffer. When set to 1, each field is treated like a frame and the following digitizer events occur relative to the grabbed field: M_GRAB_FRAME_START, M_GRAB_END, and M_GRAB_FRAME_END. To achieve 60 fps in NTSC or 50 fps in PAL, M_GRAB_START_MODE must be set to M_FIELD_START.	c														
	2	Specifies to grab two fields.	c														
M_GRAB_FRAME_MISSED_COUNTER		Sets whether to count the number of frames sent by the camera, but not received by the digitizer when performing a grab operation (that is, MdigGrab(), MdigGrabContinuous(), or MdigProcess()). To inquire the current value of this counter, use MdigInquire() with M_GRAB_FRAME_MISSED. INQ				f						j	k	l	m		
	M_DEFAULT	Same as M_DISABLE.				f						j	k	l	m		
	M_DISABLE	Specifies to disable frames missed detection.				f						j	k	l	m		
	M_ENABLE	Specifies to enable frames missed detection.				f						j	k	l	m		
M_GRAB_FRAME_MISSED_RESET		Resets the counter for the number of frames sent by the camera but not received by the digitizer.				f						j	k	l	m		
	M_DEFAULT	Implements the default behavior.				f						j	k	l	m		

M_GRAB_LINE_COUNTER	Sets whether a function hooked to an M_GRAB_END, M_ROTARY_ENCODER, or M_GRAB_FRAME_END event can inquire the number of lines grabbed, performed using MdigGrab(), MdigGrabContinuous(), or MdigProcess(); the function could have been hooked using either MdigProcess() or MdigHookFunction(). To inquire the number of lines grabbed at any other time, use MdigGetHookInfo() with M_GRAB_LINE_COUNT. INQ										j	k	l	m		
M_DEFAULT	Same as M_DISABLE.										j	k	l	m		
M_DISABLE	Specifies that the hooked function cannot inquire the number of lines grabbed.										j	k	l	m		
M_ENABLE	Specifies that the hooked function can inquire the number of lines grabbed.										j	k	l	m		
M_GRAB_LUT_PALETTE	Sets which LUT to use. Note that this control value is only supported on color cameras. INQ								i							
M_DEFAULT	Specifies to use a 3-band RGB LUT.								i							
M_GAMMA	Specifies to use the native Bayer LUT that is applied to the Bayer image before it is converted to an RGB image.								i							
M_GRAB_MODE	Sets how the grab should be synchronized with the Host when grabbing data with MdigGrab(). Note that this does not affect MdigGrabContinuous(), which always runs asynchronously; otherwise, MdigHalt() could not stop the grab. In addition, this control type does not affect MdigProcess(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Note that, for this control type to function on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									
M_DEFAULT	Same as M_SYNCHRONOUS.	c		e	f	g	a		i		j	k	l	m	n	b
M_ASYNCHRONOUS	Specifies that your application continues after one grab is queued, rather than waiting for the grab to finish. Note that only one MdigGrab() function can be queued at a time. If one MdigGrab() is already queued, subsequent grabs will cause your application to wait until the current grab is finished. This allows the currently queued grab to start.	c		e	f	g	a		i		j	k	l	m	n	b
M_ASYNCHRONOUS_QUEUED	Specifies that your application continues after each grab is queued, rather than waiting for the grab to finish. Note that more than one MdigGrab() function can be queued. This causes the grabs to be truly asynchronous.	c		e	f	g	a		i		j	k	l	m	n	b
M_SYNCHRONOUS	Specifies that your application is synchronized with the end of a grab operation (that is, your application waits for the grab to finish before returning from the grab function).	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB_SCALE	Sets the vertical and horizontal scaling factor when grabbing data with MdigGrab(), MdigGrabContinuous(), or MdigProcess(). If the scale entered is not supported, the closest scaling factor will be used instead. If, however, the scale entered is larger than the maximum value supported, an error is generated. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	The camera must support the ability to scale grabbed data for this control type to be available on the camera. This control type is also available on the board.					g									n	

	Note that, for this control type to function on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									
	Note that this control type will subsample 2 adjacent pixels and 2 adjacent lines.								i							
M_FILL_DESTINATION	Specifies that the scaling factor is calculated and set to fill the destination buffer.	c			f						j	k	l	m		
	System specific															
	Note that this control value uses software scaling rather than hardware scaling.	c														
1	Specifies that no scaling is applied.			e	f	g	a		i		j	k	l	m	n	b
Value = 1/*n*	Specifies to reduce the image size. To reduce the image size, the specified value must always be less than 1.0.	c		e	f		a		i		j	k	l	m		b
	System specific															
	The value of *n* can be any integer between 1 and 16.	c			f						j	k	l	m		
	The value of *n* can be 1 or 2.								i							
	Note that data grabbed into a YUV buffer can only have a ½ scaling factor.	c		e					i							b
M_GRAB_SCALE_INTERPOLATION_MODE	Sets the interpolation mode used when performing a vertical and/or horizontal scaling. Note that, for this control type to function on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED). INQ						a									
M_DEFAULT	Same as M_NEAREST_NEIGHBOR.						a									
M_AUTOMATIC	Specifies that the best interpolation mode is automatically selected, based on the input filter (set using M_INPUT_FILTER).						a									
M_BICUBIC	Specifies bicubic interpolation. Note that the sum of the weights used for bicubic interpolation might be greater than one. If this occurs and the result reflects an overflow or underflow, the result is saturated according to the depth and data type of the destination buffer. This interpolation mode is software-based.						a									
M_BILINEAR	Specifies bilinear interpolation. This interpolation mode is software-based.						a									
M_NEAREST_NEIGHBOR	Specifies nearest neighbor interpolation. By default, nearest neighbor interpolation is software-based.						a									
M_GRAB_SCALE_X	Sets the horizontal scaling factor when grabbing data. If the scale entered is not supported, the closest scaling factor will be used instead. If, however, the scale entered is larger than the maximum value supported, an error is generated. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	The camera must support the ability to scale grabbed data for this control type to be available on the camera. This control type is also available on the board.					g									n	

	Note that, for this control type to function on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									
M_FILL_DESTINATION	Specifies that the scaling factor is calculated and set to fill the destination buffer. INFO	c			f					j	k	l	m			
1	Specifies that no scaling is applied.			e	f	g	a		i		j	k	l	m	n	b
Value = 1/n	Specifies to reduce the image size. INFO	c		e	f		a		i		j	k	l	m		b
M_GRAB_SCALE_Y	Sets the vertical scaling factor when grabbing data. If the scale entered is not supported, the closest scaling factor will be used instead. If, however, the scale entered is larger than the maximum value supported, an error is generated. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	The camera must support the ability to scale grabbed data for this control type to be available on the camera. This control type is also available on the board.					g									n	
	When down-scaling the image width, a smaller image will result but its scan rate will be proportionally higher than if the image is not scaled.								i							
	Note that, for this control type to function on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									
M_FILL_DESTINATION	Specifies that the scaling factor is calculated and set to fill the destination buffer. INFO	c			f					j	k	l	m			
1	Specifies that no scaling is applied.			e	f	g	a		i		j	k	l	m	n	b
Value = 1/n	See the description of this setting in M_GRAB_SCALE.	c		e	f		a		i		j	k	l	m		b
	System specific															
	Note that the restriction of using a ½ scaling factor when grabbing into a YUV buffer does not apply.			e	f				i		j	k	l	m		
M_GRAB_START_MODE	Sets the type of field on which to grab. Note that this value applies only when dealing with interlaced cameras. INQ	c									j	k				
M_DEFAULT	Same as M_FIELD_START_ODD.	c									j	k				
M_FIELD_START	Specifies that the grab starts on any field.										j	k				
M_FIELD_START_EVEN	Specifies that the grab starts on an even field.	c									j	k				
M_FIELD_START_ODD	Specifies that the grab starts on an odd field.	c									j	k				
M_GRAB_TIMEOUT	Sets the maximum time to wait for a frame before generating an error. Note that when used with MdigProcess(), M_GRAB_TIMEOUT sets the time to wait before checking whether a grab occurred. If no grab occurred within this time period, a second time period of the same duration might elapse before an error is generated. INQ	c		e	f	g			i		j	k	l	m	n	b
M_DEFAULT	Specifies the default amount of time to wait. Note that this is the same as M_INFINITE when performing a triggered grab.	c		e	f	g			i		j	k	l	m	n	b

		System specific														
		The amount of time is either 5 sec or 5 times the frame period, which ever is higher.		e	f	g				j	k	l	m	n	b
		The amount of time is 200 times the frame period.							i							
		The amount of time is 5000 msec.	c													
	M_INFINITE	Specifies to wait indefinitely. This is recommended only for triggered grabs.	c		e	f	g		i		j	k	l	m	n	b
	Value > 0	Specifies the time to wait, in msec.	c		e	f	g		i		j	k	l	m	n	b
M_HARDWARE_DEINTERLACING		Sets whether hardware deinterlacing is used when the video source is interlaced. Note that this control type is only available when your DCF uses an interlaced video signal. INQ	c													
	M_DEFAULT	Same as M_DISABLE.	c													
	M_BOB_METHOD	Specifies to use the BOB algorithm for hardware deinterlacing.	c													
	M_DISABLE	Specifies that hardware deinterlacing is disabled.	c													
	M_MADI_METHOD	Specifies to use the MADI (motion adaptive deinterlacing) algorithm for hardware deinterlacing.	c													
M_INPUT_FILTER		Sets the low-pass filter applied to incoming data. INQ	c													
	M_DEFAULT	Same as M_BYPASS.	c													
	M_AUTOMATIC	Specifies that the best hardware filter is automatically selected or the filter is bypassed when the video input is interlaced and hardware deinterlacing is disabled.	c													
	M_BYPASS	Specifies to not use a filter.	c													
	M_LOW_PASS_0	Specifies to use the first low-pass filter. Uses a Kaiser filter.	c													
	M_LOW_PASS_1	Specifies to use the second low-pass filter. Uses a Gaussian filter.	c													
M_LAST_GRAB_IN_TRUE_BUFFER		Sets whether a monoshot grab is performed when MdigHalt() is called after performing a continuous grab operation. This control type only has an effect if the target image buffer of the continuous grab is selected on a display. When the target buffer is selected on the display, the continuous grab will typically grab into a temporary internal buffer and only update the target image buffer when MdigHalt() is called. This control type establishes how to update to the target image buffer. INQ	c		e	f	g	a		i		j	k	l	m	n	b
		System specific														
		Note that, for this control type to function on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a								
	M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies that the last frame grabbed during the continuous grab operation is copied to the target image buffer, instead of performing one last grab. This setting might reduce the time it takes for the continuous grab to terminate once MdigHalt() is called (since copying a frame tends to take less time than grabbing a frame).	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that a monoshot grab is performed. Note that after performing a monoshot grab in the target image buffer selected to the display, the display is updated.	c		e	f	g	a		i		j	k	l	m	n	b
M_LIGHTING_BRIGHT_FIELD		Sets the intensity of light to emit parallel to the optical axis, making the flat parts of an object stand out in stark contrast in the image. Increasing the value of this control type increases the intensity of the light transmitted by your lighting controller. Note that this control type is only for controlling the intensity of an LED (as a signal passed to a lighting controller). To control the timing of when the strobe light goes on or off, use M_GRAB_TRIGGER... + M_TIMER_STROBE or M_TIMER_TRIGGER_SOURCE with M_TIMER_STROBE. For more information, refer to your board's Aurora Imaging Library Hardware-specific Notes. Note that some consecutive bright field intensity level settings might produce the same result due to the fact that there are only 250 distinct adjustments (adjustments of 0.4% each). INQ								i							
	M_DEFAULT	Specifies the default value; the default value is 255.								i							
	0 <= *Value* <= 255	Specifies the intensity of light to be emitted on the flat parts of the object.								i							
M_LUT_ID		Sets whether to map the input data through the physical LUT of the specified digitizer and the values with which to initialize the LUT. Aurora Imaging Library uses the data format (DCF) of the digitizer to determine whether a physical LUT is supported. If it is not, and you try to initialize it, an error is generated. To learn how the digitizer's physical LUT is configured, refer to Mapping grabbed data through a LUT. If the destination grab buffer's depth is larger than that of the digitizer's physical LUT, the bits from the physical LUT are copied over and the remaining upper bits in the destination grab buffer are set to 0. If the digitizer's physical LUT depth is greater than that of the destination host grab buffer, the most-significant bits of the data (the non-zero values) are used when the data is grabbed. INQ						a		i		j	k	l	m		
		System specific															
		Note that if 14-, or 16-bit data is grabbed, the digitizer's physical LUT is not used.												l	m		
		Note that, for this control type to function on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									
	M_DEFAULT	Specifies that the digitizer's physical LUT is not used.						a		i		j	k	l	m		
	LUT buffer identifier	Specifies the identifier of the LUT buffer (allocated with MbufAlloc...() with M_LUT) with which to initialize the digitizer's physical LUT. To copy the data from a LUT buffer to the digitizer's physical LUT, the number of entries in the LUT buffer must match those of the digitizer's physical LUT. Note that if the digitizer's physical LUT cannot support the depth of the specified LUT buffer, an error will occur. LUT buffer data is loaded into the digitizer's physical LUT as follows. In the case where a 1-band LUT buffer is loaded into a 3-band digitizer's physical LUT, the LUT buffer is copied into each component of the digitizer's physical LUT. When a 3-band LUT buffer is loaded into a 1-band digitizer's physical LUT, only the first band of the LUT buffer is copied into the digitizer's physical LUT. In all other cases, each band of the LUT buffer is copied into the associated band of the digitizer's physical LUT.						a		i		j	k	l	m		
		System specific															

	When grabbing 8-bit data, the digitizer's physical LUT has 256 entries.						a					l	m			
	Zebra Iris GTX only supports an 8- or 10-bit LUT buffer.								i							
	When grabbing 10-bit data, the digitizer's physical LUT has 1024 entries. When grabbing 12-bit data, the digitizer's physical LUT has 4096 entries.						a					l	m			
	Zebra Rapixo CXP and Zebra Rapixo CoF on-board programmable LUTs can map 8-bit, 10-bit, and 12-bit (monochrome or color). When a link is receiving color data, all bands of the data use the same specified LUT mapping. As soon as one link is receiving 12-bit data, all links (CoaXPress connections) share the same specified LUT mapping. Data of other depths are mapped through transparent LUTs.											l	m			
M_POWER_OVER_CABLE	Sets whether to enable PoCL (power over Camera Link) and PoCXP (power over CoaXPress). INQ										j	k		m		
	System specific															
	Note that the camera must be PoCL-capable. Once enabled, your PoCL-capable camera receives power over its Camera Link connection.										j	k				
	Note that the camera must be PoCXP-capable. Once enabled, your PoCXP-capable camera receives power over its CoaXPress connection.													m		
M_DEFAULT	Same as M_ON.										j	k		m		
M_OFF	Specifies to disable PoCL/PoCXP.										j	k		m		
M_ON	Specifies to enable PoCL/PoCXP.										j	k		m		
M_PROCESS_GRAB_MONITOR	Sets whether to create an internal grab monitoring thread when using MdigProcess() with either M_START or M_SEQUENCE. The internal grab monitoring thread will produce an error if there is no longer any grab activity (for example, when a camera is unplugged) after M_PROCESS_TIMEOUT expires. A typical use of this control type would be to disable the grab monitoring thread before a camera is unplugged, so that an error is not generated. INQ	c		e	f	g			i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g			i		j	k	l	m	n	b
M_DISABLE	Specifies not to create a grab monitoring thread.	c		e	f	g			i		j	k	l	m	n	b
M_ENABLE	Specifies to create a grab monitoring thread.	c		e	f	g			i		j	k	l	m	n	b
M_PROCESS_TIMEOUT	Sets the maximum amount of time to wait for MdigProcess() to complete the current grab (M_STOP) or all the queued grabs (M_STOP + M_WAIT), before generating an error. In an ideal situation, MdigProcess() would complete all the queued grabs before this timeout expires. If the timeout does expire either the specified time was too short, or a grab error occurred. INQ	c		e	f	g			i		j	k	l	m	n	b
M_DEFAULT	Same as M_INFINITE.	c		e	f	g			i		j	k	l	m	n	b
M_INFINITE	Specifies to wait indefinitely.	c		e	f	g			i		j	k	l	m	n	b

Value >= 0	Specifies the time to wait, in msec.	c		e	f	g			i		j	k	l	m	n	b
M_SOURCE_OFFSET_X	Sets the X-offset of the input signal capture window. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Note that, for this control type to function on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									
	Note that this value must be multiple of 32.								i							
Value	Specifies the X-offset, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOURCE_OFFSET_Y	Sets the Y-offset of the input signal capture window. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Note that, for this control type to function on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									
	Note that this value must be multiple of 2.								i							
Value	Specifies the Y-offset, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOURCE_SIZE_X	Sets the width of the input signal capture window. Note that certain source sizes, offsets, and destination buffer sizes can affect a grab. Refer to MdigGrab() and MdigGrabContinuous() for more information. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	On some cameras, you might not be able to set the width (that is, this information might be read-only).			e	f	g									n	b
	Note that, for this control type to function on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									
	Note that this value must be multiple of 32.								i							
Value	Specifies the width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOURCE_SIZE_Y	Sets the height of the input signal capture window. Note that certain source sizes, offsets, and destination buffer sizes can affect a grab. Refer to MdigGrab() and MdigGrabContinuous() for more information. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	On some cameras, you might not be able to set the height (that is, this information might be read-only).			e	f	g									n	b
	Note that, for this control type to function on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									
	Note that this value must be multiple of 2.								i							
Value	Specifies the height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

M_TL_TRIGGER_ACTIVATION	Sets the signal variation upon which to generate a trigger signal to the camera through the transport layer interface. INQ											l	m			
M_DEFAULT	Specifies to use the camera's default value or, if supported, to enable the camera's auto control state for this feature(that is, the camera adjusts this value automatically).											l	m			
M_ANY_EDGE	Specifies that a trigger will be generated both upon a high-to-low and a low-to-high signal transition.											l	m			
M_EDGE_RISING	Specifies that a trigger will be generated upon a low-to-high signal transition.											l	m			
M_WHITE_BALANCE	Sets whether to perform white balancing. The white balance coefficients used for white balancing are taken from an internal buffer, which is created and updated when you specify coefficients (using **MdigControl**() with M_BAYER_COEFFICIENTS_ID) or when you let the driver calculate them (using this control type with M_CALCULATE). Note that to use this control type, M_BAYER_CONVERSION must be enabled. INQ		e	f	g			i		j	k	l	m	n	b
	System specific														
	Note that this control type is used to perform Bayer conversion if performed by the Host. To control the Bayer conversion on the camera, use MdigControlFeature(); refer to your camera's documentation for details regarding the features to set.		e		g									n	b
	Note that this control type is only available when using color versions of Zebra Iris GTX (such as 2000C).							i							
M_DEFAULT	Same as M_DISABLE.											l	m			
M_CALCULATE	Calculates new white balance coefficients, overwriting old coefficients. Calling **MdigControl**() with M_WHITE_BALANCE set to M_CALCULATE grabs an image. The image is assumed to be a uniform shade of gray that is not saturated. It is used to calculate the new white balance coefficients. Use MdigInquire() with M_BAYER_COEFFICIENTS_ID to determine the calculated values.		e	f	g			i		j	k	l	m	n	b
	System specific														
	If the digitizer is configured to perform a software triggered grab, calling **MdigControl**() with this value will automatically trigger a grab. For more information, see White balancing your Bayer images.		e		g			i						n	b
M_DISABLE	Specifies that white balancing is disabled.		e	f	g			i		j	k	l	m	n	b
M_ENABLE	Specifies that white balancing is enabled. If you have not called **MdigControl**() with M_BAYER_COEFFICIENTS_ID or with M_WHITE_BALANCE set to M_CALCULATE before enabling white balancing, it will automatically calculate the white balance coefficients and then use those coefficients for subsequent images. Note that if M_WHITE_BALANCE is disabled and then re-enabled, the previously calculated coefficients are used.		e	f	g			i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Combination values for M_GC_FEATURE_BROWSER.

You can add one of the following values to the above-mentioned value to specify which GenTL configuration file (XML file) is associated with the feature.

m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the configuration file associated with the feature																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_GENTL_STREAM_NUMBER(AIL_INT IndexValue)	Specifies the feature is associated with an instance of the GenTL stream configuration information.			e												
	Parameters															
	IndexValue Specifies the instance (index number) of the GenTL stream library associated with the feature.			e												
	0 <= *Value* <= M_GENTL_STREAM_COUNT Specifies the index.			e												
M_GENTL_DEVICE	Specifies that the feature is associated with the GenTL device configuration information.			e												
M_GENTL_REMOTE_DEVICE	Specifies that the feature is associated with the GenTL remote device configuration information. This is the default value.			e												

Combination value for M_FOCUS.

You can add the following value to the above-mentioned value to specify to wait for the camera to focus.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to wait for the camera to perform its focusing operation.																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_WAIT	Waits for the amount of time it takes the camera's lens motor to move the lens of the camera to a position that produces optimum focus. When used, this combination constant delays the callback function (hook-handler function) of MdigFocus() a predetermined amount of time.								i							

Combination values for [when ControlType=M_GC_FEATURE_BROWSER: M_OPEN].

You must add one of the following values to the above-mentioned value to set whether the Feature Browser should be synchronous or asynchronous.

Specifying whether Feature Browser should be synchronous or asynchronous.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether Feature Browser should be synchronous or asynchronous																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ASYNCHRONOUS	Specifies that this function returns immediately once a Feature Browser window opens.	c		e	f	g							l	m	n	
M_SYNCHRONOUS	Specifies that this function is blocked until a Feature Browser window closes. This is the default value.	c		e	f	g							l	m	n	

The following control types and control values set (if available) the reference levels used to digitize the analog signal received from a camera. These control types are specific to analog input devices. Depending on the type of digitizer and input signal, some reference types are not applicable. The smallest voltage increment supported by your board can differ such that consecutive reference-level settings might produce the same result. Note, some digitizers might take a few milliseconds before the reference level stabilizes.

To inquire the maximum and minimum levels for your camera, set the reference level to M_MIN_LEVEL or M_MAX_LEVEL, and then inquire the reference level (for example, set M_BRIGHTNESS_REF to M_MIN_LEVEL, and then use MdigInquire() with M_BRIGHTNESS_REF to return the minimum level for the black reference level).

[Zebra GenTL driver; Zebra GigE Vision driver; Zebra USB3 Vision driver; Video4Linux2]

Note that the following reference types are only available if they are supported by the camera. Refer to your camera's documentation for more details.

[Zebra Clarity UHD]

Note that the following control types are only supported when the device number of the specifies digitizer is M_DEV2 or M_DEV3. These control types are supported when the DCF specifies to grab either an analog or digital input signal.

[Zebra Clarity UHD; Zebra GenTL driver; Zebra GigE Vision driver; Zebra USB3 Vision driver; Video4Linux2]

Note that the following control types are only available when the DCF specifies to grab an analog input signal.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the general reference settings																	
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																	
M_BLACK_OFFSET	Sets the offset to the black level of the image in hardware before the image reaches the digitization module of the sensor. INQ																
0<= *Value* <= 255	Specifies the value of the black offset.								i								
M_BLACK_REF +	Sets the input signal's digitization black reference level. Note that when setting black and white reference levels, always set black reference levels first. INQ			e	f	g									n		
M_AUTOMATIC	Sets the reference level automatically.			e	f	g									n		
Min. black level <= *Value* <= *Max. black level*	Specifies the black reference level. Note that to determine the camera-specific minimum and maximum black reference level, use MdigInquire() with M_BLACK_REF + M_MAX_VALUE or + M_MIN_VALUE, respectively.			e	f	g									n		
M_BRIGHTNESS_REF	Sets the brightness level for composite input signals. INQ	c															
M_DEFAULT	Specifies the default reference level for the specified digitizer data format.	c															
M_MAX_LEVEL	Specifies the maximum value.	c															
M_MIN_LEVEL	Specifies the minimum value.	c															
M_MIN_LEVEL <= *Value* <= M_MAX_LEVEL	Specifies the brightness reference level.	c															
M_CONTRAST_REF	Sets the contrast level for composite input signals. INQ	c															
M_DEFAULT	Specifies the default reference level for the specified digitizer data format.	c															
M_MAX_LEVEL	Specifies the maximum value.	c															
M_MIN_LEVEL	Specifies the minimum value.	c															
M_MIN_LEVEL <= *Value* <= M_MAX_LEVEL	Specifies the contrast reference level.	c															
M_HUE_REF	Sets the hue level for composite input signals. M_HUE_REF is available when grabbing color data (not monochrome data). INQ	c															

	M_DEFAULT	Specifies the default reference level for the specified digitizer data format.	c														
	M_MAX_LEVEL	Specifies the maximum value.	c														
	M_MIN_LEVEL	Specifies the minimum value.	c														
	M_MIN_LEVEL <= *Value* <= M_MAX_LEVEL	Specifies the hue level.	c														
M_SATURATION_REF		Sets the saturation level for composite input signals. M_SATURATION_REF is available when grabbing color data (not monochrome data). INQ	c														
	M_DEFAULT	Specifies the default reference level for the specified digitizer data format.	c														
	M_MAX_LEVEL	Specifies the maximum value.	c														
	M_MIN_LEVEL	Specifies the minimum value.	c														
	M_MIN_LEVEL <= *Value* <= M_MAX_LEVEL	Specifies the saturation level.	c														
M_WHITE_REF +		Sets the input signal's digitization white reference level. Note that when setting black and white reference levels, always set black reference levels first. INQ			e	f	g									n	
	M_DEFAULT	Specifies the default reference level for the specified digitizer data format.			e	f	g									n	
	M_AUTOMATIC	Sets the reference level automatically.			e	f	g									n	
	Min. white level <= *Value* <= *Max. white level*	Specifies the white reference level. Note that to determine the camera-specific minimum and maximum white reference level, use MdigInquire() with M_WHITE_REF + M_MAX_VALUE or + M_MIN_VALUE, respectively.			e	f	g									n	

Combination values for M_BLACK_REF; M_WHITE_REF.

You can add one of the following values to the above-mentioned values to set the reference level of a specific acquisition path when using RGB or multi-tap input.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling the reference level of a specific acquisition path																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALL_REF	Sets the reference level on all available acquisition paths.			e	f	g									n	
M_CH0_REF	Sets the reference level on acquisition path 0.			e	f	g									n	
M_CH1_REF	Sets the reference level on acquisition path 1.			e	f	g									n	
M_CH2_REF	Sets the reference level on acquisition path 2.			e	f	g									n	

The following control types allow you to control the input gain and shading correction.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling the input gain and shading correction

ControlType	ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_GAIN		Sets the input gain with which to amplify the input signal. Note that gain affects the acquisition path before the data is digitized, and is applied to all the color bands of the image. INQ															
	M_DEFAULT	Specifies to use the camera's default value or, if supported, to enable the camera's auto control state for this feature (that is, the camera adjusts this value automatically).			e	f	g			i						n	b
	Min. gain <= Value <= Max. gain	Specifies a value between the minimum and maximum values supported by the camera. Use MdigInquire() to determine these values.			e	f	g			i						n	b
M_GRAB_AUTOMATIC_INPUT_GAIN		Sets whether the input gain should be automatically set. INQ	c		e	f	g									n	
		System specific															
		This control type is only supported when the device number of the specified digitizer is M_DEV2 or M_DEV3. Note that this control type is available when the DCF specifies to grab either an analog or digital input signal.	c														
	M_DEFAULT	Same as M_ENABLE.	c		e	f	g									n	
	M_DISABLE	Specifies that the input gain is not automatically set.	c		e	f	g									n	
	M_ENABLE	Specifies that the input gain is automatically set.	c		e	f	g									n	
M_GRAB_INPUT_GAIN		Sets the input gain with which to amplify the input signal. This allows you to optimize the amplitude of the analog video input signal so that the full dynamic range of the component digitizing the video is used. To automatically set your input gain, use M_GRAB_AUTOMATIC_INPUT_GAIN. To manually set your input gain, M_GRAB_AUTOMATIC_INPUT_GAIN must first be disabled. This control type is only supported when the device number of the specified digitizer is M_DEV2 or M_DEV3. Note that this control type is available when the DCF specifies to grab either an analog or digital input signal. INQ	c														
	M_DEFAULT	Specifies the default input gain factor. When dealing with a composite or RGB analog input signal, the input gain factor is 128.	c														
	M_MAX_LEVEL	Specifies the maximum input gain factor.	c														
	M_MIN_LEVEL	Specifies the minimum input gain factor.	c														
	0 <= *Value* <= 255	Specifies an integer value which is used to determine the input gain factor. With this range, 0 is mapped to the minimum gain factor supported by the camera, while 255 is mapped to the maximum gain factor supported by the camera. This value is supported only when using a DCF that uses either a composite or RGB analog video signal.	c														

M_SHADING_CORRECTION	Sets whether to perform a gain and offset correction per pixel. This shading correction operation is performed by the digitizer when transferring grabbed data to the Host, using the following equation: (*Grab Image* x M_SHADING_CORRECTION_GAIN_ID) + M_SHADING_CORRECTION_OFFSET_ID Each row of gain and offset values is applied to the equivalent row of the grabbed image. To set the gain and offset values for shading correction, use M_SHADING_CORRECTION_GAIN_ID and M_SHADING_CORRECTION_OFFSET_ID, respectively. INQ									j	k	l	m		
	System specific															
	Note that this control type is only available on Zebra Rapixo CXP Pro and Zebra Rapixo CoF 100, and only if the loaded FPGA configuration contains the Zebra PU that supports this functionality.											l	m		
M_DEFAULT	Same as M_DISABLE.									j	k	l	m		
M_DISABLE	Disables shading correction.									j	k	l	m		
M_ENABLE	Enables shading correction.									j	k	l	m		
M_SHADING_CORRECTION_GAIN_FIXED_POINT	Sets the number of bits in the fractional part of the gain values specified for performing shading correction, where values are in fixed point format. The image resulting from the shading correction will be rounded to the nearest whole number. To enable on-board shading correction and provide gain values, use M_SHADING_CORRECTION and M_SHADING_CORRECTION_GAIN_ID, respectively. INQ									j	k	l	m		
	System specific															
	Note that this control type is only available on Zebra Rapixo CXP Pro and Zebra Rapixo CoF 100, and only if the loaded FPGA configuration contains the Zebra PU that supports this functionality.											l	m		
M_DEFAULT	Specifies the default value. For an 8-bit buffer, the default value is 8. For a 16-bit buffer, the default value is 14.									j	k	l	m		
0 < *Value* <= 16	Specifies the number of bits in the fractional part of the fixed-point gain values.									j	k	l	m		
M_SHADING_CORRECTION_GAIN_ID	Sets the buffer containing the gain values that the digitizer should use when it performs shading correction. This is interpreted as a fixed point buffer. To enable on-board shading correction and provide an offset, use M_SHADING_CORRECTION and M_SHADING_CORRECTION_OFFSET_ID, respectively. INQ									j	k	l	m		
	System specific															
	Note that this control type is only available on Zebra Rapixo CXP Pro and Zebra Rapixo CoF 100, and only if the loaded FPGA configuration contains the Zebra PU that supports this functionality.											l	m		
M_DEFAULT	Same as M_NULL.									j	k	l	m		
M_NULL	Specifies not to apply a gain correction when performing shading correction.									j	k	l	m		

	ShadingCorrectionGainID	Specifies the identifier of an M_IMAGE buffer containing the gain values. The buffer must be a single-band, unsigned 8-bit or 16-bit buffer. It is interpreted as a fixed point buffer with 2-bits representing the mantissa (significant bits). Use M_SHADING_CORRECTION_GAIN_FIXED_POINT to set the number of bits for the fractional part. The buffer can have a maximum X-size of 16384 pixels and a Y-size equal to the size of the DCF.									j	k	l	m		
M_SHADING_CORRECTION_OFFSET_ID		Sets the buffer containing the offset values that the digitizer should use when it performs shading correction. To enable on-board shading correction and provide a gain, use M_SHADING_CORRECTION and M_SHADING_CORRECTION_GAIN_ID, respectively. INQ									j	k	l	m		
		System specific														
		Note that this control type is only available on Zebra Rapixo CXP Pro and Zebra Rapixo CoF 100, and only if the loaded FPGA configuration contains the Zebra PU that supports this functionality.											l	m		
	M_DEFAULT	Same as M_NULL.									j	k	l	m		
	M_NULL	Specifies not to apply an offset correction when performing shading correction.									j	k	l	m		
	ShadingCorrectionOffsetID	Specifies the identifier of an M_IMAGE buffer containing the offset values. The buffer must be a single-band, unsigned 8-bit or 16-bit buffer. It can have a maximum X-size of 16384 pixels.									j	k	l	m		

The following control types allow you to set the mode and the purpose of your Zebra imaging board's I/O signals (such as, auxiliary, Camera Link control, and transport layer trigger signal). Once the format, routing, and mode are determined for an I/O signal, you can further control the I/O signal using the control types described in the following tables: For setting the state of specified user-bits in a static-user-output register, For controlling the settings of a timer, For controlling the camera's exposure, and For controlling the settings to grab using a trigger.

> Note that for all Aurora Imaging Library supported hardware that have I/O signals, but are not supported with the constants below, see MsysControl().

Each Zebra imaging board and Aurora Imaging Library driver has its own list of limitations regarding the signals you can control with this function. While general limitations are listed in the table below, for a complete list of the available signals and their limitations, see the Connectors and signal names section of the Aurora Imaging Library Hardware-specific Notes chapter for your Zebra imaging board or Aurora Imaging Library driver.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For routing I/O signals and setting their mode																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_IO_DEBOUNCE_TIME +		Sets the amount of time that the specified auxiliary input signal is debounced. A maximum of 4 inputs can have a debounce set. INQ You must specify a combination value from the following table: • To set the type and number of the I/O signal to affect															
	0 <= *Value* <= 8300000	Specifies the minimum amount of time to ignore any additional signal transitions after accepting a signal transition, in nsec.										j	k	l	m		

M_IO_FORMAT +	Sets whether to enable a specific transmitter/receiver for an I/O signal, on systems whose transmitters/receivers are enabled through software and where the option of two or more signal formats are possible. Note that some signals cannot be affected individually. In this case, if you affect any one of the signals in the group, they will all be affected. For more information, refer to the Technical information appendix in the Installation and Hardware Reference manual for your Zebra imaging board. INQ You must specify a combination value from the following table: • To set the type and number of the I/O signal to affect			e		g							l	m	n	
M_DISABLE	Specifies that the specified I/O signal is not available for use.			e		g									n	
M_LINK_SIGNAL	Specifies to use the transport layer link trigger for the specified I/O signal.												l	m		
M_LVDS	Specifies to use the LVDS transmitter/receiver for the specified I/O signal.			e		g							l	m	n	
M_OPEN_DRAIN	Specifies to use the open collector (open drain) transmitter/receiver for the specified I/O signal.			e		g									n	
M_OPTO	Specifies to use the opto-coupled transmitter/receiver for the specified I/O signal.			e		g							l	m	n	
M_RS422	Specifies to use the RS-422 transmitter/receiver for the specified I/O signal.			e		g									n	
M_TRI_STATE	Specifies to use the tri-state transmitter/receiver for the specified I/O signal.			e		g									n	
M_TTL	Specifies to use the TTL transmitter/receiver for the specified I/O signal.			e		g							l	m	n	
M_IO_INTERRUPT_ACTIVATION +	Sets the signal transition upon which to generate an interrupt, if interrupt generation has been enabled for the specified I/O signal. Use M_IO_INTERRUPT_STATE to enable interrupt generation. Note that this only applies to input signals. Note that this control type only has an effect when M_IO_INTERRUPT_STATE is enabled. INQ You must specify a combination value from the following table: • To set the type and number of the I/O signal to affect			e		g				j	k	l	m	n	
M_DEFAULT	Same as the signal transition specified by the DCF. If this value is not specified in the DCF, the default is the same as M_EDGE_RISING.			e		g				j	k	l	m	n	
M_ANY_EDGE	Specifies to generate an interrupt upon both a low-to-high and a high-to-low signal transition.			e		g				j	k	l	m	n	
M_EDGE_FALLING	Specifies that an interrupt will be generated upon a high-to-low signal transition.			e		g				j	k	l	m	n	
M_EDGE_RISING	Specifies that an interrupt will be generated upon a low-to-high signal transition.			e		g				j	k	l	m	n	
M_IO_INTERRUPT_STATE +	Sets whether to generate an interrupt upon the specified transition of the I/O signal. Use M_IO_INTERRUPT_ACTIVATION to specify the transition. Note that this only applies to input signals, or I/O signals set to input (using M_IO_MODE). INQ You must specify a combination value from the following table: • To set the type and number of the I/O signal to affect			e		g				j	k	l	m	n	
M_DEFAULT	Same as the transition specified by the DCF. If this value is not specified in the DCF, the default is the same as M_DISABLE.			e		g				j	k	l	m	n	

	M_DISABLE	Specifies not to generate an interrupt.			e		g					j	k	l	m	n
	M_ENABLE	Specifies to generate an interrupt.			e		g					j	k	l	m	n
M_IO_MODE +		Sets the mode of the specified I/O signal. Note that you can only change the mode of a bidirectional (I/O) signal. INQ You must specify a combination value from the following table: • To set the type and number of the I/O signal to affect			e		g					j	k	l	m	n
	M_DEFAULT	Same as the mode specified by the DCF, or disabled when dealing with a tri-state I/O signal.			e		g					j	k	l	m	n
	M_INPUT	Specifies that the signal is for input.			e		g					j	k	l	m	n
	M_OUTPUT	Specifies that the signal is for output.			e		g					j	k	l	m	n
M_IO_SOURCE +		Sets the type of signal to route to an output signal, or a bidirectional signal set to output mode. INQ You must specify a combination value from the following table: • To set the type and number of the I/O signal to affect			e		g					j	k	l	m	n
	M_DEFAULT	Same as the type of signal specified by the DCF. If this value is not specified in the DCF and dealing with an auxiliary signal, the default is M_USER_BITn, where *n* is the number of the user-bit that can drive the output signal. For the number of the user-bit that can drive the specified output signal, see the Connectors and signal names section of the Aurora Imaging Library Hardware-specific Notes chapter for your Zebra imaging board or Aurora Imaging Library driver.			e		g					j	k	l	m	n
		System specific														
		If this value is not specified in the DCF and dealing with a Camera Link camera control output signal, the default is M_USER_BIT_CC_IOn, where *n* is either 0 or 1.										j	k			
	M_AUX_IO*n*	Specifies to reroute auxiliary input signal *n* to the output signal, where *n* is the number of the auxiliary input signal. Note that the specified auxiliary signal can also be a bidirectional signal set to input (using M_IO_MODE set to M_INPUT). You can typically reroute an auxiliary input signal to an output signal that is hard-wired to the camera (for example, a Camera Control (CC) output signal or a transport layer (TL) trigger signal). Note that you cannot reroute auxiliary input signals to auxiliary output signals.			e		g					j	k	l	m	n
		System specific														
		For a list of the available auxiliary I/O signals, see the Connectors and signal names section of the Aurora Imaging Library Hardware-specific Notes chapter for your Zebra imaging board or Aurora Imaging Library driver.										j	k	l	m	
		n is a number from 0 to 31.			e		g									n
		The only output signal on which an auxiliary input signal can be routed to is a TL signal (using M_IO_SOURCE + M_TL_TRIGGER).												l	m	

		The only output signal on which an auxiliary input signal can be routed to is a CC signal (using M_IO_SOURCE + M_CC_IOn). For more information, refer to the Connectors and signal names section of the Aurora Imaging Library Hardware-specific Notes chapter for your Zebra hardware.									j	k				
	M_DISABLE	Specifies to not route any signal to the specified signal.			e		g				j	k	l	m	n	
	M_LINK_TRIGGER_*n*_IN	Specifies to route the transport layer link trigger input signal *n* (from the camera) to the output signal, where *n* is the number of the link trigger input signal.											l	m		
	M_TIMER*n*	Specifies to route the output of timer *n*, where *n* is the number of timers available.			e		g				j	k	l	m	n	
		System specific														
		In this case, *n* can be a value from 1 to 4.									j	k	l	m		
		With GigE Vision, USB3 Vision, and GenTL, *n* can be a value from 0 to 8.			e		g								n	
	M_TL_TRIGGER	Specifies to use the transport layer trigger signal (input mode only). The transport layer trigger signal is an embedded bidirectional signal from the physical transport layer connection of your camera. Typically, the TL Trigger is reserved for trigger information and is sent with other control and data signals along the same cable.											l	m		
	M_USER_BIT_CC_IO*n*	Specifies to route the state of bit *n* of the camera control static-user-output register, where *n* is a value from 0 to 1. Note that you can route a bit from the camera control static-user-output register to any of the 4 Camera Link camera control output signals (such as routing bit 0 to the M_CC_IO1 signal). Note that there is one camera control static-user-output register per acquisition path.									j	k				
	M_USER_BIT_TL_TRIGGER0	Specifies to route the state of the bit of the TL trigger static-user-output register.											l	m		
	M_USER_BIT*n*	Specifies to route the state of bit *n* of the main static-user-output register, where *n* is the bit number. Note that there is one main static-user-output register per acquisition path. However, if one of its bits can be routed to a signal that is shared across acquisition paths, changing that bit in one register affects the register of all the other acquisition paths. For the number of the user-bit that can drive the specified output signal, see the Connectors and signal names section of the Aurora Imaging Library Hardware-specific Notes chapter for your Zebra imaging board or Aurora Imaging Library driver.			e		g				j	k	l	m	n	
		System specific														
		With GigE Vision, USB3 Vision, and GenTL, *n* is a number from 0 to 31.			e		g								n	

Combination values for the values listed in For routing I/O signals and setting their mode.

You must add one of the following values to the above-mentioned values to set the type and number of the I/O signal to affect.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type and number of the I/O signal to affect																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AUX_IO*n*	Specifies to affect auxiliary signal *n*, where *n* is the signal number. > Note that, when using this value with M_IO_SOURCE, the auxiliary signal *n* must be an output signal, or an I/O signal set to output (using M_IO_MODE).			e		g					j	k	l	m	n	
	System specific															

	For a list of the available auxiliary I/O signals, see the Connectors and signal names section of the Aurora Imaging Library Hardware-specific Notes chapter for your Zebra imaging board or Aurora Imaging Library driver.									j	k	l	m		
	With GigE Vision, USB3 Vision, and GenTL, *n* is a number from 0 to 31.			e		g								n	
M_CC_IO*n*	Specifies to affect Camera Link camera control signal *n*, where *n* is a value from 1 to 4. Note that this combination value is only available for M_IO_SOURCE. For a list of the available camera control signals, see the Connectors and signal names section of the Aurora Imaging Library Hardware-specific Notes chapter for your Zebra imaging board or Aurora Imaging Library driver.									j	k				
M_LINK_TRIGGER_*n*_IN	Specifies to affect the link trigger input signal *n* (from the camera), where *n* is the number of the link trigger input signal. Note that this combination value is only available for M_IO_FORMAT, M_IO_INTERRUPT_ACTIVATION, M_IO_MODE, and M_IO_STATUS.											l	m		
M_LINK_TRIGGER_*n*_OUT	Specifies to affect the transport layer link trigger output signal *n* (to the camera), where *n* is the number of the link trigger output signal. Note that this combination value is only available for M_IO_FORMAT, M_IO_SOURCE, M_IO_MODE, and M_IO_STATUS.											l	m		
M_TL_TRIGGER	Specifies to affect the transport layer trigger signal. The transport layer trigger signal is an embedded bidirectional signal from the physical transport layer connection of your camera. Typically, the TL trigger signal is reserved for trigger information and is sent with other control and data signals along the same cable. Note that this combination value is only available for M_IO_FORMAT, M_IO_INTERRUPT_ACTIVATION, M_IO_SOURCE, M_IO_MODE, and M_IO_STATUS.											l	m		

The following control types and control values allow you to set the bits in a static-user-output register. You can route the bits to output signals or I/O signals set to output; to do so, use M_IO_SOURCE with M_USER_BIT...). To establish which user-bits can be routed to a specific signal, see the connectors and signal names section of the Aurora Imaging Library Hardware-specific Notes chapter for your Zebra imaging board.

> Note that for all Aurora Imaging Library supported hardware that have I/O signals, but are not supported with the constants below, see MsysControl().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For setting the state of specified user-bits in a static-user-output register																
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																
M_USER_BIT_STATE +	Sets the state of the specified bit of a static-user-output register. INQ You must specify a combination value from the following table: • To specify the static-user-output register and bit to affect															
M_OFF	Specifies that the specified bit is set to off.			e		g					j	k	l	m	n	
M_ON	Specifies that the specified bit is set to on.			e		g					j	k	l	m	n	
M_USER_BIT_STATE_ALL +	Sets the state of all the bits of the main static-user-output register or another specified static-user-output register. INQ			e		g					j	k	l	m	n	
	System specific															

		To affect a static-user-output register other than the main one, use one of the combination values documented below.											l	m		
	Value	Specifies a bit-encoded value that establishes the value of all the bits of the specified static-user-output register. It is recommended to specify the value in hexadecimal notation (0x), so that it is more legible to what you are setting each bit of the register.			e		g					j	k	l	m	n	

Combination values for M_USER_BIT_STATE.

You must add one of the following values to the above-mentioned value to specify the static-user-output register and bit to affect.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the bit in a static-user-output register to affect																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_USER_BIT_CC_IO*n*	Specifies which bit of the camera control static-user-output register to affect, where *n* is a value from 0 to 1. In this case, *n* represents the bit in the camera control static-user-output register. Note that there is one camera control static-user-output register per acquisition path.										j	k				
M_USER_BIT_TL_TRIGGER0	Specifies to affect the bit of the TL trigger static-user-output register.												l	m		
M_USER_BIT*n*	Specifies to affect bit *n* of the main static-user-output register. Note that there is one main static-user-output register per acquisition path. However, if one of its bits can be routed to a signal that is shared across acquisition paths, changing that bit in one register affects the register of all the other acquisition paths.			e		g					j	k	l	m	n	
	System specific															
	With GigE Vision, USB3 Vision, and GenTL, *n* is a number from 0 to 31.			e		g									n	

Combination values for M_USER_BIT_STATE_ALL.

You can add one of the following values to the above-mentioned value to specify the static-user-output register to affect, if you don't want to affect the main static-user-output register.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the static-user-output register																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_USER_BIT	Specifies to control the main static-user-output register. This is the default value.										j	k	l	m		
M_USER_BIT_CC_IO	Specifies to control the static-user-output register associated with Camera Link camera control signals (the camera control static-user-output register).										j	k				
M_USER_BIT_TL_TRIGGER	Specifies to control the static-user-output register associated with TL trigger signals (the TL trigger static-user-output register).												l	m		

The following control types and control values allow you to set the controls associated with triggered grabbing. For more information, see the Grabbing with triggers section of Chapter 27: Grabbing with your digitizer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling the settings to grab using a trigger																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																

M_GRAB_CONTINUOUS_END_TRIGGER	Sets whether to automatically generate a trigger after MdigHalt() is issued when performing a triggered continuous grab. INQ															
M_DEFAULT	Specifies the default value. M_ENABLE is the default for a software triggered grab. M_DISABLE is the default for a hardware triggered grab.	c		e	f	g			i		j	k	l	m	n	b
M_DISABLE	Specifies not to generate the trigger automatically. With a software triggered continuous grab operation, MdigHalt() will wait indefinitely until a software trigger is issued on a separate thread. With a hardware triggered continuous grab operation, MdigHalt() will wait indefinitely until a hardware trigger is generated before invoking the last grab.	c		e	f	g			i		j	k	l	m	n	b
M_ENABLE	Specifies to generate the trigger automatically. With a software or hardware triggered continuous grab operation, MdigHalt() will generate a software trigger that invokes the last grab.	c		e	f	g			i		j	k	l	m	n	b
M_GRAB_TRIGGER_ACTIVATION	Sets the signal transition upon which to generate a grab trigger. Use M_GRAB_TRIGGER_STATE to enable triggered grabbing. INQ			e	f	g			i		j	k	l	m	n	
M_DEFAULT	Same as the signal transition specified by the DCF. If the transition upon which to generate a grab trigger is not in the DCF, M_EDGE_RISING is the default.								i		j	k	l	m		
M_ANY_EDGE	Specifies that a trigger will be generated upon both a low-to-high and a high-to-low signal transition.			e	f	g									n	
M_EDGE_FALLING	Specifies that a trigger will be generated upon a high-to-low signal transition.			e	f	g			i		j	k	l	m	n	
M_EDGE_RISING	Specifies that a trigger will be generated upon a low-to-high signal transition.			e	f	g			i		j	k	l	m	n	
M_LEVEL_HIGH	Specifies that a trigger is continuously issued during a high signal polarity.			e	f	g			i		j	k	l	m	n	
M_LEVEL_HIGH_END_WHEN_INACTIVE	Specifies that triggered grabs are initiated by a high trigger signal polarity, and completed when the trigger signal goes low.			e	f	g			i		j	k	l	m	n	
M_LEVEL_LOW	Specifies that a trigger is continuously issued during a low signal polarity.			e	f	g			i		j	k	l	m	n	
M_LEVEL_LOW_END_WHEN_INACTIVE	Specifies that triggered grabs are initiated by a low trigger signal polarity, and completed when the trigger signal goes high.			e	f	g			i		j	k	l	m	n	
M_GRAB_TRIGGER_DELAY	Sets the delay between the trigger and the grab. Note, an error is generated if the specified delay cannot be respected. INQ			e	f	g									n	b
Value >= 0	Specifies the delay, in nsec.			e	f	g									n	b
M_GRAB_TRIGGER_MISSED	Sets whether the number of grab triggers missed should be counted. INQ								i							
M_DISABLE	Specifies to not count the number of grab triggers missed. This is the default value.								i							

M_ENABLE	Specifies to count the number of grab triggers missed.								i							
M_GRAB_TRIGGER_OVERLAP	Sets how to deal with a new grab trigger that occurs when the current grab is in progress. INQ			e	f	g			i		j	k	l	m	n	b
	System specific															
	On the camera, this control type allows the exposure phase of a triggered grab to occur while the previous image is still being transferred to the Host. If disabled, the exposure phase and the data transfer phase will occur sequentially. If the exposure phase ends before the previously grabbed image is transferred to the Host, disabling this control type can lead to dropped frames. If, however, the exposure phase ends after the previously grabbed image is transferred to the Host, disabling this control type can increase the speed of your application.			e	f	g			i						n	b
M_DEFAULT	Specifies the default value.			e	f	g			i		j	k	l	m	n	b
	System specific															
	Same as M_ENABLE.			e	f	g			i						n	b
	Same as M_RESET.										j	k	l	m		
M_DISABLE	Specifies that a triggered grab will not overlap the transfer of the previous image. Note that, when grab trigger overlap is disabled, the exposure phase and the data transfer phase will occur sequentially.			e	f	g			i						n	b
M_ENABLE	Specifies that a triggered grab can overlap the transfer of the previous image.			e	f	g			i						n	b
M_OFF	Specifies that a new trigger is ignored.										j	k	l	m		
M_PREVIOUS_FRAME	Specifies that a trigger received, while a grab is in progress, will be latched (stored). As soon as the current grab finishes, a new grab will be started.										j	k	l	m		
M_RESET	Specifies that the current grab will be immediately stopped and a new grab will be started.										j	k	l	m		
M_GRAB_TRIGGER_SOFTWARE	Issues a software trigger. To use this control type, the trigger source must be set to software (M_GRAB_TRIGGER_SOURCE set to M_SOFTWARE). The call to issue a software trigger should be made in a thread separate from the thread performing the continuous grab operation (or the series of asynchronous grabs) to be triggered. If using MdigProcess(), the grab operation is automatically performed in a separate thread from the software trigger (along with any other issued M_GRAB_TRIGGER... controls); whereas, MdigGrabContinuous() and MdigGrab() are performed in the same thread as the software trigger, unless otherwise programmed.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Note that, for this control type to function on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									
M_ACTIVATE	Specifies the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB_TRIGGER_SOURCE	Sets the signal source of the grab trigger when there are multiple sources available. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															

		Note that, when using an external trigger source to control both your external lighting source and your Zebra Iris's grab, if one out of x grabbed images is significantly darker (insufficiently lit), your external trigger source might be triggering faster than your Zebra Iris can grab, transfer the image, and prepare to grab again.							i							
	M_DEFAULT	Same as the one specified by the DCF.							i		j	k	l	m		b
		System specific														
		Same as M_NULL.														b
	M_NULL	Specifies that there is no trigger source.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUX_IO*n*	Specifies to use auxiliary input signal *n* as the trigger source, where *n* is the number of the auxiliary signal.			e	f	g			i		j	k	l	m	n	
		System specific														
		Note that the specified auxiliary signal can also be a bidirectional signal set to input (using M_IO_MODE set to M_INPUT).			e		g					j	k			n	
		With Zebra Rapixo CXP and Zebra Rapixo CoF, *n* is a number from 0 to 31, but you can only use input signals.												l	m		
		With GigE Vision, USB3 Vision, and GenTL, *n* is a number from 0 to 31.			e		g									n	
		With Zebra Iris GTX, *n* is a number from 3 to 6.								i							
	M_HSYNC	Specifies to use the horizontal synchronization signal of the camera as the trigger source.										j	k				
	M_IO_COMMAND_LIST1 +	Specifies to use a bit of the I/O command register of I/O command list 1 as a trigger source. You must specify a combination value from the following table: • To specify which I/O command register bit to use								i							
	M_ROTARY_ENCODER	Specifies to use the output of the default rotary decoder (set with M_ROTARY_ENCODER_OUTPUT_MODE) as the trigger source.								i		j	k	l	m		
		System specific														
		This control type specifies to use the rotary decoder corresponding to 1+ the device number of the specified digitizer (DigId) will be used. For example, when the device number of the specified digitizer is M_DEV2, rotary decoder M_ROTARY_ENCODER3 is used.												l	m		
	M_ROTARY_ENCODER*n*	Specifies to use rotary decoder *n* as the trigger source, where *n* is a number between 1 and 4. To use the output of the default rotary decoder, use M_ROTARY_ENCODER.								i				l	m		
		System specific														
		With Zebra Iris GTX, *n* must be 1.								i							
	M_SOFTWARE	Specifies to use a software trigger as the trigger source. Use M_GRAB_TRIGGER_SOFTWARE to issue the trigger.	c		e	f	g	a		i		j	k	l	m	n	b

M_SOFTWARE*n*	Specifies to use the software trigger being used for the grab of another digitizer (on the same board) as the trigger source. With this control value, *n* corresponds to 1+ the device number of the digitizer that is (by default) associated with the specified software trigger, and can be from 1 to 4.											l	m		
M_TIMER*n*	Specifies to use the output signal of the specified timer as the trigger source. Use the M_TIMER_... to set up the timer.			e	f	g			i		j	k	l	m	n	
	System specific															
	In this case, *n* can be a value from 1 to 4.										j	k	l	m		
	With Zebra Iris GTX, *n* is a number from 1 to 8.								i							
	With GigE Vision, USB3 Vision, and GenTL, *n* can be a value from 0 to 8.			e		g									n	
M_VSYNC	Specifies to use the vertical synchronization signal of the camera as the trigger source.										j	k				
M_GRAB_TRIGGER_STATE	Sets whether, when a grab command is issued (for example, MdigGrab()) to wait for a trigger before grabbing. If your Zebra frame grabber supports trigger input, you can set up your digitizer to perform a triggered grab, that is, to grab a frame upon the occurrence of an event. In this case, nothing is grabbed when you call MdigGrab() or MdigGrabContinuous() or MdigProcess(), until a specified event occurs. When grabbing continuously, the digitizer waits for a trigger before grabbing each frame; you must still call MdigHalt() after grabbing all required frames. Note that, you can specify the number of frames to grab per trigger received (using MdigProcess() with M_FRAMES_PER_TRIGGER()) or only use a single trigger to start the grab (using MdigProcess() with M_TRIGGER_FOR_FIRST_GRAB). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Use M_GRAB_TRIGGER_SOURCE to specify the trigger source.			e	f	g			i						n	b
	The trigger source is automatically set to use a software trigger.	c					a									
	Note that, for this control type to function on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									
M_DEFAULT	Same as the state specified by the DCF or, if none specified, M_DISABLE.	c							i		j	k	l	m		
M_DISABLE	Specifies that, when a grab command is issued, the grab occurs without waiting for a trigger. Note that, this is the default behavior when using a software trigger.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that, when a grab command is issued, the grab waits for a trigger before occurring.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for [when ControlType=M_GRAB_TRIGGER_SOURCE: M_IO_COMMAND_LIST1].

You must add the following value to the above-mentioned value to specify which I/O command register bit to use.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying which I/O command register bit to use																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_IO_COMMAND_BIT*n*	Specifies I/O command register bit *n*, where *n* represents the bit number. With Zebra Iris GTX, *n* can be a value from 0 to 4.								i							

The following control types allow you to control the settings for a grab of sequential frames into a multi-frame image buffer (frame burst).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling frame bursts																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_GRAB_FRAME_BURST_END_TRIGGER_SOURCE		Sets the signal from which a rising edge signals the end of a grab of sequential frames into a multi-frame image buffer (end-of-frame-burst trigger). This control type only has an effect if the M_GRAB_FRAME_BURST_END_TRIGGER_STATE is set to M_ENABLE. INQ															
	M_DEFAULT	Same as M_AUX_IO0.										j	k	l	m		
	M_AUX_IO*n*	Specifies to use auxiliary input signal *n* as the trigger source, where *n* is the number of the auxiliary signal. Note that the specified auxiliary signal can also be a bidirectional signal set to input, using M_IO_MODE set to M_INPUT.										j	k	l	m		
M_GRAB_FRAME_BURST_END_TRIGGER_STATE		Sets whether a grab into a multi-frame image buffer is ended upon an end-of-frame-burst trigger. Specify the trigger source using M_GRAB_FRAME_BURST_END_TRIGGER_SOURCE. Note that regardless of this setting, the grab also ends if the multi-frame image buffer is filled or the timeout is reached (M_GRAB_FRAME_BURST_MAX_TIME). INQ										j	k	l	m		
	M_DEFAULT	Same as M_DISABLE.										j	k	l	m		
	M_DISABLE	Specifies that an end-of frame-burst trigger is not used to end the grab.										j	k	l	m		
	M_ENABLE	Specifies that an end-of frame-burst trigger is used to end the grab.										j	k	l	m		
M_GRAB_FRAME_BURST_MAX_TIME		Sets the maximum amount of time to wait for all the frames to be grabbed into the multi-frame image buffer. The timer starts when the first frame is grabbed. The number of frames in the buffer can be inquired using MdigGetHookInfo() with M_GRAB_FRAME_BURST_COUNT. This is useful when the camera stops sending frames and the multi-frame buffer is only partially full. INQ										j	k	l	m		
	M_DEFAULT	Same as M_INFINITE.										j	k	l	m		
	M_INFINITE	Specifies to wait indefinitely.										j	k	l	m		
	0.000008 <= *Value* <= 1.000000	Specifies the maximum amount of time to wait, in secs.										j	k	l	m		

M_GRAB_FRAME_BURST_SIZE		Sets the number of sequential frames to grab into a multi-frame image buffer with one grab command (MdigGrab(), or one grab of MdigProcess()); the defined number of frames are stored contiguously in the same buffer. The end-of-grab event only occurs once the entire group of frames has been grabbed, reducing the number of events that need to be handled. The Y-size of the grab buffer must be equal to (height of the frame * M_GRAB_FRAME_BURST_SIZE). For more information about the grab buffer size, see the Specifying the dimensions of a multi-frame image buffer subsection of the Specifying the dimensions of a data buffer section of Chapter 23: Data buffers. INQ										j	k	l	m		
	M_DEFAULT	Specifies the default value; the default value is 1.										j	k	l	m		
	1 <= *Value* <= 4095	Specifies the number of frames to grab.										j	k	l	m		

The following control types and control values specify the settings for controlling a timer and the signal generated from the timer (timer output signals). For more information, see the Grabbing with triggers section of Chapter 27: Grabbing with your digitizer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling the settings of a timer																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_TIMER_ARM +		Sets whether to enable the timer arming mechanism. If the timer arming mechanism is enabled, then the timer will ignore its trigger signal (M_TIMER_TRIGGER_SOURCE) until a signal transition specified using M_TIMER_ARM_ACTIVATION occurs on the signal specified using M_TIMER_ARM_SOURCE. INQ You must specify a combination value from the following table: • To specify which on-board timer to control															
	M_DEFAULT	Same as M_DISABLE.												l	m		
	M_DISABLE	Specifies that timer arming is disabled.												l	m		
	M_ENABLE	Specifies that timer arming is enabled.												l	m		
M_TIMER_ARM_ACTIVATION +		Sets the signal transition upon which to arm the timer, if the timer arming mechanism is enabled. Use M_TIMER_ARM to enable timer arming. Use M_TIMER_ARM_SOURCE to set which input signal will arm the timer. INQ You must specify a combination value from the following table: • To specify which on-board timer to control												l	m		
	M_DEFAULT	Same as M_EDGE_RISING.												l	m		
	M_EDGE_FALLING	Specifies that the timer will be armed upon a high-to-low signal transition.												l	m		
	M_EDGE_RISING	Specifies that the timer will be armed upon a low-to-high signal transition.												l	m		
	M_LEVEL_HIGH	Specifies that a timer is continuously armed during a high signal polarity.												l	m		
	M_LEVEL_LOW	Specifies that a timer is continuously armed during a low signal polarity.												l	m		

M_TIMER_ARM_SOFTWARE +	Issues a software trigger to arm the timer. To use this setting, the timer's arm source must be set to software (M_TIMER_ARM_SOURCE set to M_SOFTWARE). You must specify a combination value from the following table: • To specify which on-board timer to control												I	m		
M_ACTIVATE	Specifies the default behavior.												I	m		
M_TIMER_ARM_SOURCE +	Sets which input signal will arm the timer, if the timer arming mechanism is enabled. Use M_TIMER_ARM to enable timer arming. INQ You must specify a combination value from the following table: • To specify which on-board timer to control												I	m		
M_DEFAULT	Specifies to use the input source set in the DCF.												I	m		
M_NULL	Specifies to use no trigger source.												I	m		
M_AUX_IO*n*	Specifies to use auxiliary input signal *n* as the trigger source for the specified timer, where *n* is the number of the auxiliary signal. Note that the specified auxiliary signal can also be a bidirectional signal set to input (using M_IO_MODE set to M_INPUT). For a list of the available auxiliary I/O signals, see the Connectors and signal names section of the Aurora Imaging Library Hardware-specific Notes chapter for your Zebra imaging board or Aurora Imaging Library driver.												I	m		
M_CONTINUOUS	Specifies to automatically arm the specified timer immediately after the timer's duration expires.												I	m		
M_EXPOSURE_END	Specifies to use the exposure end signal as the trigger source. The exposure end signal is a quick pulse that occurs when the exposure ends.												I	m		
M_EXPOSURE_START	Specifies to use the exposure start signal as the trigger source. The exposure start signal is a quick pulse that occurs when the exposure starts.												I	m		
M_GRAB_TRIGGER	Specifies to use the grab trigger source signal as the trigger source. To set the grab trigger source signal, use M_GRAB_TRIGGER_SOURCE.												I	m		
M_HSYNC	Specifies to use the horizontal synchronization signal as the trigger source.												I	m		
M_ROTARY_ENCODER	Specifies to use the output of the default rotary decoder (set with M_ROTARY_ENCODER_OUTPUT_MODE) as the trigger source.												I	m		
M_ROTARY_ENCODER*n*	Specifies to use rotary decoder n, where *n* is a number between 1 and 4. To use the output of the default rotary decoder, use M_ROTARY_ENCODER.												I	m		
M_SOFTWARE	Specifies to use a software trigger as the trigger source. Use M_TIMER_TRIGGER_SOFTWARE to issue the trigger.												I	m		
M_TIMER*n*	Specifies to use the output signal of the specified timer as the trigger source, where *n* is the timer number. > Note that, *n* must be a value from 1 to 4.												I	m		

	M_TL_TRIGGER	Specifies to use the transport layer trigger signal. The transport layer trigger signal is an embedded bidirectional signal from the physical transport layer connection of your camera. Typically, the TL Trigger is reserved for trigger information and is sent with other control and data signals along the same cable.											l	m		
	M_VSYNC	Specifies to use the vertical synchronization signal as the trigger source.											l	m		
M_TIMER_CLOCK_SOURCE +		Sets the source of the clock that drives the specified timer. The clock source must have a frequency greater than or equal to 1 Hz. INQ You must specify a combination value from the following table: • To specify which on-board timer to control									j	k	l	m		
	M_DEFAULT	Specifies to use the most appropriate clock source (as determined by the driver).									j	k	l	m		
	M_HSYNC	Specifies to use the horizontal synchronization frequency of your camera.									j	k	l	m		
	M_PIXCLK	Specifies to use the pixel clock frequency of your camera.									j	k				
	M_SYSCLK	Specifies to use the frequency of the allocated system's clock source.									j	k	l	m		
		System specific														
		The frequency is programmable in the DCF.									j	k				
		For Zebra Rapixo CXP and Zebra Rapixo CoF, the frequency is set to 125MHz, and it cannot be changed.											l	m		
	M_TIMER*n*	Specifies to use the frequency of the output of the specified timer, where *n* is the timer number. Note that, in this case, the timer output pulse is used as a clock tick. The specified timer should be in continuous mode (that is, M_TIMER_TRIGGER_SOURCE is set to M_CONTINUOUS). Only a continuous timer can clock another timer. References where timer 1 points to timer 2, and timer 2 points back to timer 1, are not supported and will generate an error.									j	k	l	m		
		System specific														
		In this case, *n* can be a value from 1 to 4.									j	k	l	m		
	M_VSYNC	Specifies to use the vertical synchronization frequency of your camera.									j	k	l	m		
M_TIMER_DELAY +		Sets the delay between the timer trigger and the active portion of the timer output signal. Note, an error is generated if the specified delay cannot be respected. INQ You must specify a combination value from the following table: • To specify which on-board timer to control			e		g			i		j	k	l	m	n	
	M_DEFAULT	Specifies the default value.								i		j	k	l	m		
		System specific														
		Same as the delay specified by the DCF.										j	k	l	m		
	0	Specifies that there is no delay.			e		g			i						n	

	Value > 0	Specifies the delay, in nsec.			e		g			i		j	k	l	m	n	
M_TIMER_DELAY2 +		Sets the delay between the end of the first active portion of the timer output signal and the start of the second pulse. INQ You must specify a combination value from the following table: • To specify which on-board timer to control										j	k	l	m		
	M_DEFAULT	Specifies the default value. This is the same as specified in the DCF or, if not specified in the DCF, 0.										j	k	l	m		
	Value > 0	Specifies the delay, in nsec.										j	k	l	m		
M_TIMER_DURATION +		Sets the duration for the active portion of the timer output signal. Note, an error is generated if the specified duration cannot be respected. INQ You must specify a combination value from the following table: • To specify which on-board timer to control			e		g			i		j	k	l	m	n	
		System specific															
		Unlike with other products, when using the grab trigger signal to trigger the strobe, the specified duration of the strobe's timer output signal specifies how long the timer should remain active during the exposure period. The active period of the strobe's timer output will actually start after the specified delay, but the count for how long it should remain active only starts when the camera starts exposing its image sensor. Once counting begins, the signal will typically be active for the specified duration. If the duration of the strobe's timer is greater than the exposure time, the strobe's timer's output signal will end when the exposure time ends (set using M_EXPOSURE_TIME).								i							
	M_DEFAULT	Specifies to use the duration specified by the DCF.								i		j	k	l	m		
	Value > 0	Specifies the duration of the active portion of the timer output signal, in nsec.			e		g			i		j	k	l	m	n	
M_TIMER_DURATION2 +		Sets the duration for the active portion of the second pulse of the timer output signal. INQ You must specify a combination value from the following table: • To specify which on-board timer to control										j	k	l	m		
	M_DEFAULT	Specifies the default value. This is the same as specified in the DFC or, if not specified in the DCF, 0.										j	k	l	m		
	Value > 0	Specifies the duration of the active portion of the second pulse of the timer output signal, in nsecs.										j	k	l	m		
M_TIMER_OUTPUT_INVERTER +		Sets whether the output of the timer should be inverted. This causes the low portion of the signal (the delay period) to be high and the high portion of the signal (the active portion) to be low. INQ You must specify a combination value from the following table: • To specify which on-board timer to control								i		j	k	l	m		
	M_DEFAULT	Specifies the default value.										j	k	l	m		
		System specific															

		Same as the value specified by the DCF.									j	k	l	m			
	M_DISABLE	Specifies not to invert the output of the timer.								i		j	k	l	m		
	M_ENABLE	Specifies to invert the output of the timer.								i		j	k	l	m		
M_TIMER_RESET_SOURCE +		Sets the signal source to use to reset the timer to 0. INQ You must specify a combination value from the following table: • To specify which on-board timer to control												l	m		
	M_DEFAULT	Specifies to use the trigger source set in the DCF.												l	m		
	M_NULL	Specifies to use no trigger source.												l	m		
	M_AUX_IO*n*	Specifies to use auxiliary input signal *n* as the trigger source, where *n* is the number of the auxiliary signal. Note that the specified auxiliary signal can also be a bidirectional signal set to input (using M_IO_MODE set to M_INPUT). For a list of the available auxiliary I/O signals, see the Connectors and signal names section of the Aurora Imaging Library Hardware-specific Notes chapter for your Zebra imaging board or Aurora Imaging Library driver.												l	m		
	M_EXPOSURE_END	Specifies to use the exposure end signal as the trigger source. The exposure end signal is a quick pulse that occurs when the exposure ends.												l	m		
	M_EXPOSURE_START	Specifies to use the exposure start signal as the trigger source. The exposure start signal is a quick pulse that occurs when the exposure starts.												l	m		
	M_GRAB_TRIGGER	Specifies to use the grab trigger source signal as the trigger source. To set the grab trigger source signal, use M_GRAB_TRIGGER_SOURCE.												l	m		
	M_HSYNC	Specifies to use the horizontal synchronization signal as the trigger source.												l	m		
	M_ROTARY_ENCODER	Specifies to use the output of the default rotary decoder (set with M_ROTARY_ENCODER_OUTPUT_MODE) as the trigger source.												l	m		
	M_ROTARY_ENCODER*n*	Specifies to use rotary decoder n, where *n* is a number between 1 and 4. To use the output of the default rotary decoder, use M_ROTARY_ENCODER.												l	m		
	M_SOFTWARE	Specifies to use a software trigger as the trigger source. Use M_TIMER_TRIGGER_SOFTWARE to issue the trigger.												l	m		
	M_TIMER*n*	Specifies to use the output signal of the specified timer as the trigger source, where *n* is the timer number.												l	m		
	M_TL_TRIGGER	Specifies to use the transport layer trigger signal. The transport layer trigger signal is an embedded bidirectional signal from the physical transport layer connection of your camera. Typically, the TL Trigger is reserved for trigger information and is sent with other control and data signals along the same cable.												l	m		
	M_VSYNC	Specifies to use the vertical synchronization signal as the trigger source.												l	m		

M_TIMER_STATE +	Sets the state of the specified timer. When a timer is enabled, the timer waits for a trigger to be received. To set the source of the trigger, use M_TIMER_TRIGGER_SOURCE. Once the trigger is received, the timer starts by outputting a low signal. This lasts for the duration of the delay period (set using M_TIMER_DELAY). The timer then changes to output a high signal for the duration of the active period (set using M_TIMER_DURATION). To invert this signal, use M_TIMER_OUTPUT_INVERTER. INQ You must specify a combination value from the following table: • To specify which on-board timer to control			e		g			i		j	k	l	m	n	
M_DEFAULT	Specifies the default value.								i		j	k	l	m		
	System specific															
	Same as the state specified by the DCF.										j	k	l	m		
	Same as M_TIMER_STATE set to M_DISABLE.								i							
M_DISABLE	Specifies that the timer is disabled.			e		g			i		j	k	l	m	n	
M_ENABLE	Specifies that the timer is enabled.			e		g			i		j	k	l	m	n	
M_TIMER_TRIGGER_ACTIVATION +	Sets the signal variation upon which to generate a timer trigger, if the specified timer is enabled. Use M_TIMER_STATE to enable the timer. INQ You must specify a combination value from the following table: • To specify which on-board timer to control			e		g					j	k	l	m	n	
M_DEFAULT	Specifies the default value.			e		g					j	k	l	m	n	
	System specific															
	Same as the signal variation specified by the DCF.										j	k	l	m		
	Same as M_EDGE_RISING.			e		g									n	
M_ANY_EDGE	Specifies that a timer trigger will be generated both upon a high-to-low and a low-to-high signal transition.			e		g									n	
M_EDGE_FALLING	Specifies that a timer trigger will be generated upon a high-to-low signal transition.			e		g					j	k	l	m	n	
M_EDGE_RISING	Specifies that a timer trigger will be generated upon a low-to-high signal transition.			e		g					j	k	l	m	n	
M_LEVEL_HIGH	Specifies that a timer trigger is continuously issued during a high signal polarity.			e		g									n	
M_LEVEL_LOW	Specifies that a timer trigger is continuously issued during a low signal polarity.			e		g									n	
M_TIMER_TRIGGER_MISSED +	Sets whether to count the number of trigger pulses missed. INQ You must specify a combination value from the following table: • To specify which on-board timer to control										j	k	l	m		

	M_DEFAULT	Same as M_DISABLE.									j	k	l	m		
	M_DISABLE	Specifies not to count the number of trigger pulses missed.									j	k	l	m		
	M_ENABLE	Specifies to count the number of trigger pulses missed.									j	k	l	m		
M_TIMER_TRIGGER_OVERLAP +		Sets how to deal with a new trigger that occurs while the associated timer has not yet expired (both its delay and duration). This is applied to both the delay and duration. INQ You must specify a combination value from the following table: • To specify which on-board timer to control									j	k	l	m		
	M_DEFAULT	Same as M_RESET.									j	k	l	m		
	M_LATCH	Specifies that a trigger received, while the associated timer has not expired, will be latched (stored). As soon as the current timer expires, a new trigger is issued by software.									j	k	l	m		
	M_OFF	Specifies that a new trigger is ignored.									j	k	l	m		
	M_RESET	Specifies that a new trigger automatically resets the timer (regardless of whether it is in its delay or active period) and then restarts the timer. This process will repeat for each new trigger received.									j	k	l	m		
M_TIMER_TRIGGER_RATE_DIVIDER +		Sets the frequency with which to accept trigger pulses (for example, if set to 2, the first trigger pulse is ignored and the second is accepted). INQ You must specify a combination value from the following table: • To specify which on-board timer to control									j	k	l	m		
	M_DEFAULT										j	k	l	m		
	1 <= *Value* <= 255	Specifies the frequency with which to accept a trigger out of a series of trigger pulses. Note that, if set to 1, all trigger pulses are accepted.									j	k	l	m		
M_TIMER_TRIGGER_SOFTWARE +		Issues a software trigger for the specified timer. To use this setting, the timer's trigger source must be set to software (M_TIMER_TRIGGER_SOURCE set to M_SOFTWARE). You must specify a combination value from the following table: • To specify which on-board timer to control			e		g				j	k	l	m	n	
	M_ACTIVATE	Specifies the default behavior.			e		g				j	k	l	m	n	
M_TIMER_TRIGGER_SOURCE +		Selects the trigger source for the specified timer when there are multiple sources available. INQ You must specify a combination value from the following table: • To specify which on-board timer to control			e		g			i	j	k	l	m	n	
	M_DEFAULT	Same as the one specified by the DCF.			e		g				j	k	l	m	n	
	M_NULL	Specifies that no trigger source is specified.			e		g			i	j	k	l	m	n	

	M_AUX_IO*n*	Specifies to use auxiliary input signal *n* as the trigger source for the specified timer, where *n* is the number of the auxiliary signal. Note that the specified auxiliary signal can also be a bidirectional signal set to input (using M_IO_MODE set to M_INPUT). For a list of the available auxiliary I/O signals, see the Connectors and signal names section of the Aurora Imaging Library Hardware-specific Notes chapter for your Zebra imaging board or Aurora Imaging Library driver.			e		g					j	k	l	m	n	
	M_CONTINUOUS	Specifies to run the specified timer in periodic mode; no actual trigger signal is used. The timer is automatically reset after the timer's duration expires. The timer loops between a delay and an active period.			e		g					j	k	l	m	n	
	M_EXPOSURE_END	Specifies to use the exposure end signal as the trigger source. The exposure end signal is a quick pulse that occurs when the exposure ends.			e		g									n	
	M_EXPOSURE_START	Specifies to use the exposure start signal as the trigger source. The exposure start signal is a quick pulse that occurs when the exposure starts.			e		g			i						n	
	M_GRAB_TRIGGER	Specifies to use the grab trigger source signal as the trigger source. To set the grab trigger source signal, use M_GRAB_TRIGGER_SOURCE.			e		g			i						n	
	M_HSYNC	Specifies to use the horizontal synchronization signal as the trigger source.										j	k	l	m		
	M_ROTARY_ENCODER	Specifies to use the output of the default rotary decoder (set with M_ROTARY_ENCODER_OUTPUT_MODE) as the trigger source.										j	k	l	m		
		System specific															
		This control type specifies to use the rotary decoder corresponding to 1+ the device number of the specified digitizer (DigId) will be used. For example, when the device number of the specified digitizer is M_DEV2, rotary decoder M_ROTARY_ENCODER3 is used.												l	m		
	M_ROTARY_ENCODER*n*	Specifies to use rotary decoder n, where *n* is a number between 1 and 4. To use the output of the default rotary decoder, use M_ROTARY_ENCODER.												l	m		
	M_SOFTWARE	Specifies to use a software trigger as the trigger source. Use M_TIMER_TRIGGER_SOFTWARE to issue the trigger.			e		g					j	k	l	m	n	
	M_TIMER*n*	Specifies to use the output signal of the specified timer as the trigger source, where *n* is the timer number.			e		g					j	k	l	m	n	
		System specific															
		In this case, *n* can be a value from 1 to 4.										j	k	l	m		
		With GigE Vision, USB3 Vision, and GenTL, *n* can be a value from 0 to 8.			e		g									n	
	M_TL_TRIGGER	Specifies to use the transport layer trigger signal. The transport layer trigger signal is an embedded bidirectional signal from the physical transport layer connection of your camera. Typically, the TL Trigger is reserved for trigger information and is sent with other control and data signals along the same cable.												l	m		
	M_VSYNC	Specifies to use the vertical synchronization signal as the trigger source.										j	k	l	m		

The following control types and control values relate to connection testing.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For connection testing

ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_CONNECTION_TEST_ERROR_COUNT +		Reset the current connection error count for test packets. INQ You must specify a combination value from the following table: • To specify the CXP input connector															
	Value = 0	Specifies to immediately reset the current connection error count for test packets.												l	m		
M_CONNECTION_TEST_MODE +		Sets whether to enable the CXP test mode. INQ You must specify a combination value from the following table: • To specify the CXP input connector												l	m		
	M_MODE1	Specifies that the CXP test mode is enabled.												l	m		
	M_OFF	Specifies that the CXP test mode is disabled.												l	m		
M_CONNECTION_TEST_PACKET_RECEIVED_COUNT +		Resets the count of test packets received. INQ You must specify a combination value from the following table: • To specify the CXP input connector												l	m		
	Value = 0	Specifies to immediately reset the count of test packets received.												l	m		
M_CONNECTION_TEST_PACKET_TRANSMITTED_COUNT +		Resets the count of test packets transmitted. INQ You must specify a combination value from the following table: • To specify the CXP input connector												l	m		
	Value = 0	Specifies to immediately reset the count of test packets transmitted.												l	m		

Combination value for the values listed in For connection testing.

You must add the following value to the above-mentioned values to specify the CXP input connector.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the CXP input connector																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CONNECTION*n*	Specifies the connection at CXP input connector *n*, where *n* is the CXP input connector number from 0 to 3.												l	m		

The following control types and control values specify the settings for controlling the camera's exposure. For more information, see the Aurora Imaging Library Hardware-specific Notes for your Zebra imaging board.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling the camera's exposure

ControlType	ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AUTO_EXPOSURE		Sets the automatic exposure mode of the camera. INQ															
	M_DEFAULT	Specifies to use the camera's default values or, if supported, to enable the camera's auto control state for the gain, iris, and shutter features (that is, the camera adjusts these values automatically).			e	f	g									n	b
	M_AUTOMATIC	Specifies that the camera's exposure duration is constantly adapted by the device to maximize its effect.			e	f	g									n	b
	M_DISABLE	Specifies that the camera will not control the gain, iris, and shutter settings automatically.			e	f	g									n	b
	M_ENABLE	Specifies that the camera controls the gain, iris, and shutter settings automatically.			e	f	g									n	b
M_EXPOSURE_DELAY		Sets the required delay between the exposure trigger and the start of your camera's image sensor being exposed. Note, an error is generated if the specified delay cannot be respected. INQ								i							
	M_DEFAULT	Specifies the default value; the default value is 0 nsec.								i							
	Value >= 0	Specifies the delay, in nsec.								i							
M_EXPOSURE_MODE		Sets whether the exposure duration is set using an exposure timer or using the width of the trigger signal. INQ								i							
	M_TIMED	Specifies to use M_EXPOSURE_TIME to control the length of the exposure.								i							
	M_TRIGGER_WIDTH	Specifies to use duration of the active portion of the grab trigger's output signal to control the length of the exposure. Use M_GRAB_TRIGGER_ACTIVATION to specify the signal transition upon which to generate a grab trigger. Note that, M_TRIGGER_WIDTH can only be used when M_GRAB_TRIGGER_ACTIVATION is set to M_LEVEL_HIGH or M_LEVEL_LOW.								i							
M_EXPOSURE_TIME		Sets the amount of time to expose the camera's image sensor. Note, an error is generated if the specified duration cannot be respected. INQ			e	f	g			i						n	b
	M_DEFAULT	Specifies to use the camera's default exposure time, as specified by the DCF.								i							
	Value >= 0	Specifies the amount of time to expose the camera's image sensor, in nsec.			e	f	g			i						n	b
		System specific															
		For Iris GTX, the minimum exposure time is 50000 nsec for monochrome cameras and 240000 nsec for color cameras.								i							

The following control types allow you to control the settings of a quadrature decoder with inputs from a rotary or linear encoder.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling the settings of a quadrature decoder with inputs from a rotary or linear encoder																	
ControlType	ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_ROTARY_ENCODER_BIT0_SOURCE		Sets the auxiliary input signal on which to receive bit 0 of the 2-bit Gray code. Note that if the auxiliary input signal for bit 0 is set using this constant, the corresponding auxiliary input signal for bit 1 will automatically be assigned; you do not need to specify it using M_ROTARY_ENCODER_BIT1_SOURCE. INQ														
	M_DEFAULT	Same as DCF.									j	k	l	m		
	M_AUX_IO*n*	Specifies the auxiliary input signal to use. The auxiliary input signal must support bit 0 of quadrature input.									j	k	l	m		
M_ROTARY_ENCODER_BIT1_SOURCE		Sets the auxiliary input signal on which to receive bit 1 of the 2-bit Gray code. Note that if the auxiliary input signal for bit 1 is set using this constant, the corresponding auxiliary input signal for bit 0 will automatically be assigned; you do not need to specify it using M_ROTARY_ENCODER_BIT0_SOURCE. INQ									j	k	l	m		
	M_DEFAULT	Same as DCF.									j	k	l	m		
	M_AUX_IO*n*	Specifies the auxiliary input signal to use. The auxiliary input signal must support bit 1 of quadrature input.									j	k	l	m		
M_ROTARY_ENCODER_DIRECTION +		Sets the direction of movement occurring when the Gray code sequence received by the rotary decoder is 00 - 01 - 11 - 10; this essentially establishes whether to increment or decrement the counter when receiving this sequence. For more information on how to set the direction, refer to the Interpreting the direction of movement from the Gray code subsection of the Using quadrature input from a rotary encoder section of Chapter 56: I/O signals and communicating with external devices. INQ									j	k	l	m		
	M_DEFAULT	Same as DCF.									j	k	l	m		
	M_BACKWARD	Specifies a backward direction of movement and to decrement the counter when the Gray code sequence is 00 - 01 - 11 - 10.									j	k	l	m		
		System specific														
		With Zebra Radient eV-CL, this control value increments the counter when the Gray code sequence is 00 - 01 - 11 - 10.									j					
	M_FORWARD	Specifies a forward direction of movement and to increment the counter when the Gray code sequence is 00 - 01 - 11 - 10.									j	k	l	m		
		System specific														
		With Zebra Radient eV-CL, this control value decrements the counter when the Gray code sequence is 00 - 01 - 11 - 10.									j					
M_ROTARY_ENCODER_FORCE_VALUE_SOURCE +		Sets the signal source to use to set the rotary decoder's counter to 0xFFFFFFFF. The rotary decoder will set the counter to this value on the rising edge of a signal transition of the selected source. INQ									j	k	l	m		
	M_DEFAULT	Specifies to use the source set in the DCF.									j	k	l	m		
	M_NULL	Specifies not to set the counter to 0xFFFFFFFF upon a signal.									j	k	l	m		

	M_AUX_IO*n*	Specifies to use an auxiliary input signal or an auxiliary bidirectional signal set to input (using M_IO_MODE set to M_INPUT).										j	k	l	m		
		System specific															
		Only auxiliary input signals M_AUX_IO2 and M_AUX_IO7 can be specified.										j	k				
	M_COUNTER_OVERFLOW	Specifies to set the counter to 0xFFFFFFFF and keep it at this value after the counter increments past 0x7FFFFFFF. To reset the counter to 0, use M_ROTARY_ENCODER_POSITION.										j	k	l	m		
	M_POSITION_TRIGGER	Specifies to use the trigger signal generated by the rotary decoder when the counter reaches the value specified with M_ROTARY_ENCODER_POSITION_TRIGGER.										j	k	l	m		
	M_STEP_BACKWARD_WHILE_POSITIVE	Specifies to set the counter to 0xFFFFFFFF upon a decrement, only if the counter value is in the range of 0x0 to 0x7FFFFFFF before the decrement occurs; when interpreting the counter value as signed, this would be the positive counter value range. A typical use of this control value is to identify the number of steps taken backwards by the rotary encoder, so that the corresponding lines of data are not regrabbed. For a complete example, refer to the Using the rotary decoder's output to trigger a timer or a grab subsection of the Using quadrature input from a rotary encoder section of Chapter 56: I/O signals and communicating with external devices.										j	k	l	m		
	M_TL_TRIGGER	Specifies to use a transport layer trigger input signal.												l	m		
M_ROTARY_ENCODER_FRAME_END_READ +		Sets whether to enable the rotary decoder to store the counter value at the end of the last grab or frame interrupt. If enabled, the value of the counter can be inquired at anytime using MdigInquire() with M_ROTARY_ENCODER_FRAME_END_POSITION, or retrieved at the end of the last grab from a hooked function, using MdigGetHookInfo() with M_ROTARY_ENCODER_FRAME_END_POSITION. INQ										j	k	l	m		
	M_DEFAULT	Same as M_DISABLE.										j	k	l	m		
	M_DISABLE	Specifies that the counter value will not be available to be inquired at the end of the last frame grabbed.										j	k	l	m		
	M_ENABLE	Specifies to enable the rotary decoder to store the counter value at the end of the last grabbed frame, so that it can be inquired.										j	k	l	m		
M_ROTARY_ENCODER_MULTIPLIER		Sets the multiplying factor to apply to each increment/decrement of the rotary decoder's counter for every rotary encoder step (change in position); this in turn applies a multiplying factor to the number of pulses that the rotary decoder outputs per step. For example, if this control value is set to 4, and M_ROTARY_ENCODER_OUTPUT_MODE is set to M_STEP_ANY, the rotary decoder will increment its counter by 4 and output 4 pulses for every rotary encoder step. INQ										j	k				
	M_DEFAULT	Specifies the default value; the default value is 1.										j	k				
	0 < *Value* <= 4096	Specifies the multiplying factor to use.										j	k				

M_ROTARY_ENCODER_OUTPUT_MODE +	Sets the rotary decoder's counter value and/or the direction of movement upon which the rotary decoder should output a pulse. The pulse can be used to trigger a timer or a grab. To trigger a timer or a grab, set M_TIMER_TRIGGER_SOURCE or M_GRAB_TRIGGER_SOURCE (respectively) to M_ROTARY_ENCODER. To decimate (subsample) the rotary decoder output signal before sending it to a timer or a grab controller, set this control type to M_POSITION_TRIGGER and set M_ROTARY_ENCODER_POSITION_TRIGGER to the required decimation value. For more information, refer to the Pixel aspect ratio subsection of the Using quadrature input from a rotary encoder section of Chapter 56: I/O signals and communicating with external devices. INQ										j	k	l	m		
M_DEFAULT	Same as DCF.										j	k	l	m		
M_POSITION_TRIGGER	Specifies to output a pulse upon the trigger generated by M_ROTARY_ENCODER_POSITION_TRIGGER.										j	k	l	m		
M_POSITION_TRIGGER_MULTIPLE	Specifies to start triggering only when the counter value set with M_ROTARY_ENCODER_POSITION_START_TRIGGER is reached and then to generate a trigger at a set interval (change) in counter value. Set the interval with M_ROTARY_ENCODER_POSITION_START_TRIGGER.										j	k	l	m		
M_STEP_ANY	Specifies to output a pulse upon any change in the rotary decoder's counter value (position change in any direction).										j	k	l	m		
M_STEP_ANY_WHILE_POSITIVE	Specifies to output a pulse upon any change in the rotary decoder's counter value (position change in any direction), only if the counter value is in the range of 0x0 to 0x7FFFFFFF before the increment or decrement occurs; when interpreting the counter value as signed, this would be the positive counter value range. Note that, if the counter value is treated as a signed integer and the counter is 0x7FFFFFFF, the next incremented value is 0x80000000 (falling into the negative counter value range). If you want to remain in the positive range, reset the counter to 0 using either M_ROTARY_ENCODER_POSITION or M_ROTARY_ENCODER_RESET_SOURCE.										j	k	l	m		
M_STEP_FORWARD	Specifies to output a pulse upon a rotary decoder counter increment only.										j	k	l	m		
M_STEP_FORWARD_WHILE_POSITIVE	Specifies to output a pulse upon a rotary decoder counter increment, only if the counter value is in the range of 0x0 to 0x7FFFFFFF before the increment occurs; when interpreting the counter value as signed, this would be the positive counter value range. Note that, if the counter value is treated as a signed integer and the counter is 0x7FFFFFFF, the next incremented value is 0x80000000 (falling into the negative counter value range). If you want to remain in the positive range, reset the counter to 0 using either M_ROTARY_ENCODER_POSITION or M_ROTARY_ENCODER_RESET_SOURCE.										j	k	l	m		
M_ROTARY_ENCODER_POSITION +	Resets the rotary decoder's counter to 0 immediately. To reset the counter to 0 upon a signal, use M_ROTARY_ENCODER_RESET_SOURCE. Note that a call to MdigInquire() with M_ROTARY_ENCODER_POSITION inquires the current value of the rotary decoder counter.										j	k	l	m		
Value = 0	Implements the default behavior. Note that, if a non-zero value is specified, an error is generated.										j	k	l	m		

M_ROTARY_ENCODER_POSITION_START_TRIGGER +	Sets the rotary decoder's counter value upon which to start generating triggers, when M_ROTARY_ENCODER_OUTPUT_MODE is set to M_POSITION_TRIGGER_MULTIPLE. If it is not set to this value, then M_ROTARY_ENCODER_POSITION_START_TRIGGER is ignored. INQ									j	k	l	m		
0 <= *Value* <= 0*xFFFFFFFF*	Specified the value of the counter upon which to start generating triggers.									j	k	l	m		
M_ROTARY_ENCODER_POSITION_TRIGGER +	Sets the rotary decoder's counter value upon which a trigger is generated. You can hook a function to the trigger generated, using MdigHookFunction() with M_ROTARY_ENCODER. You can also output this trigger to a timer or a grab controller using M_ROTARY_ENCODER_OUTPUT_MODE set to M_POSITION_TRIGGER. INQ									j	k	l	m		
0 <= *Value* <= 0*xFFFFFFFF*	Specifies the value of the counter upon which a trigger is generated. If a value beyond the supported range is specified, an error is generated. If you are treating the counter values as a signed range of values (for example, forcing the counter to reset to 0 at 0x80000000) and you want to generate a trigger upon a negative value, specify the equivalent value in the range of 0x80000000 to 0xFFFFFFFF. If M_ROTARY_ENCODER_OUTPUT_MODE is set to M_POSITION_TRIGGER_MULTIPLE, this will specify the change in counter value at which to generate a trigger, beginning at the counter value specified by M_ROTARY_ENCODER_POSITION_START_TRIGGER. If M_ROTARY_ENCODER_OUTPUT_MODE is set to M_POSITION_TRIGGER, this will specify the counter value at which to generate a trigger each time this value is reached. If M_POSITION_TRIGGER is set to any other control value, this value will be ignored.									j	k	l	m		
M_ROTARY_ENCODER_RESET_SOURCE +	Sets the signal source to use to reset the rotary decoder's counter to 0. The rotary decoder will reset the counter on the rising edge of a signal transition of the selected source. Alternatively, you can immediately reset the counter to 0 (without using a signal to trigger the reset), using M_ROTARY_ENCODER_POSITION. INQ									j	k	l	m		
M_DEFAULT	Specifies to use the source set in the DCF.									j	k	l	m		
M_NULL	Specifies not to reset using a hardware signal source.									j	k	l	m		
M_AUX_IO*n*	Specifies to use an auxiliary input signal or an auxiliary bidirectional signal set to input (using M_IO_MODE set to M_INPUT).									j	k	l	m		
	System specific														
	Only auxiliary input signals M_AUX_IO2 and M_AUX_IO7 can be specified.									j	k				
M_POSITION_TRIGGER	Specifies to use the trigger signal generated by the rotary decoder when the counter reaches the value specified with M_ROTARY_ENCODER_POSITION_TRIGGER.									j	k	l	m		
M_TL_TRIGGER	Specifies to use a transport layer trigger input signal.											l	m		

Combination value for M_ROTARY_ENCODER_DIRECTION; M_ROTARY_ENCODER_FORCE_VALUE_SOURCE; M_ROTARY_ENCODER_FRAME_END_READ; M_ROTARY_ENCODER_OUTPUT_MODE; M_ROTARY_ENCODER_POSITION;

M_ROTARY_ENCODER_POSITION_START_TRIGGER; M_ROTARY_ENCODER_POSITION_TRIGGER; M_ROTARY_ENCODER_RESET_SOURCE.

You can add the following value to the above-mentioned values to specify which rotary decoder to set.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying which rotary decoder to set																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ROTARY_ENCODER*n*	Specifies to set rotary decoder n, where *n* is a number between 1 and 4. Note that if you do not use this combination constant to specify a rotary decoder, the rotary decoder corresponding to 1+ the device number of the specified digitizer (DigId) will be used. For example, when the device number of the specified digitizer is M_DEV2, rotary decoder M_ROTARY_ENCODER3 is used.										j		l	m		

The following control types allow you to control the settings for one of the data latches of your installed Aurora Imaging Library board. Data latches store information specific to a grabbed frame when the latch is triggered; the data latch is reset at the end of every grabbed frame. To retrieve information from a data latch, use MdigGetHookInfo() with M_DATA_LATCH.... Note that data latch information is only available when MdigGetHookInfo() is called from a function hooked to an end of grabbed frame event using MdigHookFunction() with M_GRAB_FRAME_END or from the hook-handler function (callback function) of MdigProcess().

> Note that you can only use data latches in association with the Aurora Imaging Library digitizer allocated using MdigAlloc() with M_DEV0.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling a data latch																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_DATA_LATCH_MODE +		Configures whether to latch data from the point when the grab is queued until the start of the first frame of a series of queued grabs or grab sequence. If calls to MdigGrab() occur while no other grab is currently being performed and M_DATA_LATCH_MODE is set to M_PREFETCH, events that occur, from the moment the grab command is issued until the start of the grabbed frame, can also trigger a data latch and be retrieved at the end of the grabbed frame, for each MdigGrab() call. INQ You must specify a combination value from the following table: • To specify which data latch to affect															
	M_DEFAULT	Same as M_NORMAL.										j	k	l	m		
	M_NORMAL	Specifies not to latch before the start of the first grabbed frame.										j	k	l	m		
	M_PREFETCH	Specifies to latch data before the start of the first grabbed frame.										j	k	l	m		
M_DATA_LATCH_STATE +		Sets the state of the specified data latch. INQ You must specify a combination value from the following table: • To specify which data latch to affect										j	k	l	m		
	M_DEFAULT	Same as M_DISABLE.										j	k	l	m		
	M_DISABLE	Disables the data latch.										j	k	l	m		
	M_ENABLE	Enables the data latch.										j	k	l	m		

M_DATA_LATCH_TRIGGER_ACTIVATION +		Sets the trigger signal transition upon which to store the specified information to the specified data latch. Note that this is only useful when M_DATA_LATCH_TRIGGER_SOURCE is set to M_AUX_IOn or M_TIMER_ACTIVE. To set the signal with which to trigger the data latch, use M_DATA_LATCH_TRIGGER_SOURCE. INQ You must specify a combination value from the following table: • To specify which data latch to affect									j	k	l	m		
	M_DEFAULT	Same as M_EDGE_RISING.									j	k	l	m		
	M_EDGE_FALLING	Specifies to store the specified information to the data latch upon a high-to-low signal transition.									j	k	l	m		
	M_EDGE_RISING	Specifies to store the specified information to the data latch upon a low-to-high signal transition.									j	k	l	m		
M_DATA_LATCH_TRIGGER_SOURCE +		Sets what triggers storing the specified information to the specified data latch. Use M_DATA_LATCH_TYPE to specify the type of information to store. INQ You must specify a combination value from the following table: • To specify which data latch to affect									j	k	l	m		
	M_DEFAULT	Same as M_GRAB_FRAME_END.									j	k	l	m		
	M_AUX_IO*n*	Specifies to use auxiliary signal *n* as the trigger source, where *n* is the auxiliary signal number. Note that the specified auxiliary signal can also be a bidirectional signal set to input (using M_IO_MODE set to M_INPUT). For a list of the available auxiliary I/O signals, see the Connectors and signal names section of the Aurora Imaging Library Hardware-specific Notes chapter for your Zebra imaging board or Aurora Imaging Library driver. To specify the signal transition, use M_DATA_LATCH_TRIGGER_ACTIVATION.									j	k	l	m		
	M_GRAB_FRAME_END	Specifies to trigger the data latch on the event that occurs at the end of each grabbed frame; this event occurs when the frame grabber receives the last pixels (or packet) of a frame from the camera. Note that, in this case, the grabbed data might not be completely transferred to the Host.									j	k	l	m		
	M_GRAB_FRAME_START	Specifies to trigger the data latch on the event that occurs at the start of each grabbed frame.									j	k	l	m		
	M_GRAB_LINE	Specifies to trigger the data latch on the event that occurs at the end of each grabbed line; this event occurs when the frame grabber receives the last pixels (or packet) of a line from the camera. Note that, in this case, the grabbed data might not be completely transferred to the Host.									j	k	l	m		
	M_ROTARY_ENCODER*n*	Specifies to use rotary decoder *n* as the trigger source, where *n* is the rotary decoder number. To configure the rotary decoder, use the M_ROTARY_ENCODER... control types. To establish when the rotary decoder issues a trigger, use M_ROTARY_ENCODER_POSITION_TRIGGER.									j	k	l	m		

	M_TIMER_ACTIVE +	Specifies to trigger the data latch on the event that occurs when the specified timer is active. This can be at the start of the active period or the end of the active period, depending on how you set M_DATA_LATCH_TRIGGER_ACTIVATION. You must specify a combination value from the following table: • To specify which on-board timer to control									j	k	l	m		
M_DATA_LATCH_TYPE +		Sets the type of information to store in the specified data latch. Note that each type of information can only be associated with one data latch, except for timestamp, which can be associated with up to four data latches. Aurora Imaging Library returns an error when this limitation is not respected. INQ You must specify a combination value from the following table: • To specify which data latch to affect									j	k	l	m		
	M_IO_STATUS_ALL	Stores the status of all the auxiliary I/O signals.									j	k	l	m		
	M_ROTARY_ENCODER*n*	Stores the value of the counter of rotary decoder *n*, where *n* is a number between 1 and 4. To configure the rotary decoder, use the M_ROTARY_ENCODER... control types.									j	k	l	m		
	M_TIME_STAMP	Stores the timestamp upon which the data latch is triggered, in ticks. Up to four separate data latches can store a timestamp. To convert a timestamp from clock ticks to seconds, use the following equation: *Timestamp* * (*TimestampFrequencyInHz*). To inquire the clock frequency, use MdigInquire() with M_DATA_LATCH_CLOCK_FREQUENCY.									j	k	l	m		

Combination value for the values listed in For controlling a data latch.

You must add the following value to the above-mentioned values to specify which data latch to affect.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data latch to affect																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_LATCH*n*	Specifies which data latch to affect, where n is a value from 0 to 15.										j	k	l	m		

Combination values for the values listed in For controlling the settings of a timer; and for the following value: [when ControlType=M_DATA_LATCH_TRIGGER_SOURCE: M_TIMER_ACTIVE].

You must add one of the following values to the above-mentioned values to specify which on-board timer to control.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying which on-board timer to control																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TIMER_STROBE	Specifies the strobe timer.								i							
M_TIMER*n*	Specifies on-board timer *n*, where *n* is one of the timers available on your Zebra imaging board or the hardware associated with your Aurora Imaging Library driver. Note that, when dealing with timers (excluding M_DATA_LATCH_TRIGGER_SOURCE with M_TIMER_ACTIVE) only a continuous timer can clock another timer. In addition, references where timer 1 points to timer 2, and timer 2 points back to timer 1, are not supported and will generate an error.			e		g					j	k	l	m	n	
	System specific															
	n can be a value from 1 to 4.										j	k	l	m		

		With GigE Vision, USB3 Vision, and GenTL, *n* can be a value from 0 to 8.			e		g									n	

Type-specific versions of the function when using a C compiler under 64-bit

```
void MdigControlDouble (AIL_ID DigId, AIL_INT64 ControlType, AIL_DOUBLE ControlValue)
```

Details

Parameters

DigId

See DigId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void MdigControlInt64 (AIL_ID DigId, AIL_INT64 ControlType, AIL_INT64 ControlValue)
```

Details

Parameters

DigId

See DigId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MdigControlFeature

Synopsis

Controls a feature of the camera.

Syntax

```
void MdigControlFeature(
    AIL_ID DigId,                  //out
    AIL_INT64 ControlType,         //in
    AIL_CONST_TEXT_PTR FeatureName, //in
    AIL_INT64 UserVarType,         //in
    const void *UserVarPtr         //in
)
```

Description

This function allows you to directly control various GenICam standard feature naming convention (SFNC) features and manufacturer-specific features of the camera. For more information, refer to the Using Aurora Imaging Library with GenICam section of Chapter 27: Grabbing with your digitizer.

[Zebra Radient eV-CL; Zebra Rapixo CL]

For Camera Link boards to work with this function, you must first configure and enable the GenICam CLProtocol module, using MdigControl() with M_GC_CLPROTOCOL. Note that this control type is only available after the required third-party vendor-supplied, standard compliant, CLProtocol library for your Camera Link camera is installed on your computer and M_GC_CLPROTOCOL_DEVICE_ID is set.

The features described in this function are primarily available to set existing settings using Aurora Imaging Library code or to set information from which you can build your own interface for the camera device's description file (XML). Aurora Imaging Library provides two versions of an interface that you can use to interactively inquire the camera's features. At design-time, you can use Aurora Imaging Intellicam's Feature Browser. At runtime, you can launch the Feature Browser, using MdigControl() with M_GC_FEATURE_BROWSER. Both versions of the Feature Browser provide a list of available feature names and associated possible values, as well as code snippets with the Aurora Imaging Library functions and Aurora Imaging Library constants associated with the settings selected; you can copy the code snippets to your Aurora Imaging Library application code.

To inquire a camera's manufacturer-specific feature, use MdigInquireFeature().

Parameters

DigId

Specifies the identifier of the digitizer accessing the camera to control. This parameter must be given a valid digitizer identifier, previously allocated using MdigAlloc().

ControlType

Specifies the type of information to control for the specified feature.

See the Parameter associations section for possible values that can be specified.

FeatureName

Specifies the name of the camera feature to control.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the name of the feature																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
AIL_TEXT("*FeatureName*") [1]	Specifies the name of the feature. Note that the feature name is case-sensitive. Refer to your camera's documentation for a list of the features available.			e	f	g					j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

UserVarType

Specifies the data type of the address pointed to by the UserVarPtr parameter. If not setting the feature's value (using M_FEATURE_VALUE), set this parameter to **M_DEFAULT**.

Note that, in the case of a string (or a multi-byte register), the size of the array passed to the UserVarPtr parameter must be equal to or larger

than the returned information; otherwise an application crash will occur.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the UserVarPtr's data type																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TYPE_BOOLEAN	Specifies that UserVarPtr is passed an address of type *AIL_BOOL*.			e	f	g					j	k	l	m	n	b
M_TYPE_DOUBLE	Specifies that UserVarPtr is passed an address of type *AIL_DOUBLE*.			e	f	g					j	k	l	m	n	b
M_TYPE_INT64	Specifies that UserVarPtr is passed an address of type *AIL_INT64*.			e	f	g					j	k	l	m	n	b
M_TYPE_STRING +	Specifies that UserVarPtr is passed an address of type *AIL_TEXT_CHAR*. You must specify a combination value from the following table: • To specify the length of the array			e	f	g					j	k	l	m	n	b
M_TYPE_UINT8 +	Specifies that UserVarPtr is passed an address of type *AIL_UINT8*. You must specify a combination value from the following table: • To specify the length of the array			e	f	g					j	k	l	m	n	b

Combination value for M_TYPE_STRING; M_TYPE_UINT8.

You must add the following value to the above-mentioned values to specify the length of the array.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the length of the array																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FEATURE_USER_ARRAY_SIZE(AIL_INT Length)	Specifies the length of the array.			e	f	g					j	k	l	m	n	b
	Parameters															
	Length Sets the length, in bytes.															

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- array of type AIL_UINT8 [optionally, in C++: a reference to a constant std::vector<AIL_UINT8>]
- AIL_BOOL
- AIL_DOUBLE
- AIL_INT
- AIL_INT64
- M_NULL

Specifies the address of the variable in which to write the value of the feature.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **ControlType** and **UserVarPtr** parameters.

- For specifying the type of information about the feature to set and the data type returned

To determine the data type of the FeatureName specified, use MdigInquireFeature() with M_FEATURE_VALUE.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of information about the feature to set and the data type returned

ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr																
M_FEATURE_CHANGE_HOOK +	Sets whether to enable an event to occur when the value of the specified feature changes. Once enabled, use MdigHookFunction() with M_FEATURE_CHANGE to hook a specified function to the feature change event. Repeat for each feature that you want to enable a feature change event. Alternatively, to enable an event to occur when the value of any feature changes and at the same time hook a function to any feature change, use MdigHookFunction() with M_FEATURE_CHANGE + M_ALL. INQ															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g									n	b
M_DISABLE	Specifies that no event is generated when the value of the specified feature changes. This is the default value.			e	f	g									n	b
M_ENABLE	Specifies that an event is generated when the value of the specified feature changes.			e	f	g									n	b
M_FEATURE_EXECUTE +	Sets that the specified command feature must be executed.			e	f	g					j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *Set this parameter to M_NULL*			e	f	g					j	k	l	m	n	b
M_FEATURE_MAX +	Specifies to set the feature to its maximum value.			e	f	g					j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *Set this parameter to M_NULL*			e	f	g					j	k	l	m	n	b
M_FEATURE_MIN +	Specifies to set the feature to its minimum value.			e	f	g					j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *address of an AIL_INT64*			e	f	g					j	k	l	m	n	b
M_FEATURE_VALUE +	Sets the current value of the feature.			e	f	g					j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_BOOL* **|** *address of an AIL_DOUBLE* **|** *address of an AIL_INT64* **|** *array of type AIL_UINT8 [optionally, in C++: a reference to a constant std::vector<AIL_UINT8>]* **|** *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g					j	k	l	m	n	b

Combination values for the values listed in For specifying the type of information about the feature to set and the data type returned.

You can add one of the following values to the above-mentioned values to specify the GenTL configuration file (XML file) that is associated with the feature.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the configuration file associated with the feature

Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_GENTL_STREAM_NUMBER(AIL_INT IndexValue)	Specifies the feature is associated with an instance of the GenTL stream configuration information.			e												
	Parameters															
	IndexValue Specifies the instance (index number) of the GenTL library stream configuration information file associated with the feature.			e												
	0 <= *Value* <= M_GENTL_STREAM_COUNT — Specifies the index.			e												
M_GENTL_DEVICE	Specifies that the feature is associated with the GenTL device configuration information.			e												

M_GENTL_REMOTE_DEVICE	Specifies that the feature is associated with the GenTL remote device configuration information. This is the default value.			e													

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MdigFocus

Synopsis

Adjust a camera's lens motor to a position that produces optimum focus, or only calculate the focus quality of a specified image or an image grabbed at the current lens motor position.

Syntax

```
void MdigFocus(
    AIL_ID DigId,                           //in
    AIL_ID DestImageBufId,                  //out
    AIL_ID FocusImageRegionBufId,           //out
    AIL_FOCUS_HOOK_FUNCTION_PTR FocusHookPtr, //in
    void *UserDataPtr,                      //in-out
    AIL_INT MinPosition,                    //in
    AIL_INT StartPosition,                  //in
    AIL_INT MaxPosition,                    //in
    AIL_INT MaxPositionVariation,           //in
    AIL_INT64 Operation,                    //in
    AIL_INT *ResultPtr                      //out
)
```

Description

This function adjusts a camera's lens motor to a position that produces optimum focus, or only calculates an indicator of the focus quality (focus indicator) of a specified image or an image grabbed at the current lens motor position. The operation mode establishes which is performed.

To adjust a camera's lens motor to a position that produces optimum focus, this function calls a user-defined function to move the lens motor to a specified initial position, grabs an image, calculates the focus indicator of the grabbed image, and based on the focus indicator and the specified optimum focus search strategy, calculates a new position for the lens motor. The process repeats, calling the user-defined function to move the lens motor to the newly calculated position and comparing the new focus indicator with the old. The process continues repeating until the optimum focus position is found. You can optionally have the user-defined function perform the grab, instead of having **MdigFocus**() directly perform the grab.

Regardless of the operation mode, the image is subsampled by a factor of 4, by default using a bilinear interpolation operation, before the focus indicator is calculated; you can override this default.

This function establishes the focus indicator of an image by analyzing the edges within a specified focus region (a child image of the grabbed or provided image). An image with a good focus indicator has a sharp intensity transition between its edges and their background. The resulting optimal focus position is an position between the specified minimum and maximum lens position.

Alternatively, this function can evaluate the current position and calculate the focus indicator (using the M_EVALUATE operation mode). In this case, the focus indicator ranges from 0 to a very large number, based on the number of edges found in the grabbed image.

In either case, as the returned value increases, your camera's focus also increases, resulting in future grabbed image becoming more in focus.

The specified search strategy (set using the operation mode parameter) determines how each new position of the lens motor is calculated between grabs. For more information, see the Auto-focusing section of Chapter 27: Grabbing with your digitizer.

Parameters

DigId

Specifies the identifier of the digitizer. If you want to provide the image (that is, either have the user-defined function perform the grab, or use an existing image) set this parameter to **M_NULL**. The provided image must be stored in the destination image buffer.

DestImageBufId

Specifies the identifier of the buffer that contains the image to analyze or in which to grab. This buffer should be of an appropriate type to hold the image. This buffer must be a 1- or 3-band buffer, but cannot be a signed 3-band buffer, a 32-bit buffer, or a 1-bit buffer.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

FocusImageRegionBufId

Specifies the identifier of a child buffer of the destination buffer. Analysis of each grabbed image is limited to the region specified by this buffer. To analyze the entire image, set this parameter to **M_DEFAULT**.

You should pass a child buffer when there are objects at different distances within the camera's field of view; this is because in this case, each object will have a different optimum focus position.

This child buffer must be a 1- or 3-band buffer and must not have a region of interest (ROI) associated with it. Using a child buffer with an ROI

will cause an error.

FocusHookPtr

Specifies the address of the user-defined function to call before each grab. When only calculating the focus indicator (that is, using the M_EVALUATE operation mode), this parameter is not used, and must be set to **M_NULL**.

The user-defined function must be declared as follows:

```
AIL_INT MFTYPE FocusHook(
    AIL_INT HookType,
    AIL_INT Position,
    void *UserDataPtr
)
```

Parameters

HookType

Indicates whether the optimum focus position was found or whether a new position needs to be analyzed.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining whether the optimum focus position was found or a new position needs to be analyzed																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CHANGE	Specifies that the lens position must change to the position specified by the Position parameter, and a new position needs to be analyzed.	c		e	f	g			i		j	k	l	m	n	b
M_ON_FOCUS	Specifies that the optimum focus position was found and is specified by the Position parameter.	c		e	f	g			i		j	k	l	m	n	b

Position

Focus position.

UserDataPtr

Pointer to data.

Upon successful completion, the user-defined function should return **M_NULL**. Otherwise, the user-defined function should return **M_STOP_FOCUS** to abort the **MdigFocus**() operation. In this case, the value returned by **MdigFocus**() will be undetermined.

UserDataPtr

Specifies the address of the data that you want to make available to the user-defined function. When only calculating the focus indicator (that is, using the M_EVALUATE operation mode), this parameter is not used, and must be set to M_NULL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the address of the data																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that the function does not require data.	c		e	f	g			i		j	k	l	m	n	b
Value	Specifies the address of the data. This address is passed to the user-defined function, through its UserDataPtr parameter, when the user-defined function is called.	c		e	f	g			i		j	k	l	m	n	b

MinPosition

Specifies the minimum focus position of the search. When only calculating the focus indicator (that is, using the M_EVALUATE operation mode), this parameter is not used, and must be set to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the minimum focus position of the search																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g			i		j	k	l	m	n	b
Value >= 0	Specifies the position, in lens motor steps.	c		e	f	g			i		j	k	l	m	n	b
	System specific															
	For a smart camera with the Variopic Caspian C-39NO-160-I2C lens, the valid range of focus positions is from 0 to 1023. If using an auto-focus lens from another manufacturer, to determine the hardware-specific minimum focus position of the search, use MdigInquire() with M_FOCUS + M_MIN_VALUE.								i							

StartPosition

Specifies the starting focus position of the search. When only calculating the focus indicator (that is, using the M_EVALUATE operation mode), this parameter is not used, and must be set to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the starting focus position of the search																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value; the default value is MinPosition.	c		e	f	g			i		j	k	l	m	n	b
MinPosition <= *Value* <= *MaxPosition*	Specifies the position, in lens motor steps.	c		e	f	g			i		j	k	l	m	n	b
	System specific															
	For a smart camera with the Variopic Caspian C-39NO-160-I2C lens, the valid range of focus positions is from 0 to 1023. If using an auto-focus lens from another manufacturer, to determine the hardware-specific minimum and maximum focus positions of the search, use MdigInquire() with M_FOCUS + M_MIN_VALUE and M_FOCUS + M_MAX_VALUE, respectively.								i							

MaxPosition

Specifies the maximum focus position of the search. When only calculating the focus indicator (that is, using the M_EVALUATE operation mode), this parameter is not used, and must be set to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the maximum focus position of the search																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value; the default value is 255.	c		e	f	g			i		j	k	l	m	n	b
Value > *MinPosition*	Specifies the position, in lens motor steps.	c		e	f	g			i		j	k	l	m	n	b
	System specific															
	For a smart camera with the Variopic Caspian C-39NO-160-I2C lens, the valid range of focus positions is from 0 to 1023. If using an auto-focus lens from another manufacturer, you can determine the hardware-specific maximum focus position of the search, use MdigInquire() with M_FOCUS + M_MAX_VALUE.								i							

MaxPositionVariation

Specifies the positional increment to use in the search. When only calculating the focus indicator (that is, using the M_EVALUATE operation mode), or when performing an M_SCAN_ALL or M_REFOCUS operation, this parameter is not used, and must be set to M_DEFAULT.

When performing a smart scan operation (using M_SMART_SCAN), this parameter is used during the first pass. The second pass uses the square root of this value. Each subsequent pass uses a progressively smaller value, with the last pass using a value of 1.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the positional increment

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value. The default value is equal to 1/16 of the value specified by MaxPosition.	c		e	f	g			i		j	k	l	m	n	b
Value > 0	Specifies the position, in lens motor steps.	c		e	f	g			i		j	k	l	m	n	b

Operation

Specifies the operation mode used by this function.

Each operation mode provides a different strategy to find the optimum focus position. These strategies determine how the position is updated (in which direction and by how much) between grabs. They can affect the speed and accuracy of the operation. The default search strategy used by Aurora Imaging Library is the smart-scan strategy.

The operation mode can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the optimum focus position search strategy																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_SMART_SCAN.	c		e	f	g			i		j	k	l	m	n	b
M_BISECTION +	Specifies that the bisection search strategy will be used. This strategy breaks down the given positional range, step-by-step, until it finds the optimum focus position. In general, the bisection strategy processes the fewest amount of images. However, it is most sensitive to noise and requires that the lens motor travel the greatest distance.	c		e	f	g			i		j	k	l	m	n	b
M_REFOCUS +	Specifies that the refocus search strategy will be used. This strategy scans upward or downward until it finds the optimum focus position or until it reaches the minimum or maximum position. The refocus strategy is the best strategy to use when the current focus position is close to optimum.	c		e	f	g			i		j	k	l	m	n	b
M_SCAN_ALL +	Specifies that the scan-all search strategy will be used. This strategy scans, by 1, all positions between the minimum and maximum. The scan-all strategy is the slowest but most accurate.	c		e	f	g			i		j	k	l	m	n	b
M_SMART_SCAN +	Specifies that the smart-scan search strategy will be used. This strategy performs three refocus searches, each with a smaller positional increment. The smart-scan strategy is a compromise between the speed of a bisection and the accuracy of a scan-all.	c		e	f	g			i		j	k	l	m	n	b

Combination value for M_REFOCUS; M_SMART_SCAN.

You can add the following value to the above-mentioned values to specify the number of focus positions at which to compare the focus indicator when a potential optimum focus position (a peak in focus indicator values) is found.

When a potential optimum focus position is found, the process repeats the specified number of times to examine positions beyond that peak. This verifies that the peak focus indicator is truly the optimum focus position.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the number of focus positions at which to compare the focus indicator																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
1 <= *Value* <= 255	Specifies the required number of focus positions. By default, the value is 2. Note that this number cannot be greater than the difference between the maximum and minimum positions specified by MaxPosition and MinPosition, respectively.	c		e	f	g			i		j	k	l	m	n	b

To calculate only the focus indicator value for a given image or for the image grabbed at the current lens position, set the Operation parameter to the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying that only the focus indicator is calculated																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_EVALUATE +	Specifies to only calculate the focus indicator. The user-defined function and all position parameters are unused (but must be set to the specific values mentioned previously). Instead, this operation mode grabs an image and uses it to calculate the focus indicator value. Note that, if you don't specify a digitizer identifier, the image contained in the specified destination buffer is used.	c		e	f	g			i		j	k	l	m	n	b

Combination values for any of the possible values of the Operation parameter.

You can add one or more of the following values to the above-mentioned values to specify additional information about the mode of operation.

If both M_NO_FILTER and M_NO_SUBSAMPLING are specified, the source image is used directly without filtering or subsampling.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying additional information about the mode of operation																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NO_FILTER	Specifies to not perform a smoothing convolution. Instead, a subsampling operation is performed (using a nearest neighbor interpolation), with a scaling factor of 4.	c		e	f	g			i		j	k	l	m	n	b
M_NO_SUBSAMPLING	Specifies to not perform a subsampling operation (using a nearest neighbor interpolation), with a scaling factor of 4. Instead, a smoothing convolution is performed.	c		e	f	g			i		j	k	l	m	n	b

ResultPtr

Specifies the address in which to return the resulting value from the focusing operation.

If performing a focus operation on a single image (that is, using the M_EVALUATE operation mode), only the focus indicator is returned.

If performing a focus operation on multiple grabbed images (that is, not using the M_EVALUATE operation mode), then the optimum focus position is returned. Alternatively, if you do not want to return the optimum focus position, you can set this parameter to **M_NULL**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MdigFree

Synopsis

Free a digitizer.

Syntax

```
void MdigFree(
    AIL_ID DigId //in
)
```

Description

This function deallocates a digitizer previously allocated with MdigAlloc().

All digitizers allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

Parameter

DigId

Specifies the identifier of the digitizer.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MdigGetHookInfo

Synopsis

Get information about a digitizer hook event.

Syntax

```
AIL_INT MdigGetHookInfo(
    AIL_ID EventId,     //in
    AIL_INT64 InfoType, //in
    void *UserVarPtr    //out
)
```

Description

This function retrieves information about the digitizer event that caused the hook-handler function to be called. **MdigGetHookInfo**() should only be called within the scope of a digitizer hook-handler function (see MdigProcess() and MdigHookFunction()).

Parameters

EventId

Specifies the digitizer event identifier received by the hook-handler function (see MdigProcess() and MdigHookFunction()).

InfoType

Specifies the type of information about the event to return.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT64
- a pointer of type void

Specifies the address in which to write the requested information.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InfoType** parameter and possible values returned to the **UserVarPtr** parameter.

- For retrieving information about a general digitizer hook event
- For retrieving information about a GenICam SFNC-compliant event
- For retrieving information about a grab event
- For retrieving information about a timer
- For retrieving information about an I/O event
- For retrieving information about a reference latch value
- For retrieving information about a rotary decoder event
- For retrieving information from a data latch
- For retrieving information from a GenICam feature change event
- For retrieving information from a grab frame burst event

The following allows you to retrieve information about various digitizer hook events. Unless otherwise specified, you can retrieve these information types if you call this function from within a function hooked to any digitizer event using MdigHookFunction() or MdigProcess().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving information about a general digitizer hook event																		
InfoType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																	

M_DIGITIZER_ID	Retrieves the Aurora Imaging Library identifier of the digitizer that was used by MdigProcess() or MdigHookFunction().														
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
Digitizer identifier	Specifies the Aurora Imaging Library identifier of the digitizer.	c		e	f	g	a		i		j	k	l	m	n	b
M_GC_CAMERA_TIME_STAMP	Retrieves the time at which the event occurred. This value was generated by the camera, using a specialized counter. To inquire the time stamp generated by the operating system, use M_TIME_STAMP.			e	f	g									n	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*			e	f	g									n	
Value	Specifies the camera's time stamp, in secs.			e	f	g									n	
M_GC_EVENT_TYPE	Retrieves the GenICam event type, as sent by the camera. Refer to your GenICam SFNC specification, or your camera's documentation for a list of GenICam event types.			e	f	g									n	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g									n	
Value	Specifies the GigE Vision event type.			e	f	g									n	
M_GC_PACKET_SIZE	Retrieves the size of the data packet, used by the GenICam SFNC-compliant camera when streaming data to the Host.				f	g										
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g										
Value	Specifies the size of the packet, in bytes.				f	g										
M_GRAB_TRIGGER_MISSED	Retrieves the number of grab triggers missed. This result is available only when MdigControl() with M_GRAB_TRIGGER_MISSED is set to M_ENABLE.							i							
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*							i							
Value	Specifies the number of missed triggers.							i							
M_TIME_STAMP	Retrieves the operating system's time stamp at the point which the event generated a service request. The time stamp is generated by the operating system's performance counter. Time stamps can be used to see if a frame has been missed since the last time stamp.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value is generated by the operating system. To use the time stamp generated by the camera, use M_GC_CAMERA_TIME_STAMP.			e	f	g									n	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the operating system's time stamp, in secs.	c		e	f	g	a		i		j	k	l	m	n	b

The following allows you to retrieve information about a GenICam SFNC-compliant event. The following information types are only available if **MdigGetHookInfo**() was called from a function hooked to a GenICam event using M_GC_EVENT. In addition, the GenICam event must be enabled using MdigControlFeature(), and the message channel must be supported by your camera.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving information about a GenICam SFNC-compliant event																
InfoType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																

M_COUNTER_INDEX	Retrieves the index of a counter on your camera. You can retrieve this value if the hook-handler function was hooked using MdigHookFunction() with M_GC_EVENT + M_COUNTER_START + *n* or M_GC_EVENT + M_COUNTER_END + *n*, where *n* is the counter's index. Counters are unique to GenICam SFNC-compliant cameras. To configure your GenICam SFNC-compliant counters, use MdigControlFeature().														
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g								n	
Value	Specifies the index of the counter.			e	f	g								n	
M_LINE_INDEX	Retrieves the index of the I/O signal that generated the GenICam SFNC-compatible camera's event. You can only retrieve this information if the hook-handler function was hooked using MdigHookFunction() with M_GC_EVENT + M_LINE_ANY_EDGE + *n*, M_GC_EVENT + M_LINE_FALLING_EDGE + *n*, or M_GC_EVENT + M_LINE_RISING_EDGE + *n*, where *n* is the I/O signal's number.			e	f	g								n	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g								n	
Value	Specifies the number of the I/O signal that generated the event.			e	f	g								n	

The following allows you to retrieve information about a grab event.

Unless otherwise specified, the following information types are only available if **MdigGetHookInfo**() was called from a function hooked to an event using MdigHookFunction() or MdigProcess() with M_GRAB_..._END. If inquired from any other hook function, these information types will not return an error, but the result will be irrelevant.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving information about a grab event																
InfoType UserVarPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CORRUPTED_FRAME	Retrieves whether the currently grabbed frame is corrupt.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g									n	b
M_NO	Specifies that the frame is not corrupt.	c		e	f	g									n	b
M_YES	Specifies that the frame is corrupt.	c		e	f	g									n	b
M_GC_FRAME_BLOCK_ID	Retrieves the value of the counter (block ID) set by the camera for each grabbed frame. In this case, the value of the counter for the last frame grabbed is returned.			e	f	g									n	b
	System specific															
	Note that a GigE Vision 2.0 camera has a 64-bit frame counter, while a GigE Vision 1.0 camera has a 16-bit frame counter.				f	g										
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g									n	b
Value	Specifies the value of the counter.			e	f	g									n	b
M_GC_FRAME_BLOCK_ID64	Retrieves the value of the 64-bit counter (block ID) set by the camera for each grabbed frame. In this case, the value of the counter for the last frame grabbed is returned.			e	f	g									n	b
	System specific															
	Note that a GigE Vision 2.0 camera has a 64-bit frame counter, while a GigE Vision 1.0 camera has a 16-bit frame counter.				f	g										
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e	f	g									n	b

	Value	Specifies the value of the counter.			e	f	g									n	b
M_GC_FRAME_ERROR_CODE		Retrieves the error code returned by the camera for the last frame grabbed.				f	g									n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g									n	
	Value	Specifies the GigE Vision error code.				f	g									n	
M_GC_FRAME_LINE_COUNT		Retrieves the number of rows in the last frame grabbed.			e	f	g									n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g									n	b
	Value	Specifies the row count.			e	f	g									n	b
M_GC_FRAME_PIXEL_TYPE		Retrieves the data depth and raw color format of the last frame grabbed. Note that this value is a value, specified by GenICam's standard feature naming convention (SFNC). The predefined list of possible pixel formats are in the *PFNC.h* header file in the installed aildyn folder (for example, *C:\Program Files\Aurora Imaging Library\11\include\classic\aildyn*).			e	f	g									n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g									n	b
	Value	Specifies the data depth and raw color format of the last frame grabbed.			e	f	g									n	b
M_GC_FRAME_SIZE_X		Retrieves the width (size X) of the last frame grabbed.			e	f	g									n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g									n	b
	Value	Specifies the width of the last frame grabbed.			e	f	g									n	b
M_GC_FRAME_SIZE_Y		Retrieves the height (size Y) of the last frame grabbed.			e	f	g									n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g									n	b
	Value	Specifies the height of the last frame grabbed, in pixels.			e	f	g									n	b
M_GC_FRAME_STATUS_CODE		Retrieves the status code from the last frame grabbed. The predefined list of frame status codes are in the *GVCP_Status_Codes.h* header file in the installed aildyn folder (for example, *C:\Program Files\Aurora Imaging Library\11\include\classic\aildyn*).				f	g									n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g									n	
	Value	Specifies the status code.				f	g									n	
M_GC_PACKETS_MISSED		Retrieves the number of missing packets in the corrupted frame. If 0 is returned, the frame is not corrupted.					g										
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*					g										
	Value	Specifies the number of missing packets.					g										
M_GC_PACKETS_RECEIVED		Retrieves the number of packets in the frame.				f	g										
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g										
	Value	Specifies the number of packets in the frame.				f	g										
M_GC_PACKETS_RECOVERED		Retrieves the number of packets successfully recovered in the frame.					g										
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*					g										

Value	Specifies the number of recovered packets.					g										
M_GC_PACKETS_RESENDS_NUM	Retrieves the number of resend commands sent to the camera for a frame. Note that one resend command can contain multiple consecutive packets.					g										
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*					g										
Value	Specifies the number of resend commands sent.					g										
M_GRAB_LINE	Retrieves the number of the line (row) that, upon it being received by the digitizer, generated the event. You can only retrieve this information if the hook-handler function was hooked using MdigHookFunction() with M_GRAB_LINE + n, where *n* is the row number. Note that upon this event, the row has not necessarily been written to the grab buffer.								i			k	l	m		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*								i			k	l	m		
Value	Specifies the row number.								i			k	l	m		
M_GRAB_LINE_COUNT	Retrieves the number of lines (rows) grabbed. Note that the data contained in the line is available for processing. This is useful when grabbing from a line-scan camera with frames of variable length.				f	g				j	k	l	m	n		
	System specific															
	You can only retrieve this value if the hook-handler function was hooked using MdigProcess() or MdigHookFunction() with M_GRAB_END or M_GRAB_FRAME_END.					g								n		
	You can only retrieve this value if the row count was enabled using MdigControl() with M_GRAB_LINE_COUNTER and the hook-handler function was hooked using MdigProcess() or MdigHookFunction() with M_GRAB_END or M_GRAB_FRAME_END.									j	k	l	m			
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g				j	k	l	m	n		
Value	Specifies the row number.				f	g				j	k	l	m	n		
M_GRAB_LINE_END	Retrieves the number of the line (row) that, upon it being available in the grab buffer, generated the event. You can only retrieve this information if the hook-handler function was hooked using MdigHookFunction() with M_GRAB_LINE_END + n, where *n* is the row number. Note that upon this event, the row is available in the grab buffer.										k	l	m			
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										k	l	m			
Value	Specifies the row number.										k	l	m			
M_MODIFIED_BUFFER +	Retrieves information about a modified buffer or container type of event. You must specify a combination value from the following table: • To specify the type of information to return about the modified image buffer	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an void*	c		e	f	g	a		i		j	k	l	m	n	b

The following allows you to retrieve information about a timer. Refer to the individual information types for restrictions.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving information about a timer																
InfoType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																

M_TIMER_INDEX	Retrieves the index of the timer that generated the event. You can only retrieve this value if **MdigGetHookInfo**() was called from a function hooked to an event using MdigHookFunction() with M_TIMER_START + M_TIMERn, M_TIMER_END + M_TIMERn, M_GC_EVENT + M_TIMER_START + M_TIMERn, or M_GC_EVENT + M_TIMER_END + M_TIMERn, where *n* is the timer index.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g					j	k	l	m	n	
Value	Specifies the index of the timer that generated the event.			e	f	g					j	k	l	m	n	
M_TIMER_TRIGGER_MISSED +	Retrieves the number of triggers missed by the specified timer. You can only retrieve this value if **MdigGetHookInfo**() was called from a function hooked to an event using MdigProcess(), or MdigHookFunction() with M_GRAB_END, M_TIMER_START + M_TIMERn, or M_TIMER_END + M_TIMERn, where *n* is the timer index. You must specify a combination value from the following table: • To set the timer to inquire										j	k	l	m		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k	l	m		
Value	Specifies the number of missed triggers.										j	k	l	m		

Combination value for M_TIMER_TRIGGER_MISSED.

You must add the following value to the above-mentioned value to set the timer to inquire.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the timer to inquire																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TIMER*n*	Specifies the timer, where *n* can be from 1 to 4.										j	k	l	m		

Combination values for M_MODIFIED_BUFFER.

You must add one of the following values to the above-mentioned value to specify the type of information to return about the modified image buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving information about the modified image buffer																
InfoType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_BUFFER_ID	Retrieves the Aurora Imaging Library identifier of the modified image buffer or container.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
Container identifier	Specifies the Aurora Imaging Library identifier of the modified container associated with the event.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the Aurora Imaging Library identifier of the modified image buffer associated with the event.	c		e	f	g	a		i		j	k	l	m	n	b
M_BUFFER_INDEX	Retrieves the index of the modified image buffer in the image buffer array passed to MdigProcess(), if the hook-handler function was hooked using MdigProcess(). This index always starts at 0.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the index of the modified image buffer.	c		e	f	g	a		i		j	k	l	m	n	b

M_REGION_OFFSET_X	Retrieves the X-offset of the modified region (of the image buffer).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-offset of the modified region.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_OFFSET_Y	Retrieves the Y-offset of the modified region (of the image buffer).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-offset of the modified region.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_SIZE_X	Retrieves the width of the modified region (of the image buffer).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the width of the modified region.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_SIZE_Y	Retrieves the height of the modified region (of the image buffer).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the height of the modified region.	c		e	f	g	a		i		j	k	l	m	n	b

The following allows you to retrieve information about an I/O event. These information types are only available if **MdigGetHookInfo**() was called from a function hooked to event using MdigHookFunction() with M_IO_CHANGE.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving information about an I/O event																
InfoType UserVarPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_IO_INTERRUPT_SOURCE	Retrieves the auxiliary input signal that generated the interrupt.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g					j	k	l	m	n	
M_AUX_IO*n*	Specifies the auxiliary input signal that generated the interrupt, where *n* is the signal number.			e	f	g					j	k	l	m	n	
M_IO_STATUS	Retrieves the state of the input signal that generated the interrupt. The returned value reflects the state of the input signal at the time of the interrupt.			e	f	g					j	k	l	m	n	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g					j	k	l	m	n	
M_OFF	Specifies that the auxiliary input signal that generated the interrupt is off.			e	f	g					j	k	l	m	n	
M_ON	Specifies that the auxiliary input signal that generated the interrupt is on.			e	f	g					j	k	l	m	n	

The following allows you to retrieve information about a reference latch event. The following information types are only available if using a latch that has previously saved the time or counter value upon a hardware event and using MdigHookFunction() with any hook event.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving information about a reference latch value																
InfoType UserVarPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_REFERENCE_LATCH_VALUE +		Retrieves the last timestamp or counter value stored by the specified latch. You must specify a combination value from the following table: • To specify which of the I/O command lists to use														
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*							i							
	Value	Specifies the last timestamp or counter value stored by the specified latch.							i							

Combination value for M_REFERENCE_LATCH_VALUE.

You must add the following value to the above-mentioned value to specify which of the I/O command lists to use.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the I/O command list to use																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_IO_COMMAND_LIST1	Specifies to inquire about I/O command list1.								i							

The following allows you to retrieve information about a rotary decoder event. This information type is only available if **MdigGetHookInfo**() was called from a function hooked to event using MdigHookFunction() with M_ROTARY_ENCODER. In addition, you must have enabled the rotary decoder to store this information (before the grab), using MdigControl() with M_ROTARY_ENCODER_FRAME_END_READ. Note that the current value of the rotary decoder's counter can be inquired using MdigInquire() with M_ROTARY_ENCODER_POSITION.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving information about a rotary decoder event																	
InfoType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_ROTARY_ENCODER_FRAME_END_POSITION +		Retrieves the value of the rotary decoder's counter at the end of the last frame grabbed before the counter reached its trigger position.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k	l	m		
	Value	Specifies the value of the rotary decoder's counter when the last frame was grabbed.										j	k	l	m		

Combination value for M_ROTARY_ENCODER_FRAME_END_POSITION.

You can add the following value to the above-mentioned value to specify which rotary decoder to inquire.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying which rotary decoder to inquire																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ROTARY_ENCODER*n*	Specifies to inquire rotary decoder n, where *n* is a number between 1 and 4. Note that if you do not use this combination constant to specify a rotary decoder, the rotary decoder corresponding to 1+ the device number of the specified digitizer (using MdigHookFunction() with DigId) will be used. For example, when the device number of the specified digitizer is M_DEV2, rotary decoder M_ROTARY_ENCODER3 is used.										j	k	l	m		

The following allows you to retrieve information from a data latch. These information types are only available if **MdigGetHookInfo**() was called from a function hooked to an end of grabbed frame event using MdigHookFunction() with M_GRAB_FRAME_END or from the hook-handler function (callback function) of MdigProcess(). In addition, you must have enabled the data latch to store this information, using MdigControl() with M_DATA_LATCH_STATE.

> Note that you can only use data latches in association with the Aurora Imaging Library digitizer allocated using MdigAlloc() with M_DEV0.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof,

m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving information from a data latch																	
InfoType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_DATA_LATCH_VALUE +		Retrieves the stored data of the specified data latch. If there are multiple instances of this data (for example, your data latch is triggered at the end of each grabbed line), use + M_VALUE_INDEX(). You must specify a combination value from the following table: • To specify which data latch to inquire about															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*										j	k	l	m		
	M_NULL	Specifies that the stored data could not be read.										j	k	l	m		
	Value	Specifies the stored data.										j	k	l	m		
M_DATA_LATCH_VALUE_ALL +		Retrieves all the stored data of the specified data latch. You must specify a combination value from the following table: • To specify which data latch to inquire about										j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*										j	k	l	m		
	Value	Specifies all the stored data.										j	k	l	m		
M_DATA_LATCH_VALUE_COUNT +		Counts the number of items stored inside a data latch. Note that a data latch is reset at every frame event. You must specify a combination value from the following table: • To specify which data latch to inquire about										j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*										j	k	l	m		
	Value	Specifies the number of instances. Note that a data latch is reset at every frame event.										j	k	l	m		

Combination value for the values listed in For retrieving information from a data latch.

You must add the following value to the above-mentioned values to specify which data latch to inquire about.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data latch to inquire																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_LATCH*n*	Specifies the data latch from which to retrieve information, where *n* is a value from 0 to 15.										j	k	l	m		

Combination value for M_DATA_LATCH_VALUE.

You can add the following value to the above-mentioned value to specify the instance of the stored information to retrieve.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For identifying the instance of the stored information to retrieve																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_VALUE_INDEX(AIL_INT64 IndexValue)	Specifies the instance of stored information to retrieve.										j	k	l	m		

		Parameters															
		IndexValue Sets the instance of stored information. For example, to read the information latched when the last line finishes being grabbed for a frame, with 1024 lines, the instance would be 1023.									j	k	l	m			

The following allows you to retrieve information from a GenICam feature change event. These information types are only available if **MdigGetHookInfo**() was called from a function hooked to GenICam feature change event using MdigHookFunction() with M_FEATURE_CHANGE.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving information from a GenICam feature change event																	
InfoType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_GC_FEATURE_CHANGE_NAME +		Retrieves the name of the GenICam feature that changed. This information type is only available if **MdigGetHookInfo**() is used from a function hooked to GenTL feature change events using MdigHookFunction() with M_FEATURE_CHANGE.															
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g					j	k	l	m	n	
	Value	Specifies the feature name.			e	f	g					j	k	l	m	n	

Combination value for M_GC_FEATURE_CHANGE_NAME.

You can add the following value to the above-mentioned value to get the string's length.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For getting the string size																	
InfoType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_STRING_SIZE		Retrieves the length of the string, including the terminating null character ("\0").															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g					j	k	l	m	n	

The following allows you to retrieve information about events that occur when grabbing into a multi-frame buffer (frame burst events). Unless otherwise specified, you can retrieve these information types if you call this function from within a function hooked to a frame end event using MdigHookFunction() with M_GRAB_FRAME_END or MdigProcess().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving information from a grab frame burst event																	
InfoType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_GRAB_FRAME_BURST_COUNT		Retrieves the number of frames grabbed into the multi-frame buffer.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*											k	l	m		
	Value	Specifies the number of frames grabbed.											k	l	m		
M_GRAB_FRAME_BURST_END_SOURCE		Retrieves the type of event(s) that ended the grab of sequential frames into the multi-frame buffer (end of the frame burst). Simultaneous events might have caused the end of grab to happen, so a combination of the values below can be returned. Bitwise operators must be used to verify the presence of a specific returned value.											k	l	m		

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*											k	l	m		
	M_BURST_COUNT	Specifies that the entire group of sequential frames have been grabbed into the multi-frame buffer. To specify the number of frames in a frame burst, use MdigControl() with M_GRAB_FRAME_BURST_SIZE.											k	l	m		
	M_BURST_MAX_TIME	Specifies that the maximum frame burst time has elapsed. To specify the maximum time for a frame burst to complete, use MdigControl() with M_GRAB_FRAME_BURST_MAX_TIME.											k	l	m		
	M_BURST_TRIGGER	Specifies that an end-of-frame-burst trigger occurred. To enable using an end-of-frame-burst trigger to end the grab, use MdigControl() with M_GRAB_FRAME_BURST_END_TRIGGER_STATE.											k	l	m		

Return value

The returned value is **M_NULL** if successful. If the operation fails, a non-null (!**M_NULL**) value is returned.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MdigGrab

Synopsis

Grab data from a camera into a buffer or container.

Syntax

```
void MdigGrab(
    AIL_ID DigId,                  //in
    AIL_ID DstContainerOrImageBufId //out
)
```

Description

This function uses the specified digitizer to acquire a frame of data and stores this data in the destination image buffer or container. When grabbing an image, you should use an image buffer as a destination. When grabbing any other type of data (such as 3D data), you should use a container as a destination. You can determine what type of Aurora Imaging Library object the digitizer is designed to grab into using MdigInquire() with M_TARGET_BUFFER_OBJECT.

By default, when you call **MdigGrab**(), the digitizer will grab the next valid frame. You can have the digitizer wait for a trigger signal to control when a frame of data is actually grabbed; to do so, set your digitizer to triggered mode using MdigControl() with M_GRAB_TRIGGER_STATE. For more information, see the Grabbing with triggers section of Chapter 27: Grabbing with your digitizer.

When acquiring data from a line-scan type of camera, the exact number of rows grabbed is determined by the DCF.

[Host system]

When grabbing from a video or directory of images, once the last image is grabbed, grabbing will restart from the beginning of the video or from the first image in the directory.

If you need to continuously grab and process the grabbed data, without dropping frames, use MdigProcess(). You can also use MdigGrabContinuous() to continuously acquire frames of data for use with a 2D or 3D display.

Parameters

DigId

Specifies the identifier of the digitizer.

DstContainerOrImageBufId

Specifies the identifier of the destination image buffer or container.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the identifier of the destination image buffer																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Color image buffer identifier	Specifies the identifier of a color destination image buffer with an M_IMAGE + M_GRAB attribute. This image buffer must have been previously allocated, typically using MbufAllocColor(). This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. If the grab buffer is smaller than the digitizer's frame size, the image buffer will be filled and all other data will be lost. If the grab buffer is larger than the digitizer's frame size, the buffer will be filled up to the digitizer's frame size. The rest of the buffer will remain untouched. > If you grab into a buffer from a digitizer that outputs data in a format suitable for a container, the first intensity component in the frame of data will be grabbed. If there is no intensity component, the first component that is not metadata will be grabbed.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	The destination buffer's depth can only be 8 bits per band.	c														

		The destination buffer's depth can be 8 or 16 bits per band.			e	f	g								n	b
		When dealing with color smart-cameras, the destination image buffer can only be a BGR packed buffer with a 32-bit color depth (allocated using MbufAllocColor() with M_IMAGE + M_GRAB + M_BGR32). When dealing with monochrome smart-cameras, you cannot grab into an Aurora Imaging Library color image buffer.							i							
Container identifier		Specifies the identifier of a container with an M_GRAB attribute. This container must have been previously allocated, typically using MbufAllocContainer(). When grabbing into a container for the first time, Aurora Imaging Library will free all existing components and allocate the components required to store the current frame of data. Aurora Imaging Library will typically reuse these automatically allocated components during subsequent grabs, unless you add or remove components from the container (in which case all components will be freed and new components will again be allocated for the grabbed data). > Aurora Imaging Library will not grab into components of the container that you have allocated manually using MbufAllocComponent(), MbufCopyComponent(), or MbufCreateComponent(). In rare applications, you might configure your camera/3D sensor to transmit components of a different size or format with each grab. If there is any mismatch between the components from the previous grab and those required to store the current frame of data (for example, if the ordering, component type, or size has changed), all components will be automatically freed and new components will be allocated; the components will be assigned new identifiers. You can inquire the new buffer identifiers using MbufInquireContainer() with M_COMPONENT_ID. > If you need the buffer identifier of a component, and it is possible that the components might be reallocated, you should inquire the current buffer identifier after each grab.				f	g	a							n	b
Monochrome image buffer identifier		Specifies the identifier of a monochrome destination image buffer with an M_IMAGE + M_GRAB attribute. This image buffer must have been previously allocated, typically using MbufAlloc2d(). This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. It is not possible to grab into a color-band child buffer (that is, a child buffer created from one band of a color parent buffer) when the parent buffer is packed. When grabbing into an image buffer, if the grab destination buffer is smaller than the digitizer's frame size, the image buffer will be filled and all other data will be lost. If the grab destination buffer is larger than the digitizer's frame size, the buffer will be filled up to the digitizer's frame size. The rest of the buffer will remain untouched. > If you grab into a buffer from a digitizer that outputs data in a format suitable for a container, the first intensity component in the frame of data will be grabbed. If there is no intensity component, the first component that is not metadata will be grabbed.	c		e	f	g	a		i		j	k	l	m	n	b
		System specific														
		When using a color camera, the color data is first converted into YUV, and then the Y (luminance) component is stored in the buffer. The color space conversion is done in hardware.	c						i				l	m		
		The destination buffer's depth can only be 8 bits per band.	c													
		The destination buffer's depth can be 8 or 16 bits per band.			e	f	g				j	k	l	m	n	b
		The destination buffer can only be an 8-bit monochrome buffer.							i							

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MdigGrabContinuous

Synopsis

Grab data continuously from a camera.

Syntax

```
void MdigGrabContinuous(
    AIL_ID DigId,                    //in
    AIL_ID DestContainerOrImageBufId //out
)
```

Description

This function uses the specified digitizer to continuously acquire frames of data and store this data in the destination image buffer or container, until MdigHalt() is called. If the destination buffer/container is selected to a 2D or 3D display, the display is continuously updated, however only the last frame grabbed is written to the specified buffer/container.

When grabbing an image, you should use an image buffer as a destination. When grabbing any other type of data (such as 3D data), you should use a container as a destination. You can determine what type of Aurora Imaging Library object the digitizer is designed to grab into using MdigInquire() with M_TARGET_BUFFER_OBJECT.

When acquiring data from a line-scan type of camera, the exact number of rows grabbed is determined by the DCF.

If you need to save/process each grabbed frame, you should perform the grab using MdigProcess(). To perform a single grab, use MdigGrab().

[Host system]

When grabbing from a video or directory of images, once the last image is grabbed, grabbing will restart from the beginning of the video or from the first image in the directory.

Parameters

DigId

Specifies the identifier of the digitizer.

DestContainerOrImageBufId

Specifies the identifier of the destination image buffer or container.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the identifier of the destination image buffer or container																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Color image buffer identifier	Specifies the identifier of a color destination image buffer with an M_IMAGE + M_GRAB attribute. This image buffer must have been previously allocated, typically using MbufAllocColor(). This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. If the grab buffer is smaller than the digitizer's frame size, the image buffer will be filled and all other data will be lost. If the grab buffer is larger than the digitizer's frame size, the buffer will be filled up to the digitizer's frame size. The rest of the buffer will remain untouched. > If you grab into a buffer from a digitizer that outputs data in a format suitable for a container, the first intensity component in the frame of data will be grabbed. If there is no intensity component, the first component that is not metadata will be grabbed.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	The destination buffer's depth can only be 8 bits per band.	c														
	The destination buffer's depth can be 8 or 16 bits per band.			e	f	g									n	b

	When dealing with color smart-cameras, the destination image buffer can only be a BGR packed buffer with a 32-bit color depth (allocated using MbufAllocColor() with M_IMAGE + M_GRAB + M_BGR32). When dealing with monochrome smart-cameras, you cannot grab into an Aurora Imaging Library color image buffer.								i							
Container identifier	Specifies the identifier of a container with an M_GRAB attribute. This container must have been previously allocated, typically using MbufAllocContainer(). When grabbing into a container for the first time, Aurora Imaging Library will free all existing components and allocate the components required to store the current frame of data. Aurora Imaging Library will typically reuse these automatically allocated components during subsequent grabs, unless you add or remove components from the container (in which case all components will be freed and new components will again be allocated for the grabbed data). > Aurora Imaging Library will not grab into components of the container that you have allocated manually using MbufAllocComponent(), MbufCopyComponent(), or MbufCreateComponent(). In rare applications, you might configure your camera/3D sensor to transmit components of a different size or format with each grab. If there is any mismatch between the components from the previous grab and those required to store the current frame of data (for example, if the ordering, component type, or size has changed), all components will be automatically freed and new components will be allocated; the components will be assigned new identifiers. You can inquire the new buffer identifiers using MbufInquireContainer() with M_COMPONENT_ID. > If you need the buffer identifier of a component, and it is possible that the components might be reallocated, you should inquire the current buffer identifier after each grab.				f	g	a							n	b
Monochrome image buffer identifier	Specifies the identifier of a monochrome destination image buffer with an M_IMAGE + M_GRAB attribute. This image buffer must have been previously allocated, typically using MbufAlloc2d(). This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. It is not possible to grab into a color-band child buffer (that is, a child buffer created from one band of a color parent buffer) when the parent buffer is packed. When grabbing into an image buffer, if the grab destination buffer is smaller than the digitizer's frame size, the image buffer will be filled and all other data will be lost. If the grab destination buffer is larger than the digitizer's frame size, the buffer will be filled up to the digitizer's frame size. The rest of the buffer will remain untouched. > If you grab into a buffer from a digitizer that outputs data in a format suitable for a container, the first intensity component in the frame of data will be grabbed. If there is no intensity component, the first component that is not metadata will be grabbed.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	When using a color camera, the color data is first converted into YUV, and then the Y (luminance) component is stored in the buffer. The color space conversion is done in hardware.	c							i				l	m		
	The destination buffer's depth can only be 8 bits per band.	c														
	The destination buffer's depth can be 8 or 16 bits per band.			e	f	g					j	k	l	m	n	b
	The destination buffer can only be an 8-bit monochrome buffer.								i							

Compilation information

Header	Include AIL.h.

Library	Use ail11.lib.
DLL	Requires ail11.dll.

MdigGrabWait

Synopsis

Wait for the end of the grab in progress.

Syntax

```
void MdigGrabWait(
    AIL_ID DigId,         //in
    AIL_INT64 ControlFlag //in
)
```

Description

This function allows you to temporarily override the grab mode on the specified digitizer (see MdigControl() with M_GRAB_MODE).

Using this function forces the grab to wait until the grab timeout value has expired (set using MdigControl() with M_GRAB_TIMEOUT). Note that if the grab timeout is set to infinite, the grab will never end and this function will wait indefinitely. If the grabbed frame is not returned within the period of the timeout, an error is generated.

[Host system]

When grabbing from a video or directory of images, once the last image is grabbed, grabbing will restart from the beginning of the video or from the first image in the directory.

Parameters

DigId

Specifies the identifier of the digitizer.

ControlFlag

Specifies the function's control flag. This parameter must be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the function's control flag																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_GRAB_END	Wait for the end of all queued grabs. This value should not be used when grabbing data with MdigGrabContinuous().	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB_FRAME_END	Waits for the end of the current grab.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MdigHalt

Synopsis

Halt a continuous grab from an input device.

Syntax

```
void MdigHalt(
    AIL_ID DigId //in
)
```

Description

This function stops the specified digitizer from grabbing data. It should be used when performing a continuous grab with MdigGrabContinuous().

This function will wait for the end of the current frame before returning, to ensure the last frame is always valid.

Parameter

DigId

Specifies the identifier of the digitizer.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MdigHookFunction

Synopsis

Hook a function to a digitizer event.

Syntax

```
void MdigHookFunction(
    AIL_ID DigId,                           //in
    AIL_INT HookType,                       //in
    AIL_DIG_HOOK_FUNCTION_PTR HookHandlerPtr, //in
    void *UserDataPtr                       //in-out
)
```

Description

This function allows you to attach or detach a user-defined function to a specified digitizer event. Once a hook-handler function is defined and hooked to an event, it is automatically called when the event occurs.

You can hook more than one function to an event by making separate calls to **MdigHookFunction**() for each function that you want to hook. Aurora Imaging Library automatically chains and keeps an internal list of all these hooked functions. When a function is hooked, this new function is added to the end of the list. When the event happens, all user-defined functions in the list will be executed in the same order that they were hooked to the event. You can also remove any function from the list; in this case, Aurora Imaging Library preserves the order of the remaining functions in the list.

You can obtain more information about the event from within the hook-handler function using MdigGetHookInfo().

Note that functions hooked to an event execute on a distinct thread. This permits the functions to run concurrently from the operation that fired the event and from functions hooked to other events. Although there is a small queue to permit a certain amount of overlap, hooked functions should not take longer to execute than the period in which two of their associated events can occur.

Parameters

DigId

Specifies the identifier of the digitizer.

HookType

Specifies the digitizer event to which to hook the function. This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the digitizer event to which to hook the function																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CAMERA_LOCK +	Hooks the function to the event that occurs when the camera is locked. Use MdigInquire() with M_CAMERA_LOCKED to inquire if the camera is currently locked.												l	m		
M_CAMERA_PRESENT +	Hooks the function to the presence of the camera. Use MdigInquire() with M_CAMERA_PRESENT to inquire if the camera is currently present.	c			f	g					j	k			n	
	System specific															
	The preferred way to detect when a camera is connected or disconnected from the digitizer is to use MsysHookFunction() with M_CAMERA_PRESENT.	c			f	g									n	
	For an explanation of when to use MsysHookFunction() with M_CAMERA_PRESENT (instead of **MdigHookFunction**() with M_CAMERA_PRESENT), see the Automatic camera detection with Zebra Clarity UHD section of the Zebra Clarity UHD chapter in the Hardware-specific Notes.	c														
M_EXPOSURE_END +	Hooks the function to the event that occurs at the end of each exposure period of your camera.								i							
M_EXPOSURE_START +	Hooks the function to the event that occurs at the start of each exposure period of your camera.								i							

M_FEATURE_CHANGE +	Hooks the function to the event that occurs when the value of a GenICam feature changes. To enable an event to occur when the value of a specific feature changes, use MdigControlFeature() with M_FEATURE_CHANGE_HOOK set to M_ENABLE. Repeat for each feature for which you want to enable a feature change event. To enable an event to occur when the value of any feature changes, use M_FEATURE_CHANGE + M_ALL. This setting is only available for hooking to an image buffer. To retrieve the name of the feature that caused the event, use MdigGetHookInfo() with M_GC_FEATURE_CHANGE_NAME.			e	f	g				j	k	l	m	n	
M_FRAME_START +	Hooks the function to the event that occurs at the start of each frame of the incoming signal. Note that Aurora Imaging Library need not be grabbing the incoming signal for this event to occur. The event is only dependent on the presence of a camera.									j	k				
M_GC_EVENT +	Hooks the function to the message sent from your GenICam SFNC-compliant camera at the time of the specified event. Your camera sends specific messages using a dedicated message channel to inform your computer of specific events (such as the start and end of specific events in the process through which your camera acquires an image). When dealing with the messages sent from your camera, the exact timing and order of the messages is specific to your camera's manufacturer. Note that, if you don't add a combination constant to M_GC_EVENT, the function is hooked to any GenICam event that has been enabled using MdigControlFeature(). In addition, the message channel must be supported by your camera.			e	f	g								n	
M_GRAB_END +	Hooks the function to the event that occurs at the end of each grab. When grabbing continuously, the M_GRAB_END event is fired only when the last frame is grabbed. To be notified at the end of each frame, use the M_GRAB_FRAME_END event. A function hooked to an M_GRAB_END event is called when all the data of the grab is transferred to the Host. This means that this hooked function can be called after the M_GRAB_START or M_GRAB_FRAME_START event of the next frame.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB_FRAME_END +	Hooks the function to the event that occurs at the end of grabbed frames. Note that, in this case, the grabbed data might not be completely transferred to the Host.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB_FRAME_START +	Hooks the function to the event that occurs at the start of grabbed frames.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB_LINE_END + *n* +	Hooks the function to the event that occurs when the data of the specified line (row) is in the buffer and ready to be processed, where *n* is the specified line. For example, M_GRAB_LINE_END + 200 would call the hook-handler function when the data of line number 200 is in the buffer. You can call this function with this value up to 32 times during one grab.									j	k	l	m		
M_GRAB_LINE + *n* +	Hooks the function to the event that occurs when the specified line (row) is reached, where *n* is the specified line. For example, M_GRAB_LINE + 200 would call the hook-handler function when line number 200 is reached. > Note that the data of the specified line number might not be in the buffer. You can call this function with this value up to 32 times during one grab.									j	k	l	m		
M_GRAB_START +	Hooks the function to the event that occurs at the start of each grab. When grabbing continuously, the M_GRAB_START event is fired only when the first frame is grabbed. To be notified at the start of each frame, use the M_GRAB_FRAME_START event.	c		e	f	g	a		i		j	k	l	m	n	b

M_IO_CHANGE +	Hooks the function to the event that occurs when an I/O signal changes in accordance with its specified interrupt activation mode, set using MdigControl() with M_IO_INTERRUPT_ACTIVATION. Within the scope of the hook-handler function, it is prudent to verify that the appropriate I/O signal triggered the hook-handler function, using MdigGetHookInfo() with M_IO_INTERRUPT_SOURCE. If more than one input signal triggers an interrupt at the same time, the hook-handler function (or chain of hook-handler functions) is called for each input signal that generated an interrupt. As a result, any and all user-specified function(s) hooked using this hook type will be executed for each interrupt.			e		g					j	k	l	m	n	
M_ROTARY_ENCODER +	Hooks the function to the event that occurs when the rotary decoder's counter reaches its trigger position (specified using MdigControl() with M_ROTARY_ENCODER_POSITION_TRIGGER).										j	k	l	m		
M_TIMER_END +	Hooks the function to the event that occurs when the active period of the specified timer ends (specified using MdigControl() with M_TIMER_DURATION). You must specify a combination value from the following table: • To specify the timer to which to hook the function										j	k	l	m		
M_TIMER_START +	Hooks the function to the event that occurs when the specified timer is triggered. You must specify a combination value from the following table: • To specify the timer to which to hook the function										j	k	l	m		

Combination values for M_FEATURE_CHANGE.

You can add one of the following values to the above-mentioned value to specify to hook the specified function to all feature change events.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to hook to all feature change events																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_GENTL_STREAM_NUMBER(AIL_INT IndexValue)	Specifies to display the GenTL stream configuration information.			e												
	Parameters															
	IndexValue Specifies the index of the GenTL stream.			e												
	0 <= *Value* <= 33 — Specifies the index of the GenTL stream.			e												
M_ALL	Specifies to hook the specified function to all feature change events.			e	f	g					j	k	l	m	n	
M_GENTL_DEVICE	Specifies to display the GenTL device configuration information.			e												
M_GENTL_REMOTE_DEVICE	Specifies to display the GenTL remote device (camera) configuration information.			e												

Combination value for any of the possible values of the HookType parameter.

You can add the following value to the above-mentioned values to specify that the function should be unhooked.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying that the function should be unhooked																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_UNHOOK	Unhooks the specified function if hooked to the specified event. When you use M_UNHOOK, you must provide the same values for all the parameters as when you originally hooked the function.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_GC_EVENT.

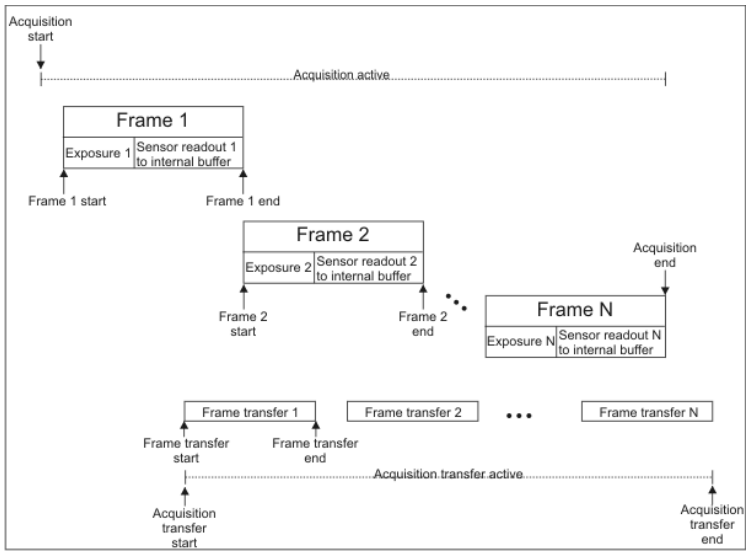
You can add one of the following values to the above-mentioned value to set the type of GenICam camera event.

The following events occur on the camera, as opposed to your frame grabber or the Host.

> Your camera informs the Host (your computer) of these events by sending specific messages on the dedicated message channel. When dealing with the messages sent from your camera, the order of the messages can be specific to your camera's manufacturer. Typically, however, they follow the GenICam SFNC. Check your camera's documentation for potential discrepancies.

Note that, if you don't add a combination constant to M_GC_EVENT, the function is hooked to any GenICam event that has been enabled using MdigControlFeature(). In addition, the message channel must be supported by your camera and the specified event must also be supported by your camera.

When dealing specifically with acquisition-related events on the camera, typically the events occur in the following sequence, which starts when your grab command sets up the camera to start the acquisition of images. You can hook a function to the start and end of each of the following events: expose the sensor (CCD or CMOS) of the camera, perform the sensor readout and save the frame to an internal buffer, and then transfer the frame from your camera to your computer.



For error handling of acquisition-related events on the camera, hook a function on the acquisition error event (using M_ACQUISITION_ERROR).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For GenICam camera events																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ACQUISITION_END	Hooks the function to the event that occurs when acquisition ends on your camera. When the camera is in single frame acquisition mode, this event occurs at (or before) the end of each frame. When the camera is in multiframe or continuous acquisition mode, this event occurs at (or before) the end of multiple frames. Note that, whether the acquisition end includes the acquisition transfer end is specific to your camera's manufacturer.			e	f	g									n	
M_ACQUISITION_ERROR	Hooks the function to the event that occurs if an error is generated at some point while the camera exposes an image, stores it in an internal buffer, and transfers it from the camera to your computer.			e	f	g									n	

M_ACQUISITION_START	Hooks the function to the event that occurs when acquisition begins on your camera. When the camera is in single frame acquisition mode, this event occurs at (or before) the beginning of each frame. When the camera is in multiframe or continuous acquisition mode, this event occurs at (or before) the beginning of multiple frames. Note that, whether the acquisition start includes the first frame start and the exposure start is specific to your camera's manufacturer.			e	f	g								n	
M_ACQUISITION_TRANSFER_END	Hooks the function to the event that occurs after transferring the image from your camera to your computer. When the camera is in single frame acquisition mode, this event occurs at the end of transferring each frame. When the camera is in multiframe or continuous acquisition mode, this event occurs at the end of transferring multiple frames. Note that, whether the acquisition transfer end includes the last frame transfer end is specific to your camera's manufacturer.			e	f	g								n	
M_ACQUISITION_TRANSFER_START	Hooks the function to the event that occurs before transferring the image from your camera to your computer. When the camera is in single frame acquisition mode, this event occurs at the beginning of transferring each frame. When the camera is in multiframe or continuous acquisition mode, this event occurs at the beginning of transferring multiple frames. Note that, whether the acquisition transfer start includes the first frame transfer start is specific to your camera's manufacturer.			e	f	g								n	
M_ACQUISITION_TRIGGER	Hooks the function to the event that occurs when acquisition is triggered on the camera. When the camera is in continuous acquisition mode, the M_ACQUISITION_TRIGGER event is fired only once, when your camera receives the initial acquisition trigger.			e	f	g								n	
M_COUNTER_END +	Hooks the function to the event that occurs when the specified camera counter stops. Counters are unique to GenICam SFNC-cameras. To configure your GenICam SFNC-counters, use MdigControlFeature(). You must specify a combination value from the following table: • To specify the GenICam SFNC-specific counter, or I/O signal to which to hook the function			e	f	g								n	
M_COUNTER_START +	Hooks the function to the event that occurs when the specified camera counter starts. Counters are unique to GenICam SFNC-cameras. To configure your GenICam SFNC-counters, use MdigControlFeature(). You must specify a combination value from the following table: • To specify the GenICam SFNC-specific counter, or I/O signal to which to hook the function			e	f	g								n	
M_EXPOSURE_END	Hooks the function to the event that occurs at the end of each exposure period of your camera.			e	f	g								n	
M_EXPOSURE_START	Hooks the function to the event that occurs at the start of each exposure period of your camera.			e	f	g								n	
M_FRAME_END	Hooks the function to the event that occurs when the frame ends on your camera. Regardless of the acquisition mode of your camera, this event occurs at the end of each frame.			e	f	g								n	
M_FRAME_START	Hooks the function to the event that occurs when the frame begins on your camera. Regardless of the acquisition mode of your camera, this event occurs at the beginning of each frame.			e	f	g								n	

M_FRAME_TRANSFER_END	Hooks the function to the event that occurs after transferring the frame from your camera to your computer. Regardless of the acquisition mode of your camera, this event occurs at the end of each frame. Instead of this event, it is more efficient to use M_GRAB_FRAME_END or MdigProcess() with a hook-handler function. This reduces overhead both on the camera and the Host.			e	f	g								n	
M_FRAME_TRANSFER_START	Hooks the function to the event that occurs before transferring the frame from your camera to your computer. Regardless of the acquisition mode of your camera, this event occurs at the beginning of each frame. Instead of this event, it is more efficient to use M_GRAB_FRAME_START. This reduces overhead both on the camera and the Host.			e	f	g								n	
M_FRAME_TRIGGER	Hooks the function to the event that occurs for each image when the acquisition of each image is triggered on the camera.			e	f	g								n	
M_LINE_ANY_EDGE +	Hooks the function to the event that occurs when the specified I/O signal changes on the camera. You must specify a combination value from the following table: • To specify the GenICam SFNC-specific counter, or I/O signal to which to hook the function			e	f	g								n	
M_LINE_FALLING_EDGE +	Hooks the function to the event that occurs on the falling edge of the specified I/O signal on the camera. You must specify a combination value from the following table: • To specify the GenICam SFNC-specific counter, or I/O signal to which to hook the function			e	f	g								n	
M_LINE_RISING_EDGE +	Hooks the function to the event that occurs on the rising edge of the specified I/O signal on the camera. You must specify a combination value from the following table: • To specify the GenICam SFNC-specific counter, or I/O signal to which to hook the function			e	f	g								n	
M_TIMER_END +	Hooks the function to the event that occurs when the specified timer ends. You must specify a combination value from the following table: • To specify the timer to which to hook the function			e	f	g								n	
M_TIMER_START +	Hooks the function to the event that occurs when the specified timer is triggered. You must specify a combination value from the following table: • To specify the timer to which to hook the function			e	f	g								n	

Combination value for M_COUNTER_END; M_COUNTER_START; M_LINE_ANY_EDGE; M_LINE_FALLING_EDGE; M_LINE_RISING_EDGE.

You must add the following value to the above-mentioned values to specify the GenICam SFNC-specific counter, or I/O signal to which to hook the function.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the GenICam SFNC-specific counter or I/O signal to which to hook the function																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

Value >= 0	Specifies to hook to counter *n* or I/O signal *n*. When dealing with counters, *n* is a value from 0 to 8. When dealing with an I/O signal, *n* is a value from 0 to 31.			e	f	g								n	

Combination value for M_TIMER_END; M_TIMER_START; M_TIMER_END; M_TIMER_START.

You must add the following value to the above-mentioned values to specify the timer to which to hook the function.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the timer to which to hook the function																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TIMER*n*	Specifies the timer to hook.			e	f	g					j	k	l	m	n	
	System specific															
	With a GenICam SFNC-compliant camera, *n* can be a value from 0 to 8.			e	f	g									n	
	In this case, *n* can be a value from 1 to 4.										j	k	l	m		

Combination value for M_ROTARY_ENCODER.

You can add the following value to the above-mentioned value to specify the rotary decoder to which to hook the function.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the rotary decoder to which to hook the function																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ROTARY_ENCODER*n*	Specifies to hook to rotary decoder n, where *n* is a number between 1 and 4. Note that if you do not use this combination constant to specify a rotary decoder, the rotary decoder corresponding to 1+ the device number of the specified digitizer (DigId) will be used. For example, when the device number of the specified digitizer is M_DEV2, rotary decoder M_ROTARY_ENCODER3 is used.												l	m		

HookHandlerPtr

Specifies the address of the function that should be called when the specified event occurs.

The hook-handler function, pointed to by HookHandlerPtr, must be declared as follows:

```
AIL_INT MFTYPE HookHandler(
    AIL_INT HookType,
    AIL_ID EventId,
    void *UserDataPtr
)
```

Parameters

HookType

Type of event hooked.

EventId

Event identifier to pass to MdigGetHookInfo() when inquiring about the hooked event.

UserDataPtr

User data pointer.

Upon successful completion, the hook-handler function should return **M_NULL**. Note, MFTYPE and AIL_DIG_HOOK_FUNCTION_PTR are reserved Aurora Imaging Library predefined types for functions and data pointers.

UserDataPtr

Specifies the address of the user data that you want to make available to the hook-handler function. This address is passed to the hook-handler

function, through its UserDataPtr parameter, when the specified event occurs. Set this parameter to **M_NULL** if not used.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MdigInquire

Synopsis

Inquire about a digitizer setting.

Syntax

```
AIL_INT MdigInquire(
    AIL_ID DigId,          //in
    AIL_INT64 InquireType, //in
    void *UserVarPtr       //out
)
```

Description

This function allows you to inquire about various digitizer settings.

Note that you can use MdigControl() to control specific digitizer settings.

You can also interactively inquire most of the digitizer settings in real-time, using Feature Browser; to access this interface, use Aurora Imaging Intellicam or MdigControl() with M_GC_FEATURE_BROWSER.

[Zebra GenTL driver; Zebra GigE Vision driver; Zebra USB3 Vision driver; Video4Linux2]

Typically, the supported inquire types inquire the GenICam-compliant camera (or device) associated with the digitizer. These inquire types assume your camera supports the associated GenICam standard feature naming convention (SFNC) feature. If the device does not support the associated GenICam SFNC feature, an error is generated. In case of error, use MdigInquireFeature() with the device-specific feature names and values specified by the device manufacturer.

Note that there are several inquire types that are available on a Host system, even without any additional Zebra imaging boards; these are to inquire a simulated digitizer (allocated using MdigAlloc() with M_EMULATED). To see the inquire types available to inquire a simulated digitizer, select the Host system as your board type in this Help file. For more information on simulated digitizers, refer to the Simulated digitizer section of Chapter 27: Grabbing with your digitizer.

Parameters

DigId

Specifies the identifier of the digitizer.

InquireType

Specifies the digitizer setting to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT64

Specifies the address in which to write the requested information. Since the **MdigInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For the general digitizer settings
- For the connection testing
- For the general reference settings
- For inquiring the input gain and shading correction
- For inquiring I/O signals and their mode
- For inquiring the state of specified user-bits in a static-user-output register
- For inquiring the settings to grab using a trigger
- For inquiring frame burst settings

- For inquiring the settings of a timer
- For inquiring the camera's exposure
- For inquiring the settings of a rotary decoder
- For inquiring a data latch
- For inquiring your GigE Vision camera's optional settings

The following inquire types allow you to inquire various digitizer settings.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the general digitizer settings

InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_BAYER_COEFFICIENTS_ID	Inquires an Aurora Imaging Library identifier of the internal buffer containing the white balance coefficients used when grabbing from a camera with a Bayer color filter. Note that you cannot free the retrieved buffer. SET															
	System specific															
	Refer to your Zebra imaging board's hardware manual for more details.										j	k	l	m		
	Note that this inquire type is used to perform Bayer conversion if performed by the Host. To inquire the Bayer conversion on the camera, use MdigInquireFeature(); refer to your camera's documentation for details regarding the features to inquire.			e		g									n	b
	Note that this inquire type is only available when using color versions of Zebra Iris GTX (such as 2000C).								i							
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*			e	f	g			i		j	k	l	m	n	b
M_NULL	Specifies that you have set M_WHITE_BALANCE to *M_DISABLE*.			e	f	g			i		j	k	l	m	n	b
Array buffer identifier	Specifies the identifier of an internally allocated M_ARRAY buffer containing the white balance coefficients.			e	f	g			i		j	k	l	m	n	b
M_BAYER_CONVERSION	Inquires whether a Bayer color conversion is performed by your digitizer. SET			e	f	g			i		j	k	l	m	n	b
	System specific															
	Refer to your Zebra imaging board's hardware manual for more details.										j	k	l	m		
	Note that this inquire type is used to perform Bayer conversion if performed by the Host. To inquire the Bayer conversion on the camera, use MdigInquireFeature(); refer to your camera's documentation for details regarding the features to inquire.			e		g									n	b
	Note that this inquire type is only available when using color versions of Zebra Iris GTX (such as 2000C).								i							
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g			i		j	k	l	m	n	b

M_DISABLE	Specifies to not perform Bayer conversion on the grabbed image.			e	f	g			i		j	k	l	m	n	b
M_ENABLE	Specifies to perform Bayer conversion on the grabbed image. INFO			e	f	g			i		j	k	l	m	n	b
M_BAYER_PATTERN	Inquires the Bayer pattern used by the camera.			e	f	g			i		j	k	l	m	n	b
	System specific															
	Refer to your Zebra imaging board's hardware manual for more details.												l	m		
	Refer to your camera's hardware manual for more details.			e	f	g									n	b
	Note that this inquire type is only available when using color versions of Zebra Iris GTX (such as 2000C).								i							
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g			i		j	k	l	m	n	b
M_NULL	Specifies that no Bayer pattern is used by the camera.			e	f	g			i		j	k	l	m	n	b
M_BAYER_BG	Specifies the Bayer pattern where the top-left pixel is band and the next pixel is green. INFO			e	f	g			i		j	k	l	m	n	b
M_BAYER_GB	Specifies the Bayer pattern where the top-left pixel is green and the next pixel is blue. INFO			e	f	g			i		j	k	l	m	n	b
M_BAYER_GR	Specifies the Bayer pattern where the top-left pixel is green and the next pixel is red. INFO			e	f	g			i		j	k	l	m	n	b
M_BAYER_RG	Specifies the Bayer pattern where the top-left pixel is red and the next pixel is green. INFO			e	f	g			i		j	k	l	m	n	b
M_BOARD_REVISION	Inquires the revision number of your digitizer.								i							
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*								i							
Value	Specifies the revision number.								i							
M_CAMERA_LOCKED	Inquires whether the digitizer is locked to the camera.	c					a						l	m		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c					a						l	m		
M_NO	Specifies that a camera lock failed.	c					a						l	m		
	System specific															
	M_CAMERA_LOCKED will return M_NO if all connections needed for the configuration of the camera are not locked.												l	m		
M_YES	Specifies that a camera lock succeeded.	c					a						l	m		
	System specific															
	M_CAMERA_LOCKED will return M_YES if all connections needed for the configuration of the camera are locked.												l	m		

M_CAMERA_MODEL +	Inquires the name of the camera model.			e	f	g							l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g							l	m	n	b
Value	Specifies the name of the camera model.			e	f	g							l	m	n	b
M_CAMERA_PRESENT	Inquires whether a camera is present. Note that this does not indicate that a camera is locked.	c			f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c			f	g	a		i		j	k	l	m	n	b
M_NO	Specifies that no camera is present.	c			f	g	a				j	k	l	m	n	b
M_YES	Specifies that a camera is present.	c			f	g	a		i		j	k	l	m	n	b
M_CAMERA_VENDOR +	Inquires the name of the camera vendor.			e	f	g							l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g							l	m	n	b
"String"	Specifies the name of the camera vendor.			e	f	g							l	m	n	b
M_CHANNEL	Inquires the channel on which the digitizer is to acquire data. Note that typically, this inquire type is only available when the specified digitizer uses acquisition paths with several multiplexed data inputs. SET	c														
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c														
M_CH0	Specifies channel 0 as the channel on which the digitizer is to receive input data. INFO	c														
M_CH1	Specifies channel 1 as the channel on which the digitizer is to receive input data. INFO	c														
M_CH2	Specifies channel 2 as the channel on which the digitizer is to receive input data. INFO	c														
M_CHANNEL_NUM	Inquires the number of available channels on the digitizer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of channels.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Zebra Iris GTX has 1 channel.								i							
M_CHANNEL_SYNC	Inquires the channel currently being used as a synchronization channel instead of a data channel. Note that using a digitizer as an external synchronization signal renders the digitizer of that signal "occupied" and not available for allocating as a digitizer.	c													n	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c													n	
M_CH*n*	Specifies the channel used for synchronization, where *n* is the channel number.	c													n	

M_COMMAND_QUEUE_MODE	Inquires whether changes to digitizer settings affect the grab digitizer immediately, even if it is currently being used in a grab, or the change will wait until the current grab is complete. SET				f					j	k	l	m		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f					j	k	l	m		
M_IMMEDIATE	Specifies that, if you change the setting of a control type of a digitizer currently being used in a grab, the change affects the current grab. INFO				f					j	k	l	m		
M_QUEUED	Specifies that, if you change the setting of a control type of a digitizer currently being used in a grab, the change does not affect the current grab. INFO				f					j	k	l	m		
M_CORRUPTED_FRAME_ERROR	Inquires whether an error is generated when a corrupted or incomplete frame is grabbed. SET	c		e	f	g			i					n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g			i					n	b
M_DISABLE	Specifies not to generate an error when grabbing a corrupted frame.	c		e	f	g			i					n	b
M_ENABLE	Specifies to generate an error when grabbing a corrupted frame.	c		e	f	g			i					n	b
M_DIG_PROCESS_IN_PROGRESS	Inquires whether MdigProcess() is currently running.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	MdigProcess() is not running.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	MdigProcess() is running.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIGITIZER_INTERNAL_BUFFERS_NUM	Inquires the number of internal grab buffers allocated and used when Aurora Imaging Library cannot grab directly into the specified buffers. SET			e		g								n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e		g								n	b
Value >= 0	Specifies the number of internal grab buffers.			e		g								n	b
M_DIGITIZER_TYPE	Inquires the type of frame grabber(s) available to allocate a digitizer on the system. The type of frame grabber is established by the type of sensor, so this inquire type returns the type of sensor.								i						
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*								i						
M_GTX2000	Specifies a Zebra Iris GTX 2000.								i						
M_GTX2000C	Specifies a Zebra Iris GTX 2000C.								i						
M_GTX3000	Specifies a Zebra Iris GTX 3000.								i						
M_GTX3000C	Specifies a Zebra Iris GTX 3000C.								i						
M_GTX5000	Specifies a Zebra Iris GTX 5000.								i						

	M_GTX5000C	Specifies a Zebra Iris GTX 5000C.								i						
	M_GTX8000	Specifies a Zebra Iris GTX 8000.								i						
	M_GTX8000C	Specifies a Zebra Iris GTX 8000C.								i						
	M_GTX12000	Specifies a Zebra Iris GTX 12000.								i						
	M_GTX12000C	Specifies a Zebra Iris GTX 12000C.								i						
	M_GTX16000	Specifies a Zebra Iris GTX 16000.								i						
	M_GTX16000C	Specifies a Zebra Iris GTX 16000C.								i						
M_EXTENDED_INIT_FLAG		Inquires the digitizer initialization flag. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEV_NUMBER +	Specifies that the digitizer was allocated for the camera with the specified device number.			e	f	g									n	b
	M_EMULATED	Specifies that the digitizer was allocated as a simulated digitizer.						a									
	M_GC_DEVICE_IP_ADDRESS +	Specifies that the digitizer was allocated for the camera with the specified IP address.			e	f	g										
	M_GC_DEVICE_NAME +	Specifies that the digitizer was allocated for the camera with the specified device's user name.			e	f	g									n	
	M_MINIMAL	Specifies that grabbing is not permitted with the allocated digitizer.	c														
M_FIX_PATTERN_NOISE_CORRECTION		Inquires the type of fixed pattern noise correction to apply when acquiring images. SET								i							
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*								i							
	M_PREPROCESSING	Specifies a correction for dark signal non-uniformity and for photo response non-uniformity, applied to the grabbed images. INFO								i							
	M_SENSOR	Specifies a correction for dark signal non-uniformity, applied to the grabbed images. INFO								i							
M_FOCUS +		Inquires the lens focus capabilities. Note that this inquire type requires that your Zebra Iris GTX use an I^2C (inter-integrated circuit) controlled lens. SET								i							
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*								i							
	Min. focus <= Value <= Max. focus	Specifies a value between the minimum and maximum values supported by the camera. INFO								i							

M_FOCUS_PERSISTENCE	Inquires whether to store a focus position for the I_2C (inter-integrated circuit) controlled lens. If there is no I_2C (inter-integrated circuit) controlled lens attached, the value is ignored. SET								i							
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*								i							
M_DISABLE	Specifies to not store the focus position.								i							
M_ENABLE	Specifies to store the focus position.								i							
M_FOCUS_PERSISTENT_VALUE	Inquires the focus position to store for the I_2C (inter-integrated circuit) controlled lens. SET								i							
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*								i							
0 <= *Value* <= 1023	Specifies to store the focus position.								i							
M_FORMAT +	Inquires the digitizer data format. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	//For monochrome or color cameras. Specifies the data format is set by the digitizer allocated as the multicast controller. INFO			e	f	g										b
AIL_TEXT("*DCF File name*") 1	//For monochrome or color cameras. Specifies the path and file name of the DCF (for example: "*C:\mydirectory\myfile*"). INFO	c		e	f	g			i		j	k	l	m	n	b
AIL_TEXT("M_DEFAULT") 1	//For monochrome or color cameras. Specifies the default digitizer format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FORMAT_DETECTED +	Inquires the detected DCF name.	c														
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c														
Value	Specifies the detected DCF name.	c														
M_GC_CAMERA_TIME_STAMP	Inquires the timestamp from the camera. To inquire the timestamp of an event, use MdigGetHookInfo() with M_GC_CAMERA_TIME_STAMP.			e	f	g										b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*			e	f	g										b
Value	Specifies the timestamp from the camera, in seconds.			e	f	g										b
M_GC_CLPROTOCOL	Inquires whether the GenICam CLProtocol module is enabled. Note that this inquire type is only available if using a GenCP-compatible camera, or if you first install the third-party, vendor-supplied, standard compliant CLProtocol library for the Camera Link camera connected to your digitizer. SET										j	k				
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k				
M_DISABLE	Specifies that the GenICam CLProtocol module is disabled.										j	k				

	M_ENABLE	Specifies that the GenICam CLProtocol module is enabled.										j	k			
M_GC_CLPROTOCOL_DEVICE_ID +		Inquires the identifier of the GenICam CLProtocol library currently associated with the Camera Link camera. Note that this inquire type is only available if using a GenCP-compatible camera, or if you first install the third-party, vendor-supplied, standard compliant CLProtocol library for the Camera Link camera connected to your digitizer. SET You must specify a combination value from the following table: • To specify which Camera Link camera to inquire										j	k			
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*										j	k			
M_GC_CLPROTOCOL_DEVICE_ID_DEFAULT +		Inquires the identifier of the GenICam CLProtocol library currently set as the default in the Aurora Imaging Configurator utility. If no default has been set in the Aurora Imaging Configurator utility, this will cause an error. Note that this inquire type is only available if using a GenCP-compatible camera, or if you first install the third-party, vendor-supplied, standard compliant CLProtocol library for the Camera Link camera connected to your digitizer.										j	k			
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*										j	k			
M_GC_CLPROTOCOL_DEVICE_ID_NUM		Inquires the number of GenICam CLProtocol camera identifiers available, given the list of the GenICam CLProtocol libraries installed on your computer. Note that this inquire type can be used to enumerate the complete list of camera identifiers installed on your computer, using M_GC_CLPROTOCOL_DEVICE_ID. For more information, refer to the Using Aurora Imaging Library with GenICam section of Chapter 27: Grabbing with your digitizer. Note that this inquire type is only available if using a GenCP-compatible camera, or if you first install the third-party, vendor-supplied, standard compliant CLProtocol library for the Camera Link camera connected to your digitizer.										j	k			
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k			
	Value	Specifies the number of the GenICam CLProtocol libraries.										j	k			
M_GC_CLPROTOCOL_DEVICE_ID_SIZE_MAX		Inquires the maximum number of characters required to store the GenICam CLProtocol camera identification string. Note that this inquire value can be used to enumerate the complete list of camera identifiers installed on your computer, using M_GC_CLPROTOCOL_DEVICE_ID. For more information, refer to the Using Aurora Imaging Library with GenICam section of Chapter 27: Grabbing with your digitizer. Note that this inquire type is only available if using a GenCP-compatible camera, or if you first install the third-party, vendor-supplied, standard compliant CLProtocol library for the Camera Link camera connected to your digitizer.										j	k			
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k			

	Value	Specifies the maximum number of characters required.										j	k				
M_GC_COUNTER_TICK_FREQUENCY		Inquires the camera's counter tick frequency. Note that a value of 0 Hz indicates that the camera does not support time stamps.			e	f	g									n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e	f	g									n	b
	Value	Specifies the camera's counter tick frequency, in Hz.			e	f	g									n	b
M_GC_FEATURE_POLLING		Inquires whether specific camera features will be periodically polled for updates when Feature Browser, accessed through Aurora Imaging Intellicam or MdigControl() with M_GC_FEATURE_BROWSER, is open. SET			e	f	g					j	k	l	m	n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g					j	k	l	m	n	
	M_DISABLE	Specifies that automatically changing camera features will not be polled.			e	f	g					j	k	l	m	n	
	M_ENABLE	Specifies that automatically changing camera features will be polled.			e	f	g					j	k	l	m	n	
M_GC_FRAME_MAX_RETRIES		Inquires the maximum number of times packets should be re-sent before flagging their associated frame as corrupt. SET					g										
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*					g										
	Value >= M_GC_PACKET_MAX_RETRIES	Specifies the maximum number of times to retry sending the packets of a frame. INFO					g										
M_GC_FRAME_STATUS_CODE		Inquires the status code from the last grabbed image.				f	g									n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g									n	
	Value	Specifies the status code.				f	g									n	
M_GC_FRAME_TIMEOUT		Inquires the maximum amount of time to wait for the remaining packets of a frame, after receiving the trailer packet. SET					g										
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*					g										
	Value >= 0	Specifies the maximum time to wait, in msec.					g										
M_GC_FRAME_TIMESTAMP		Inquires the timestamp of the last frame grabbed.			e	f	g									n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*			e	f	g									n	
	Value	Specifies the timestamp of the last frame grabbed, in sec.			e	f	g									n	
M_GC_HEARTBEAT_STATE		Inquires whether the heartbeat mechanism is used to keep the GigE Vision-compliant camera active. SET				f	g										
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g										
	M_DISABLE	Specifies that the heartbeat mechanism is disabled. INFO				f	g										

	M_ENABLE	Specifies that the heartbeat mechanism is enabled. INFO				f	g									
M_GC_HEARTBEAT_TIMEOUT		Inquires the amount of time that your GigE Vision-compliant camera will wait after the last communication before shutting down. SET				f	g									
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g									
	Value >= 0	Specifies the amount of time, in msec.				f	g									
M_GC_INTER_PACKET_DELAY		Inquires the delay between packets sent by your camera when transmitting a stream of image packets. If you require the inter-packet delay in sec, use the following calculation to convert from timestamp ticks to sec: (1/*m*)**n* , where m is the frequency of the timestamp ticks (to inquire this value, use M_GC_COUNTER_TICK_FREQUENCY), and *n* is your inter-packet delay value. SET				f	g									
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g									
	Value >= 0	Specifies the delay, in timestamp ticks.				f	g									
M_GC_INTERFACE_NAME +		Inquires the name of the Host interface, such as the Ethernet controller or frame grabber, to which your camera is connected.			e	f	g									
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g									
M_GC_LOCAL_CONTROL_PORT		Inquires the UDP port number used for the GVCP (GigE Vision Control Protocol) channel on the computer connected to your GigE Vision-compliant camera.				f	g									
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g									
	0 <= *Value* <=66535	Specifies the port number.				f	g									
M_GC_LOCAL_IP_ADDRESS		Inquires the IP address (IPv4) of the Host's network adapter, connected to the GigE Vision-compliant camera.				f	g									
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*				f	g									
	Value	Specifies the IP address (IPv4) of the Host's network adapter, connected to the GigE Vision-compliant camera. The IP address is stored as a 32-bit integer in an *AIL_INT64*. Typically, IP addresses are written as 4 separate 8-bit numbers. To retrieve these values, either inquire the IP address as a string instead using M_GC_LOCAL_IP_ADDRESS_STRING, or map the 32-bit IP address to 4 *AIL_UINT8* variables. For example, the hexadecimal representation of the IP address `192.168.109.72` is `0x00000000486da8c0` (equivalent to the decimal 1215146176). The least-significant byte (`c0`) is the first number of the IP address (192), the second least-significant byte (`a8`) is the second number of the IP address (168) and so on. The first 8 digits (4 most-significant bytes) are not part of the IP address.				f	g									
M_GC_LOCAL_IP_ADDRESS_STRING +		Inquires the IP address of the Host's network adapter, connected to the GigE Vision-compliant camera, as a string.				f	g									

	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*				f	g									
	AIL_TEXT("*nnn.nnn.nnn.nnn*") 1	Specifies the IP address of the Host's network adapter, connected to the GigE Vision-compliant camera, as a string. The address string is expressed in dotted decimal notation, where each dotted decimal (*nnn*) is a number between 000 and 255.				f	g									
M_GC_LOCAL_MAC_ADDRESS		Inquires the MAC address of the Host's network adapter, connected to the GigE Vision-compliant camera.				f	g									
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*				f	g									
	Value	Specifies the MAC address of the Host's network adapter, connected to the GigE Vision-compliant camera. The MAC address is stored as a 48-bit integer in an *AIL_INT64*. Typically, MAC addresses are written as 6 separate 8-bit numbers (hexadecimal number pairs). To retrieve these numbers, either inquire the MAC address as a string instead using M_GC_LOCAL_MAC_ADDRESS_STRING, or map the 48-bit MAC address to 6 *AIL_UINT8* variables. For example, the stored representation of the MAC address `84:20:fc:33:0e:e9` is `0x0000e90e33fc2084`. The least-significant byte (`84`) is the first number of the MAC address, the second least-significant byte (`0e`) is the second number of the MAC address and so on. The first 4 hexadecimal digits (2 most significant bytes) are not part of the MAC address.				f	g									
M_GC_LOCAL_MAC_ADDRESS_STRING +		Inquires the MAC address of the Host's network adapter, connected to the GigE Vision-compliant camera, as a string.				f	g									
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*				f	g									
	AIL_TEXT("*nn-nn-nn-nn-nn-nn*") 1	Specifies the MAC address of the Host's network adapter, connected to the GigE Vision-compliant camera, as a string. The address string is expressed in hexadecimal pairs, where each pair (*nn*) is a hexadecimal number between 00 and FF.				f	g									
M_GC_LOCAL_MESSAGE_PORT		Inquires the UDP port number used for the message channel on the computer connected to your GigE Vision-compliant camera. This inquire type is only available when dealing with a multicast controller, or multicast monitor (allocated using MdigAlloc() with M_GC_MULTICAST_CONTROLLER or M_GC_MULTICAST_MONITOR, respectively). SET				f	g									
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g									
	0 <= *Value* <=65535	Specifies the port number.				f	g									

M_GC_LOCAL_STREAM_PORT	Inquires the UDP port number used for the GVSP (GigE Vision Stream Protocol) channel on the computer connected to your GigE Vision-compliant camera. This inquire type is only available when dealing with a multicast controller, or monitor (allocated using MdigAlloc() with M_GC_MULTICAST_CONTROLLER or M_GC_MULTICAST_MONITOR, respectively). SET				f	g									
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g									
0 <= *Value* <=65535	Specifies the port number.				f	g									
M_GC_MAX_NBR_PACKETS_OUT_OF_ORDER	Inquires the maximum number of packets that can be received out-of-order, before the associated frame is marked as corrupt. SET					g									
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*					g									
Value >= 0	Specifies the maximum number of out-of-order packets to receive.					g									
M_GC_MESSAGE_CHANNEL_MULTICAST_ADDRESS_STRING +	Inquires the IP address used for the multicast message channel of your GigE Vision-compliant camera. This inquire type is only available when dealing with a multicast controller, or multicast monitor (allocated using MdigAlloc() with M_GC_MULTICAST_CONTROLLER or M_GC_MULTICAST_MONITOR, respectively). SET					g									
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*					g									
AIL_TEXT("*2nn.nnn.nnn.nnn*") 1	Specifies the IP address in dotted decimal notation.					g									
M_GC_MULTICAST_CONTROLLER_CONNECTED	Inquires whether a multicast controller digitizer is connected to the GigE Vision-compliant camera. Note that this can only be inquired from a digitizer allocated as a worker (using MdigAlloc() with M_GC_MULTICAST_WORKER).					g									
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*					g									
M_FALSE	Specifies that a multicast controller is not connected to the GigE Vision-compliant camera.					g									
M_TRUE	Specifies that a multicast controller is connected to the GigE Vision-compliant camera.					g									
M_GC_NUMBER_OF_STREAM_CHANNELS	Inquires the number of image streams available.			e	f	g								n	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g								n	
Value	Specifies the number of image streams available.			e	f	g								n	
M_GC_PACKET_MAX_RETRIES	Inquires the maximum number of times a packet can be re-sent. SET					g									
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*					g									
Value >= M_GC_FRAME_MAX_RETRIES	Specifies the maximum number of times to resend a given packet of a frame. INFO					g									

M_GC_PACKET_RESEND	Inquires whether to request packets be re-sent from your GigE Vision-compliant camera, if the packets are not receive properly (for example, when the packets are received out-of-order, or a packet timeout occurs). SET					g										
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*					g										
M_DISABLE	Specifies that packets should never be re-sent.					g										
M_ENABLE	Specifies that packets should be re-sent as required.					g										
M_GC_PACKET_SIZE	Inquires the packet size that is used by the GigE Vision-compliant camera when streaming data to the Host. SET				f	g										
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g										
Value >= 0	Specifies the packet size, in bytes. INFO				f	g										
M_GC_PACKET_TIMEOUT	Inquires the maximum amount of time to wait before flagging a packet as dropped. SET					g										
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*					g										
Value >= 0	Specifies the maximum time to wait, in msec.					g										
M_GC_PAYLOAD_SIZE	Inquires the number of bytes for each image transferred using the stream channel of your GenICam SFNC-compliant camera. Note that, if GenICam "chunk mode" is enabled on the camera, then this inquire returns the number of bytes in each chunk as well as the image transferred.			e	f	g									n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g									n	b
Value	Specifies the number of bytes transferred per image or chunk.			e	f	g									n	b
M_GC_PIXEL_FORMAT	Inquires the pixel format that the digitizer should use to create internal buffers to receive images from the camera, if the digitizer is a multicast monitor digitizer. SET			e	f	g										b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g										b
Value	Specifies the data depth and raw color format to use to create the internal buffers. INFO			e	f	g										b
M_GC_PIXEL_FORMAT_STRING +	Inquires the pixel format that the digitizer should use to create internal buffers to receive images from the camera, if the digitizer is a multicast monitor digitizer, expressed in a human readable format. SET			e	f	g										
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g										
M_GC_PIXEL_FORMAT_SWITCHING	Inquires whether to allow the camera's pixel format to change to automatically match the current grab buffer. SET			e		g									n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e		g									n	b
M_DISABLE	Specifies to disable automatic pixel format switching.			e		g									n	b

	M_ENABLE	Specifies to enable automatic pixel format switching.			e		g									n	b
M_GC_REMOTE_CONTROL_PORT		Inquires the UDP port number used for the GVCP (GigE Vision Control Protocol) channel on your GigE Vision-compliant camera.				f	g										
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g										
	0 <= *Value* <=66535	Specifies the port number.				f	g										
M_GC_REMOTE_IP_ADDRESS		Inquires the IP address (IPv4) of the GigE Vision-compliant grabberless interface camera.				f	g										
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*				f	g										
	Value	Specifies the IP address (IPv4) of the GigE Vision-compliant grabberless interface camera. The IP address is stored as a 32-bit integer in an *AIL_INT64*. Typically, IP addresses are written as 4 separate 8-bit numbers. To retrieve these values, either inquire the IP address as a string instead using M_GC_REMOTE_IP_ADDRESS_STRING, or map the 32-bit IP address to 4 *AIL_UINT8* variables. For example, the hexadecimal representation of the IP address `192.168.109.72` is `0x00000000486da8c0` (equivalent to the decimal 1215146176). The least-significant byte (`c0`) is the first number of the IP address (192), the second least-significant byte (`a8`) is the second number of the IP address (168) and so on. The first 8 digits (4 most-significant bytes) are not part of the IP address.				f	g										
M_GC_REMOTE_IP_ADDRESS_STRING +		Inquires the IP address (IPv4) of the GigE Vision-compliant camera, as a string.				f	g										
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*				f	g										
	AIL_TEXT("*nnn.nnn.nnn.nnn*") 1	Specifies the IP address of the GigE Vision-compliant camera as a string. The address string is expressed in dotted decimal notation, where each dotted decimal value (*nnn*) is a number between 0 and 255.				f	g										
M_GC_REMOTE_MAC_ADDRESS +		Inquires the MAC address of the GigE Vision-compliant camera.				f	g										
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*				f	g										
	Value	Specifies the MAC address of the GigE Vision-compliant camera. The MAC address is stored as a 48-bit integer in an *AIL_INT64*. Typically, MAC addresses are written as 6 separate 8-bit numbers (hexadecimal number pairs). To retrieve these numbers, either inquire the MAC address as a string instead using M_GC_REMOTE_MAC_ADDRESS_STRING, or map the 48-bit MAC address to 6 *AIL_UINT8* variables. For example, the stored representation of the MAC address `84:20:fc:33:0e:e9` is `0x0000e90e33fc2084`. The least-significant byte (`84`) is the first number of the MAC address, the second least-significant byte (`0e`) is the second number of the MAC address and so on. The first 4 hexadecimal digits (2 most significant bytes) are not part of the MAC address.				f	g										

M_GC_REMOTE_MAC_ADDRESS_STRING +	Inquires the MAC address of the GigE Vision-compliant camera, as a string.				f	g										
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*				f	g										
AIL_TEXT("*nn-nn-nn-nn-nn-nn*") 1	Specifies the MAC address of the GigE Vision-compliant camera, as a string. The address string is expressed in hexadecimal pairs, where each pair (*nn*) is a hexadecimal number between 00 and FF.				f	g										
M_GC_REMOTE_MESSAGE_PORT	Inquires the UDP port number used for the message channel on your GigE Vision-compliant camera.				f	g										
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g										
0 <= *Value* <=66535	Specifies the port number.				f	g										
M_GC_REMOTE_STREAM_PORT	Inquires the UDP port number used for the GVSP (GigE Vision Stream Protocol) channel on your GigE Vision-compliant camera.				f	g										
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g										
0 <= *Value* <=66535	Specifies the port number.				f	g										
M_GC_SCHEMA_MAJOR	Inquires the major version number of the schema used by the camera. The version number of the schema is structured as M_GC_SCHEMA_MAJOR. M_GC_SCHEMA_MINOR.			e	f	g				j	k	l	m	n	b
	System specific														
	Note that this inquire type is only available if using a GenCP-compatible camera, or if you first install the third-party, vendor-supplied, standard compliant CLProtocol library for the Camera Link camera connected to your digitizer.									j	k				
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g				j	k	l	m	n	b
Value	Specifies the major number of the schema version.			e	f	g				j	k	l	m	n	b
M_GC_SCHEMA_MINOR	Inquires the minor version number of the schema used by the camera. The version number of the schema is structured as M_GC_SCHEMA_MAJOR. M_GC_SCHEMA_MINOR.			e	f	g				j	k	l	m	n	b
	System specific														
	Note that this inquire type is only available if using a GenCP-compatible camera, or if you first install the third-party, vendor-supplied, standard compliant CLProtocol library for the Camera Link camera connected to your digitizer.									j	k				
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g				j	k	l	m	n	b
Value	Specifies the minor number of the schema version.			e	f	g				j	k	l	m	n	b
M_GC_SERIAL_NUMBER +	Inquires the serial number for the camera.			e	f	g						l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g						l	m	n	b
"String"	Specifies the serial number for the camera.			e	f	g						l	m	n	b

M_GC_SPECIFIC_INFO +	Inquires the specific information string for the camera.			e	f	g									n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g									n	b
"String"	Specifies the specific information string for the camera.			e	f	g									n	b
M_GC_STREAM_CHANNEL_MULTICAST_ADDRESS_STRING +	Inquires the IP address used for the multicast stream channel of your GigE Vision-compliant camera. This inquire type is only available when dealing with a multicast controller, or multicast monitor (allocated using MdigAlloc() with M_GC_MULTICAST_CONTROLLER or M_GC_MULTICAST_MONITOR, respectively). SET					g										
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*					g										
AIL_TEXT("*2nn.nnn.nnn.nnn*") 1	Specifies the IP address in dotted decimal notation.					g										
M_GC_STREAMING_MODE	Inquires the camera's image stream activation mechanism. SET			e	f	g						l	m	n		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g						l	m	n		
M_AUTOMATIC	Specifies that the image stream is started and stopped automatically. INFO			e	f	g						l	m	n		
M_MANUAL	Specifies that the image stream is started and stopped manually. INFO			e	f	g						l	m	n		
M_GC_STREAMING_STOP_CHECK_PERIOD	Inquires the maximum amount of time to wait before Aurora Imaging Library checks to see whether a grab is pending. Note that this inquire type is only available when M_GC_STREAMING_MODE is set to M_AUTOMATIC. SET			e	f	g						l	m	n		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g						l	m	n		
Value >= 0	Specifies the amount of time, in msec.			e	f	g						l	m	n		
M_GC_STREAMING_STOP_DELAY	Inquires the amount of time to wait before stopping the image stream, if no grab is pending. Note that this inquire type is only available when M_GC_STREAMING_MODE is set to M_AUTOMATIC. SET			e	f	g						l	m	n		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g						l	m	n		
Value >= 0	Specifies the amount of time, in msec.			e	f	g						l	m	n		
M_GC_THEORETICAL_INTER_PACKET_DELAY	Inquires the theoretical delay between packets sent by your camera when transmitting a stream of image packets. This theoretical delay is determined by the GigE Vision-compliant camera's current state. This inquire type is provided as a starting point to calculate a usable inter-packet delay.				f	g										
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*				f	g										

	Value	Specifies the theoretical delay between packets, in sec.				f	g									
M_GC_TOTAL_FRAME_CACHE_HITS		Inquires the number of partial or complete frames that pass through the frame cache.					g									
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*					g									
	Value	Specifies the number of frames.					g									
M_GC_TOTAL_FRAMES_CORRUPTED		Inquires the number of corrupted frames.				f	g								n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*				f	g								n	
	Value	Specifies the number of corrupted frames.				f	g								n	
M_GC_TOTAL_FRAMES_GRABBED		Inquires the number of frames grabbed. Note that this number includes the number of corrupt frames.				f	g								n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*				f	g								n	
	Value	Specifies the number of frames grabbed.				f	g								n	
M_GC_TOTAL_FRAMES_MISSED		Inquires the number of frames missed.				f	g								n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*				f	g								n	
	Value	Specifies the number of frames missed.				f	g								n	
M_GC_TOTAL_PACKET_CACHE_HITS		Inquires the number of packets that pass through the packet cache.					g									
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*					g									
	Value	Specifies the number of packets.					g									
M_GC_TOTAL_PACKETS_MISSED		Inquires the number of packets missed.					g									
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*					g									
	Value	Specifies the number of packets missed.					g									
M_GC_TOTAL_PACKETS_RECEIVED		Inquires the number of packets received.				f	g									
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*				f	g									
	Value	Specifies the number of packets received.				f	g									
M_GC_TOTAL_PACKETS_RECEIVED_OUT_OF_ORDER		Inquires the number of packets received out of order.					g									
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*					g									
	Value	Specifies the number of packets received out of order.					g									
M_GC_TOTAL_PACKETS_RECOVERED		Inquires the number of packets recovered.					g									
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*					g									
	Value	Specifies the number of packets recovered.					g									

M_GC_TOTAL_PACKETS_RESENDS_NUM	Inquires the total number of packet re-send requests sent to the GigE Vision-compliant camera. Note that, one re-send request can cause multiple consecutive packets to be re-sent.					g										
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*					g										
Value	Specifies the number of packets re-sent.					g										
M_GC_TOTAL_PACKETS_TIMEOUT	Inquires the total number of packets that have timed-out.					g										
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*					g										
Value	Specifies the number of packets timeouts.					g										
M_GC_UNIQUE_ID_STRING +	Inquires the unique identifier for your camera.				f	g									n	
	System specific															
	This is the camera's MAC address.					g										
	This is the camera's global unique identifier (GUID).														n	
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*				f	g									n	
AIL_TEXT("*nnnnnnnnnnnn*") 1	Specifies the unique identifier, as a string. The string is expressed in hexadecimal.				f	g									n	
M_GC_USER_NAME +	Inquires the camera's name. SET			e	f	g							l	m	n	
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g							l	m	n	
AIL_TEXT("*CameraName*") 1	Specifies the name of the camera.			e	f	g							l	m	n	
M_GC_VERSION +	Inquires the version number string for the camera.			e	f	g							l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g							l	m	n	b
"*String*"	Specifies the version number string for the camera.			e	f	g							l	m	n	b
M_GC_XML_MAJOR	Inquires the major version number of the GenICam-compliant camera's device description file (XML). The version number of the XML is structured as M_GC_XML_MAJOR. M_GC_XML_MINOR. M_GC_XML_SUBMINOR.			e	f	g				j	k	l	m	n	b
	System specific															
	Note that this inquire type is only available if using a GenCP-compatible camera, or if you first install the third-party, vendor-supplied, standard compliant CLProtocol library for the Camera Link camera connected to your digitizer.									j	k					
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g				j	k	l	m	n	b
Value	Specifies the major version number of the xml.			e	f	g				j	k	l	m	n	b

M_GC_XML_MINOR	Inquires the minor version number of the GenICam-compliant camera's device description file (XML). The version number of the XML is structured as M_GC_XML_MAJOR. M_GC_XML_MINOR. M_GC_XML_SUBMINOR.			e	f	g				j	k	l	m	n	b
	System specific														
	Note that this inquire type is only available if using a GenCP-compatible camera, or if you first install the third-party, vendor-supplied, standard compliant CLProtocol library for the Camera Link camera connected to your digitizer.									j	k				
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g				j	k	l	m	n	b
Value	Specifies the minor version number of the xml.			e	f	g				j	k	l	m	n	b
M_GC_XML_PATH +	Inquires the path and file name of the GenICam-compliant camera's device description file (XML).			e	f	g				j	k	l	m	n	b
	System specific														
	Note that this inquire type is only available if using a GenCP-compatible camera, or if you first install the third-party, vendor-supplied, standard compliant CLProtocol library for the Camera Link camera connected to your digitizer.									j	k				
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g				j	k	l	m	n	b
"*PathAndFilename*"	Specifies the path and file name of the XML configuration file.			e	f	g				j	k	l	m	n	b
M_GC_XML_SUBMINOR	Inquires the subminor version number of the GenICam-compliant camera's device description file (XML). The version number of the XML is structured as M_GC_XML_MAJOR. M_GC_XML_MINOR. M_GC_XML_SUBMINOR.			e	f	g				j	k	l	m	n	b
	System specific														
	Note that this inquire type is only available if using a GenCP-compatible camera, or if you first install the third-party, vendor-supplied, standard compliant CLProtocol library for the Camera Link camera connected to your digitizer.									j	k				
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g				j	k	l	m	n	b
Value	Specifies the subminor version number of the xml.			e	f	g				j	k	l	m	n	b
M_GENTL_INTERFACE_INDEX	Inquires the number of the GenTL interfaces associated to this digitizer.			e											
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e											
0 <= *Value* <=33	Specifies the total number of the GenICam GenTL camera interfaces.			e											
M_GENTL_STREAM_COUNT	Inquires the number of GenTL streams associated with the digitizer.			e											
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e											
Value >= 0	Specifies the number of GenTL streams.			e											

M_GRAB_DIRECTION_X	Inquires the horizontal grab direction. SET	c		e	f	g			i		j	k	l	m	n		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g			i		j	k	l	m	n		
M_FORWARD	Specifies to grab from left to right, in the horizontal direction.	c		e	f	g			i		j	k	l	m	n		
M_REVERSE	Specifies to grab from right to left, in the horizontal direction. INFO	c		e	f	g			i		j	k	l	m	n		
M_GRAB_DIRECTION_Y	Inquires the vertical grab direction. SET	c		e	f	g			i		j	k	l	m	n		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g			i		j	k	l	m	n		
M_FORWARD	Specifies to grab from top to bottom, in the vertical direction.	c		e	f	g			i		j	k	l	m	n		
M_REVERSE	Specifies to grab from bottom to top, in the vertical direction. INFO	c		e	f	g			i		j	k	l	m	n		
M_GRAB_FIELD_NUM	Inquires the number of fields grabbed with MdigGrab(). SET	c		e		g									n	b	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e		g									n	b	
1	Specifies to grab one field.	c		e		g									n	b	
2	Specifies to grab two fields.	c															
M_GRAB_FRAME_MISSED	Inquires the number of frames sent by the camera, but not received by the digitizer when using MdigGrab(), MdigGrabContinuous() or MdigProcess(). Note that this counter resets after calling this function with this inquire, and after calling MdigControl() with M_GRAB_FRAME_MISSED_RESET.				f						j	k	l	m			
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f						j	k	l	m			
Value	Specifies the number of frames missed.				f						j	k	l	m			
M_GRAB_FRAME_MISSED_COUNTER	Inquires whether to count the number of frames sent by the camera, but not received by the digitizer when performing a grab operation (that is, MdigGrab(), MdigGrabContinuous(), or MdigProcess()). SET				f						j	k	l	m			
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f						j	k	l	m			
M_DISABLE	Specifies to disable frames missed detection.				f						j	k	l	m			
M_ENABLE	Specifies to enable frames missed detection.				f						j	k	l	m			
M_GRAB_IN_PROGRESS	Inquires the current grab state, when either MdigGrabContinuous() or MdigProcess() is called.	c		e	f	g	a		i		j	k	l	m	n	b	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b	
M_NO	Specifies that there is no grab in progress. Note that a grab might be queued.	c		e	f	g	a		i		j	k	l	m	n	b	
M_YES	Specifies that there is a grab in progress.	c		e	f	g	a		i		j	k	l	m	n	b	

M_GRAB_LINE_COUNTER	Inquires whether a function hooked to an M_GRAB_END, M_ROTARY_ENCODER, or M_GRAB_FRAME_END event can inquire the number of lines grabbed, performed using MdigGrab(), MdigGrabContinuous(), or MdigProcess(). SET									j	k	l	m		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*									j	k	l	m		
M_DISABLE	Specifies that the hooked function cannot inquire the number of lines grabbed.									j	k	l	m		
M_ENABLE	Specifies that the hooked function can inquire the number of lines grabbed.									j	k	l	m		
M_GRAB_LUT_PALETTE	Inquires which LUT to use. Note that this inquire value is only supported on color cameras. SET							i							
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*							i							
M_DEFAULT	Specifies to use a 3-band RGB LUT.							i							
M_GAMMA	Specifies to use the native Bayer LUT that is applied to the Bayer image before it is converted to an RGB image.							i							
M_GRAB_MODE	Inquires how the grab should be synchronized with the Host when grabbing data with MdigGrab(). SET	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Note that, to use this inquire type on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_ASYNCHRONOUS	Specifies that your application continues after one grab is queued, rather than waiting for the grab to finish. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ASYNCHRONOUS_QUEUED	Specifies that your application continues after each grab is queued, rather than waiting for the grab to finish. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SYNCHRONOUS	Specifies that your application is synchronized with the end of a grab operation (that is, your application waits for the grab to finish before returning from the grab function).	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB_PERIOD	Inquires the duration of a frame (as specified in the DCF).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
0	Specifies that the inquiry of the duration of a frame is not supported.			e		g									n	b
Value > 0	Specifies the duration of a frame, in msec.	c		e	f	g	a		i		j	k	l	m	n	b

M_GRAB_SCALE_INTERPOLATION_MODE	Inquires the interpolation mode, when performing a vertical and/or horizontal scaling. Note that, to use this inquire type on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED). SET						a								
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*						a								
M_BICUBIC	Specifies bicubic interpolation. INFO						a								
M_BILINEAR	Specifies bilinear interpolation. INFO						a								
M_NEAREST_NEIGHBOR	Specifies nearest neighbor interpolation. INFO						a								
M_GRAB_SCALE_X	Inquires the horizontal scaling factor when grabbing data. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Note that, to use this inquire type on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a								
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_DESTINATION	Specifies that the scaling factor is calculated and set to fill the destination buffer. INFO	c			f						j	k	l	m		
1	Specifies that no scaling is applied.			e	f	g	a		i		j	k	l	m	n	b
Value = 1/n	Specifies to reduce the image size. INFO	c		e	f		a		i		j	k	l	m		b
M_GRAB_SCALE_Y	Inquires the vertical scaling factor when grabbing data. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Note that, to use this inquire type on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a								
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_DESTINATION	Specifies that the scaling factor is calculated and set to fill the destination buffer. INFO	c			f						j	k	l	m		
1	Specifies that no scaling is applied.			e	f	g	a		i		j	k	l	m	n	b
Value = 1/n	Specifies to reduce the image size. INFO	c		e	f		a		i		j	k	l	m		b
M_GRAB_START_MODE	Inquires the type of field on which to grab. Note that this value only applies when dealing with interlaced cameras. SET	c									j	k				b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c									j	k				b
M_FIELD_START	Specifies that the grab starts on any field.										j	k				
M_FIELD_START_EVEN	Specifies that the grab starts on an even field.	c									j	k				b
M_FIELD_START_ODD	Specifies that the grab starts on an odd field.	c									j	k				

M_GRAB_TIMEOUT	Inquires the maximum time to wait for a frame before generating an error. SET	c		e	f	g			i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g			i		j	k	l	m	n	b
M_INFINITE	Specifies to wait indefinitely. INFO	c		e	f	g			i		j	k	l	m	n	b
Value > 0	Specifies the time to wait, in msec.	c		e	f	g			i		j	k	l	m	n	b
M_HARDWARE_DEINTERLACING	Inquires whether hardware deinterlacing is used when the video source is interlaced. SET	c														
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c														
M_BOB_METHOD	Specifies to use the BOB algorithm for hardware deinterlacing.	c														
M_DISABLE	Specifies that hardware deinterlacing is disabled.	c														
M_MADI_METHOD	Specifies to use the MADI (motion adaptive deinterlacing) algorithm for hardware deinterlacing.	c														
M_INPUT_FILTER	Inquires the low-pass filter applied to incoming data. SET	c														
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c														
M_AUTOMATIC	Specifies that the best hardware filter is automatically selected or the filter is bypassed when the video input is interlaced and hardware deinterlacing is disabled.	c														
M_BYPASS	Specifies to not use a filter.	c														
M_LOW_PASS_0	Specifies to use the first low-pass filter. INFO	c														
M_LOW_PASS_1	Specifies to use the second low-pass filter. INFO	c														
M_INPUT_MODE	Inquires whether the analog or digital input mode is used, (as specified in the DCF).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_ANALOG	Specifies analog input mode.	c							i							
M_DIGITAL	Specifies digital input mode.	c		e	f	g	a		i		j	k	l	m	n	b
M_LAST_GRAB_IN_TRUE_BUFFER	Inquires whether a monoshot grab is performed when MdigHalt() is called after performing a continuous grab operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Note that, to use this inquire type on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

M_DISABLE	Specifies that the last frame grabbed during the continuous grab operation is copied to the target image buffer, instead of performing one last grab. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that a monoshot grab is performed. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_LIGHTING_BRIGHT_FIELD	Inquires the intensity of light to emit parallel to the optical axis, making the flat parts of an object stand out in stark contrast in the image. SET								i							
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*								i							
0 <= *Value* <= 255	Specifies the intensity of light to be emitted on the flat parts of the object.								i							
M_LUT_ID	Inquires whether to map the input data through the physical LUT of the specified digitizer and the values with which to initialize the LUT. SET						a		i		j	k	l	m		
	System specific															
	Note that, to use this inquire type on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*						a		i		j	k	l	m		
M_DEFAULT	Specifies that the digitizer's physical LUT is not used.						a		i		j	k	l	m		
LUT buffer identifier	Specifies the identifier of the LUT buffer (allocated with MbufAlloc...() with M_LUT) with which to initialize the digitizer's physical LUT. INFO						a		i		j	k	l	m		
M_NUMBER	Inquires the device number of the digitizer. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEV*n*	Specifies which acquisition path on your frame grabber, or which camera by rank when dealing with a grabberless interface camera (for example, a GigE Vision or USB3 Vision camera), to allocate for the digitizer. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_OWNER_SYSTEM	Inquires the Aurora Imaging Library identifier of the system on which the digitizer has been allocated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies the Aurora Imaging Library identifier of the system.	c		e	f	g	a		i		j	k	l	m	n	b
M_POWER_OVER_CABLE	Inquires whether PoCL (power over Camera Link) or PoCXP (power over CoaXPress) are enabled. SET										j	k		m		
	System specific															
	Note that the camera must be PoCL-capable. Once enabled, your PoCL-capable camera receives power over its Camera Link connection.										j	k				
	Note that the camera must be PoCXP-capable. Once enabled, your PoCXP-capable camera receives power over its CoaXPress connection.													m		

UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*									j	k		m			
M_OFF	Specifies to disable PoCL/PoCXP.									j	k		m			
M_ON	Specifies to enable PoCL/PoCXP.									j	k		m			
M_PROCESS_FRAME_CORRUPTED	Inquires the number of corrupted frames grabbed in the sequence with MdigProcess().	c		e	f	g	a		i					n	b	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i					n	b	
Value	Specifies the number of corrupted frames.	c		e	f	g	a		i					n	b	
M_PROCESS_FRAME_COUNT	Inquires the number of frames grabbed in the sequence with MdigProcess().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of grabbed frames.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROCESS_FRAME_MISSED	Inquires the number of frames believed to be missed when grabbing with MdigProcess().	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value is based on a comparison between the frame rate (M_PROCESS_FRAME_RATE) and the frame count (M_PROCESS_FRAME_COUNT). If a grab is triggered from a variable trigger or if the hooked function is slow, this value will be affected. This value is most trustworthy with a camera in continuous mode.	c					a		i					n		
	This value is based on a comparison between the previous frame's block identifier and the current frame's block identifier. The identifier is incremented by 1 for each frame transmitted. Using the block identifier allows Aurora Imaging Library to detect missed frames even when the camera is triggered from a variable trigger or if the hooked function is slow.			e		g										b
	This value is returned from your imaging board.									j	k	l	m			
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of missed frames.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROCESS_FRAME_RATE	Inquires the rate at which the frames are grabbed using MdigProcess(). Note that this inquire type is determined based on the average of the camera's frame rate, and the total amount of time to process the user-defined hooked function(s).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the rate at which the frames are grabbed, in frames/sec.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROCESS_GRAB_MONITOR	Inquires whether to create an internal grab monitoring thread when using MdigProcess() with either M_START or M_SEQUENCE. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Does not create a grab activity thread.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Creates a grab activity thread.	c		e	f	g			i		j	k	l	m	n	b
M_PROCESS_PENDING_GRAB_NUM		Inquires the number of buffers remaining in the list of buffers used by MdigProcess(). Note that this number includes the buffer used by the grab in progress. When dealing with round-robin grabbing, this number will typically be relatively close to the total number of allocated grab buffers.	c		e	f	g			i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g			i		j	k	l	m	n	b
	Value	Specifies the number of buffers remaining in the list of buffers.	c		e	f	g			i		j	k	l	m	n	b
M_PROCESS_TIMEOUT		Inquires the maximum amount of time to wait for MdigProcess() to complete the current grab (M_STOP) or all the queued grabs (M_STOP + M_WAIT), before generating an error. SET	c		e	f	g			i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g			i		j	k	l	m	n	b
	M_INFINITE	Specifies to wait indefinitely.	c		e	f	g			i		j	k	l	m	n	b
	Value >= 0	Specifies the time to wait, in msec.	c		e	f	g			i		j	k	l	m	n	b
M_PROCESS_TOTAL_BUFFER_NUM		Inquires the total number of allocated grab buffers in the list of buffers before MdigProcess() begins. SET	c		e	f	g					j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g					j	k	l	m	n	b
	Value	Specifies the total number of allocated grab buffers.	c		e	f	g					j	k	l	m	n	b
M_SCAN_MODE		Inquires the scan mode.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERLACE	Specifies interlace mode.	c		e		g									n	b
	M_LINESCAN	Specifies line-scan mode.			e		g					j	k	l	m	n	b
	M_PROGRESSIVE	Specifies progressive mode.	c		e	f	g	a		i		j	k	l	m	n	b
M_SELECTED_FRAME_RATE		Inquires the frame rate of the camera when grabbing in synchronous mode.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the frame rate to be used by the camera, in frames/sec.	c		e	f	g	a		i		j	k	l	m	n	b
M_SERIAL_NUMBER +		Inquires the serial number of the connected device.								i							
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*								i							
	"String"	Specifies the serial number of the connected device.								i							
M_SIGN		Inquires whether the data is signed or unsigned.	c		e	f	g	a		i		j	k	l	m	n	b

UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED	Specifies that the data is signed.			e	f	g	a								n	b
M_UNSIGNED	Specifies that the data is unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_BAND	Inquires the number of input color bands of the digitizer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of input color bands of the digitizer.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_BAND_LUT	Inquires the number of input color bands of the LUT buffer associated with the digitizer.			e		g			i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e		g			i		j	k	l	m	n	b
Value > 0	Specifies the number of input color bands.			e		g			i		j	k	l	m	n	b
M_SIZE_BIT	Inquires the depth per band.	c		e	f	g	a				j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a				j	k	l	m	n	b
Value	Specifies the depth per band, in bits.	c		e	f	g	a				j	k	l	m	n	b
M_SIZE_X +	Inquires the width of the image. When you specify the digitizer configuration format (DCF) appropriate for your camera, using MdigAlloc() with DataFormat, you are indirectly setting the size of the image that can be grabbed.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the width of the image, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_Y +	Inquires the height of the image. When you specify the digitizer configuration format (DCF) appropriate for your camera, using MdigAlloc() with DataFormat, you are indirectly setting the size of the image that can be grabbed.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the height of the image, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOURCE_DATA_FORMAT	Inquires the Aurora Imaging Library buffer format that is compatible with the camera's pixel format.	c		e	f	g									n	b
	System specific															
	Note that this value returns the Aurora Imaging Library equivalent of the camera's current pixel format. The camera's pixel format typically varies from camera to camera. To change the camera's pixel format, use MdigControlFeature().			e		g									n	b
	Note that this value returns the default Aurora Imaging Library buffer format, set using the DCF.	c														
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g									n	b

M_PACKED +	Specifies that the buffer's bands are stored in packed format (color buffer only); that is, each pixel's bands are stored together (RGB RGB RGB...).	c		e	f	g									n	b
M_PLANAR +	Specifies that the buffer's bands are stored in planar format; that is, each pixel's bands are stored in separate planes (RRR... GGG... BBB...).			e	f	g									n	b
M_SOURCE_NUMBER_OF_FRAMES	Inquires the number of frames that can be grabbed.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies an infinite number of frames.	c		e	f	g			i		j	k	l	m	n	b
Value > 0	Specifies the number of frames in the AVI file or folder specified when you allocated the digitizer.						a									
M_SOURCE_OFFSET_X	Inquires the X-offset of the input-signal capture window. SET	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Note that, to use this inquire type on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-offset, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOURCE_OFFSET_Y	Inquires the Y-offset of the input-signal capture window. SET	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Note that, to use this inquire type on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-offset, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOURCE_SIZE_X	Inquires the width of the input signal capture window. SET	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Note that, to use this inquire type on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOURCE_SIZE_Y	Inquires the height of the input signal capture window. SET	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Note that, to use this inquire type on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									

UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_TARGET_BUFFER_OBJECT	Inquires whether the digitizer outputs data in a format suitable for grabbing into a buffer or a container. Note that you can grab from any digitizer into either a buffer or a container. However, grabbing data suitable for a container into a buffer will result in only one of the components transmitted from the camera being grabbed. Grabbing data suitable for a buffer into a container is supported, but not typically necessary. You can automatically allocate the Aurora Imaging Library object most suitable for grabbing from a specified digitizer using MbufAllocDefault().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTAINER	Specifies that the digitizer outputs data in a format suitable for grabbing into a container.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE	Specifies that the digitizer outputs data in a format suitable for grabbing grabbing into a buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_TL_TRIGGER_ACTIVATION	Inquires the signal variation upon which to generate a trigger signal to the camera through the transport layer interface. SET												l	m		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*												l	m		
M_ANY_EDGE	Specifies that a trigger will be generated both upon a high-to-low and a low-to-high signal transition.												l	m		
M_EDGE_RISING	Specifies that a trigger will be generated upon a low-to-high signal transition.												l	m		
M_TYPE	Inquires the data type and depth of the digitizer's input. Depth is returned in bits.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
depth value + M_SIGNED	Specifies the data depth and that the data is signed.	c		e		g			i						n	b
depth value + M_UNSIGNED	Specifies the data depth and that the data is unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_WHITE_BALANCE	Inquires whether to perform white balancing. SET			e	f	g			i		j	k	l	m	n	b
	System specific															
	Note that this inquire type is used to perform Bayer conversion if performed by the Host. To inquire the Bayer conversion on the camera, use MdigInquireFeature(); refer to your camera's documentation for details regarding the features to inquire.			e		g									n	b
	Note that this inquire type is only available when using color versions of Zebra Iris GTX (such as 2000C).								i							
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g			i		j	k	l	m	n	b
M_DISABLE	Specifies that white balancing is disabled.			e	f	g			i		j	k	l	m	n	b

	M_ENABLE	Specifies that white balancing is enabled. INFO			e	f	g			i		j	k	l	m	n	b

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

Combination values that can be used alone or as combination values for [when InquireType=M_EXTENDED_INIT_FLAG: M_DEV_NUMBER; M_GC_DEVICE_IP_ADDRESS; M_GC_DEVICE_NAME].

One or more of the following values might be returned combined with each other, and/or with the above-mentioned values , to specify further details of the allocation of your camera.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying further details																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_GC_PACKET_SIZE_NEGOTIATION_SKIP	Specifies that the packet size negotiation task is skipped.				f	g										
M_GC_XML_DOWNLOAD_SKIP	Specifies that the camera's device description file (XML) should not be downloaded from the camera associated with the digitizer.				f	g									n	
M_GC_XML_FORCE_DOWNLOAD	Specifies that the camera's device description file (XML) should be downloaded from the camera associated with the digitizer.				f	g									n	

Combination values that can be used alone or as combination values for [when InquireType=M_EXTENDED_INIT_FLAG: M_DEV_NUMBER; M_GC_DEVICE_IP_ADDRESS; M_GC_DEVICE_NAME].

One of the following values might be returned on its own, or combined with the above-mentioned values , to specify whether the allocated digitizer will be a controller, monitor, or worker digitizer in a multicast controller-worker relationship.

For information about using IP multicast and configuring your Zebra GigE Vision digitizer, refer to the Using IP multicast section of the Zebra GigE Vision driver chapter in the Hardware-specific Notes.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the multicasting controller-worker relationship																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_GC_MULTICAST_CONTROLLER	Specifies that the allocated digitizer will be the controller digitizer in a multicasting controller-worker or monitor relationship.					g										
M_GC_MULTICAST_MONITOR	Specifies that the allocated digitizer will be a special type of worker digitizer (a monitor digitizer) in a multicast controller-worker relationship.					g										
M_GC_MULTICAST_WORKER	Specifies that the allocated digitizer will be a worker digitizer in a multicast controller-worker relationship.					g										

Combination values for [when InquireType=M_SOURCE_DATA_FORMAT: M_PLANAR].

One of the following values might be combined with the above-mentioned value to specify the format of the planar color buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the format of the planar color buffer																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MONO8	Specifies 8-bit monochrome depth planar pixels.			e	f	g									n	b
M_MONO16	Specifies 16-bit monochrome depth planar pixels.			e	f	g									n	b
M_MONO32	Specifies 32-bit monochrome depth planar pixels.			e	f	g									n	b

M_RGB24	Specifies 24-bit color depth planar (RGB 8:8:8) pixels.			e	f	g									n	b
M_RGB48	Specifies 48-bit color depth planar (RGB 16:16:16) pixels.			e	f	g									n	b
M_RGB96	Specifies 96-bit color depth planar (RGB 32:32:32) pixels.			e	f	g									n	b
M_YUV9	Specifies YUV9 planar pixels.			e	f	g									n	b
M_YUV16	Specifies YUV16 planar (4:2:2) pixels.			e	f	g									n	b
M_YUV16_UYVY	Specifies YUV16 planar (4:2:2) pixels, whereby the bands of each pixel are stored in the UYVY order. For more information, see the YUV buffers section of Chapter 23: Data buffers.			e	f	g									n	b
M_YUV16_YUYV	Specifies YUV16 planar (4:2:2) pixels, whereby the bands of each pixel are stored in the YUYV order. For more information, see the YUV buffers section of Chapter 23: Data buffers.			e	f	g									n	b
M_YUV411_1394	Specifies YUV411 planar (6:6:6) pixels.			e	f	g									n	b
M_YUV444	Specifies YUV444 planar (3:3:3) pixels.			e	f	g									n	b

Combination values for [when InquireType=M_SOURCE_DATA_FORMAT: M_PACKED].

One of the following values might be combined with the above-mentioned value to specify the format of the packed color buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the format of the packed color buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BGR24	Specifies 24-bit color depth packed (BGRBGR) pixels.			e	f	g									n	b
M_BGR32	Specifies 32-bit color depth packed (BGRXBGRX) pixels. The most-significant byte is a "don't care" byte.	c		e	f	g									n	b
M_RGB15	Specifies 16-bit color depth packed (XRGB 1:5:5:5) pixels. Note that when accessing this packed buffer as a 3-band 8-bit buffer, the least significant bits are set to 0.			e	f	g									n	b
M_RGB16	Specifies 16-bit color depth packed (RGB 5:6:5) pixels. Note that when accessing this packed buffer as a 3-band 8-bit buffer, the least significant bits are set to 0.	c		e	f	g									n	b
M_RGB24	Specifies 24-bit color depth packed (RGB 8:8:8) pixels.			e	f	g									n	b
M_RGB32	Specifies 32-bit color depth packed (RGB 10:12:10) pixels.	c		e	f	g									n	b
M_RGB48	Specifies 48-bit color depth packed (RGB 16:16:16) pixels.			e	f	g									n	b
M_RGB96	Specifies 96-bit color depth packed (RGB 32:32:32) pixels.	c		e	f	g									n	b
M_YUV16	Specifies YUV16 packed (4:2:2) pixels.	c		e	f	g									n	b
M_YUV16_UYVY	Specifies YUV16 packed (4:2:2) pixels, whereby the bands of each pixel are stored in the UYVY order. For more information, see the YUV buffers section of Chapter 23: Data buffers.	c		e	f	g									n	b
M_YUV16_YUYV	Specifies YUV16 packed (4:2:2) pixels, whereby the bands of each pixel are stored in the YUYV order. For more information, see the YUV buffers section of Chapter 23: Data buffers.	c		e	f	g									n	b

M_YUV411_1394	Specifies YUV411 packed (6:6:6) pixels.			e	f	g									n	b
M_YUV444	Specifies YUV444 packed (3:3:3) pixels.			e	f	g									n	b

Combination value for M_GC_CLPROTOCOL_DEVICE_ID.

You must add the following value to the above-mentioned value to specify which Camera Link camera to inquire.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying which Camera Link camera to inquire																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
0 <= *Value* <= 127	Specifies which Camera Link camera to inquire.										j	k				

Combination value for M_CAMERA_MODEL; M_CAMERA_VENDOR; M_FORMAT; M_FORMAT_DETECTED; M_GC_CLPROTOCOL_DEVICE_ID; M_GC_CLPROTOCOL_DEVICE_ID_DEFAULT; M_GC_INTERFACE_NAME; M_GC_LOCAL_IP_ADDRESS_STRING; M_GC_LOCAL_MAC_ADDRESS_STRING; M_GC_MESSAGE_CHANNEL_MULTICAST_ADDRESS_STRING; M_GC_PIXEL_FORMAT_STRING; M_GC_REMOTE_IP_ADDRESS_STRING; M_GC_REMOTE_MAC_ADDRESS; M_GC_REMOTE_MAC_ADDRESS_STRING; M_GC_SERIAL_NUMBER; M_GC_SPECIFIC_INFO; M_GC_STREAM_CHANNEL_MULTICAST_ADDRESS_STRING; M_GC_UNIQUE_ID_STRING; M_GC_USER_NAME; M_GC_VERSION; M_GC_XML_PATH; M_SERIAL_NUMBER.

You can add the following value to the above-mentioned values to get the string's length.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For getting the string size																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_STRING_SIZE	Retrieves the length of the string, including the terminating null character ("\0").															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

The following inquire types and inquire values relate to connection testing.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the connection testing																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_CONNECTION_COUNT	Inquires the number of connections between the system and the camera. The number of connections corresponds to the number of CXP input connectors used to connect with the camera.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*												l	m		
Value	Specifies the number of connections between the system and the camera.												l	m		
M_CONNECTION_ID +	Inquires which CXP connector on the camera is connected to the CXP input connector *n* on the board. You must specify a combination value from the following table: • To specify the CXP input connector												l	m		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*												l	m		

	Value	Specifies the ID.												I	m			
M_CONNECTION_STATE +		Inquires the state of the connection. You must specify a combination value from the following table: • To specify the CXP input connector												I	m			
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*												I	m			
	M_DETECTED	Specifies that the connection is detected.												I	m			
	M_UNDETECTED	Specifies that the connection is undetected.												I	m			
	M_UNKNOWN	Specifies that the state of the connection is unknown.												I	m			
M_CONNECTION_TEST_ERROR_COUNT +		Inquires the current connection error count for test packets. SET You must specify a combination value from the following table: • To specify the CXP input connector												I	m			
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*												I	m			
	Value	Specifies the current connection error count for test packets.												I	m			
M_CONNECTION_TEST_MODE +		Inquires the state of CXP test mode. SET You must specify a combination value from the following table: • To specify the CXP input connector												I	m			
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*												I	m			
	M_MODE1	Specifies that the CXP test mode is enabled.												I	m			
	M_OFF	Specifies that the CXP test mode is disabled.												I	m			
M_CONNECTION_TEST_PACKET_RECEIVED_COUNT +		Inquires the number of test packets received. SET You must specify a combination value from the following table: • To specify the CXP input connector												I	m			
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*												I	m			
	Value	Specifies the number of test packets received.												I	m			
M_CONNECTION_TEST_PACKET_TRANSMITTED_COUNT +		Inquires the number of test packets transmitted. SET You must specify a combination value from the following table: • To specify the CXP input connector												I	m			
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*												I	m			
	Value	Specifies the number of test packets transmitted.												I	m			

M_CONNECTION_TYPE +		Inquires the CXP connection type. A camera typically has only one main connection. You must specify a combination value from the following table: • To specify the CXP input connector											l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*											l	m		
	M_EXTENSION	Specifies that the CXP connection type is an extension connection.											l	m		
	M_MAIN	Specifies that the CXP connection type is a main connection.											l	m		

Combination value for M_CONNECTION_ID; M_CONNECTION_STATE; M_CONNECTION_TEST_ERROR_COUNT; M_CONNECTION_TEST_MODE; M_CONNECTION_TEST_PACKET_RECEIVED_COUNT; M_CONNECTION_TEST_PACKET_TRANSMITTED_COUNT; M_CONNECTION_TYPE.

You must add the following value to the above-mentioned values to specify the CXP input connector.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the CXP input connector																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CONNECTION*n*	Specifies the connection at CXP input connector *n*, where *n* is the CXP input connector number from 0 to 3.												l	m		

The following inquire types and inquire values relate to (if available) the reference levels used to digitize the analog signal received from a camera. These inquire types are specific to analog input devices. Depending on the type of digitizer and input signal, some reference types are not applicable. The smallest voltage increment supported by your board can differ such that consecutive reference-level settings might produce the same result. Note, some digitizers might take a few milliseconds before the reference level stabilizes.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the general reference settings																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_BLACK_OFFSET +	Inquires the offset to the black level of the image in hardware. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*								i							
0<= *Value* <= 255	Specifies the value of the black offset.								i							
M_BLACK_REF +	Inquires the input signal's digitization black reference level. SET			e	f	g									n	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*			e	f	g									n	
Value	Specifies the black reference level.			e	f	g									n	
M_BRIGHTNESS_REF	Inquires the brightness level for composite input signals. SET	c														
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c														
Value	Specifies the brightness reference value.	c														
M_CONTRAST_REF	Inquires the contrast level for composite input signals. SET	c														
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c														

Value	Specifies the contrast reference value.	c														
M_HUE_REF	Inquires the hue level for composite input signals. Note that this inquire type is not supported for monochrome cameras. SET	c														
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c														
Value	Specifies the digitizer hue reference level.	c														
M_SATURATION_REF	Inquires the saturation level for composite input signals. SET	c														
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c														
Value	Specifies the digitizer saturation reference level.	c														
M_WHITE_REF +	Inquires the input signal's digitization white reference level. SET			e	f	g									n	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*			e	f	g									n	
Value	Specifies the digitizer's white reference value.			e	f	g									n	

The following inquire types and inquire values allow you to inquire the input gain and shading correction.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the input gain and shading correction																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_GAIN +	Inquires the input gain with which to amplify the input signal. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g			i						n	b
Min. gain <= Value <= Max. gain	Specifies a value between the minimum and maximum values supported by the camera. INFO			e	f	g			i						n	b
M_GRAB_AUTOMATIC_INPUT_GAIN	Inquires whether the input gain should be automatically set. SET	c		e	f	g									n	
	System specific															
	This inquire type is only supported when the device number of the specified digitizer is M_DEV2 or M_DEV3. Note that this inquire type is available when the DCF specifies to grab either an analog or digital input signal.	c														
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g									n	
M_DISABLE	Specifies that the input gain is not automatically set.	c		e	f	g									n	
M_ENABLE	Specifies that the input gain is automatically set.	c		e	f	g									n	
M_GRAB_INPUT_GAIN	Inquires the input gain with which to amplify the input signal. This inquire type is only supported when the device number of the digitizer is M_DEV2 or M_DEV3. Note that this inquire type is available when the DCF specifies to grab either an analog or digital input signal. SET	c														
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c														
M_MAX_LEVEL	Specifies the maximum input gain factor.	c														

	M_MIN_LEVEL	Specifies the minimum input gain factor.	c													
	0 <= *Value* <= 255	Specifies an integer value which is used to determine the input gain factor. INFO	c													
M_SHADING_CORRECTION		Inquires whether to perform a gain and offset correction per pixel (shading correction). SET									j	k	l	m		
		System specific														
		Note that this inquire type is only available on Zebra Rapixo CXP Pro and Zebra Rapixo CoF 100, and only if the loaded FPGA configuration contains the Zebra PU that supports this functionality.											l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*									j	k	l	m		
	M_DISABLE	Disables shading correction.									j	k	l	m		
	M_ENABLE	Enables shading correction.									j	k	l	m		
M_SHADING_CORRECTION_GAIN_FIXED_POINT		Inquires the number of bits in the fractional part of the gain values specified for performing shading correction, where values are in fixed point format. The image resulting from the shading correction will be rounded to the nearest whole number. SET									j	k	l	m		
		System specific														
		Note that this inquire type is only available on Zebra Rapixo CXP Pro and Zebra Rapixo CoF 100, and only if the loaded FPGA configuration contains the Zebra PU that supports this functionality.											l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*									j	k	l	m		
	0 < *Value* <= 16	Specifies the number of bits in the fractional part of the fixed-point gain values.									j	k	l	m		
M_SHADING_CORRECTION_GAIN_ID		Inquires the buffer containing the gain values that your digitizer should use when it performs shading correction. SET									j	k	l	m		
		System specific														
		Note that this inquire type is only available on Zebra Rapixo CXP Pro and Zebra Rapixo CoF 100, and only if the loaded FPGA configuration contains the Zebra PU that supports this functionality.											l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*									j	k	l	m		
	M_NULL	Specifies not to apply a gain correction when performing shading correction. This is the default value.									j	k	l	m		
	ShadingCorrectionGainID	Specifies the identifier of an M_IMAGE buffer containing the gain values. INFO									j	k	l	m		
M_SHADING_CORRECTION_OFFSET_ID		Inquires the buffer containing the offset values that your digitizer should use when it performs a shading correction. SET									j	k	l	m		
		System specific														

		Note that this inquire type is only available on Zebra Rapixo CXP Pro and Zebra Rapixo CoF 100, and only if the loaded FPGA configuration contains the Zebra PU that supports this functionality.											l	m			
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*										j	k	l	m		
	M_NULL	Specifies not to apply an offset correction when performing shading correction. This is the default value.										j	k	l	m		
	ShadingCorrectionOffsetID	Specifies the identifier of an M_IMAGE buffer containing the offset values. INFO										j	k	l	m		

The following inquire types and inquire values allow you to inquire the mode and the purpose of your Zebra imaging board's I/O signals (such as, auxiliary, Camera Link control, and transport layer trigger signal). Once the format, routing, and mode are determined for an I/O signal, you can further inquire the I/O signal using the inquire types described in the following tables: For inquiring the state of specified user-bits in a static-user-output register, For inquiring the settings of a timer, For inquiring the camera's exposure, and For inquiring the settings to grab using a trigger.

> Note that for all Aurora Imaging Library supported hardware that have I/O signals, but are not supported with the constants below, see MsysInquire().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring I/O signals and their mode																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_AUX_IO_COUNT		Inquires the total number of auxiliary signals.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e		g					j	k			n	
	Value	Specifies the total number of auxiliary signals.			e		g					j	k			n	
M_AUX_IO_COUNT_IN		Inquires the total number of auxiliary input and bidirectional signals.			e		g					j	k			n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e		g					j	k			n	
	Value	Specifies the total number of auxiliary input and bidirectional signals.			e		g					j	k			n	
M_AUX_IO_COUNT_OUT		Inquires the total number of auxiliary output and bidirectional signals.			e		g					j	k			n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e		g					j	k			n	
	Value	Specifies the total number of auxiliary output and bidirectional signals.			e		g					j	k			n	
M_CC_IO_COUNT_OUT		Inquires the total number of camera control output signals.			e		g					j	k			n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e		g					j	k			n	
	Value	Specifies the total number of camera control output and bidirectional signals.			e		g					j	k			n	
M_IO_DEBOUNCE_TIME +		Inquires the amount of time that the specified auxiliary input signal is debounced. Note that a maximum of 4 inputs can have a debounce set. SET You must specify a combination value from the following table: • To specify the type and number of the I/O signal to inquire										j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k	l	m		

	0 <= *Value* <= 8300000	Specifies the minimum amount of time to ignore any additional signal transitions after accepting a signal transition, in nsec.									j	k	l	m		
M_IO_FORMAT +		Inquires the type of transmitter/receiver enabled for the specified I/O signal. SET You must specify a combination value from the following table: • To specify the type and number of the I/O signal to inquire			e		g				j	k	l	m	n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e		g				j	k	l	m	n	
	M_DISABLE	Specifies that the specified I/O signal is not available for use.			e		g								n	
	M_LINK_SIGNAL	Specifies to use the transport layer link trigger for the specified I/O signal.											l	m		
	M_LVDS	Specifies to use the LVDS transmitter/receiver for the I/O signal.			e		g				j	k	l	m	n	
	M_OPEN_DRAIN	Specifies to use the open collector (open drain) transmitter/receiver for the specified I/O signal.			e		g								n	
	M_OPTO	Specifies to use the opto-coupled transmitter/receiver for the specified I/O signal.			e		g				j	k	l	m	n	
	M_RS422	Specifies to use the RS-422 transmitter/receiver for the specified I/O signal.			e		g								n	
	M_TRI_STATE	Specifies to use the tri-state transmitter/receiver for the specified I/O signal.			e		g								n	
	M_TTL	Specifies to use the TTL transmitter/receiver for the specified I/O signal.			e		g				j	k	l	m	n	
M_IO_INTERRUPT_ACTIVATION +		Inquires the signal transition upon which to generate an interrupt, if interrupt generation has been enabled for the specified I/O signal. SET You must specify a combination value from the following table: • To specify the type and number of the I/O signal to inquire			e		g				j	k	l	m	n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e		g				j	k	l	m	n	
	M_ANY_EDGE	Specifies to generate an interrupt upon both a low-to-high and a high-to-low signal transition.			e		g				j	k	l	m	n	
	M_EDGE_FALLING	Specifies that an interrupt will be generated upon a high-to-low signal transition.			e		g				j	k	l	m	n	
	M_EDGE_RISING	Specifies that an interrupt will be generated upon a low-to-high signal transition.			e		g				j	k	l	m	n	
M_IO_INTERRUPT_STATE +		Inquires whether to generate an interrupt upon the specified transition of the I/O signal. SET You must specify a combination value from the following table: • To specify the type and number of the I/O signal to inquire			e		g				j	k	l	m	n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e		g				j	k	l	m	n	
	M_DISABLE	Specifies not to generate an interrupt.			e		g				j	k	l	m	n	
	M_ENABLE	Specifies to generate an interrupt.			e		g				j	k	l	m	n	

M_IO_MODE +		Inquires the mode of the specified I/O signal. SET You must specify a combination value from the following table: • To specify the type and number of the I/O signal to inquire			e		g					j	k	l	m	n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e		g					j	k	l	m	n	
	M_INPUT	Specifies that the signal is for input.			e		g					j	k	l	m	n	
	M_OUTPUT	Specifies that the signal is for output.			e		g					j	k	l	m	n	
M_IO_SOURCE +		Inquires the type of signal routed onto an output signal or onto a bidirectional signal set to output mode. SET You must specify a combination value from the following table: • To specify the type and number of the I/O signal to inquire			e		g					j	k	l	m	n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e		g					j	k	l	m	n	
	M_AUX_IO*n*	Specifies to reroute auxiliary input signal *n* to the output signal, where *n* is the number of the auxiliary input signal. INFO			e		g					j	k	l	m	n	
	M_DISABLE	Specifies to not route any signal to the specified signal.			e		g					j	k	l	m	n	
	M_LINK_TRIGGER_*n*_IN	Specifies to route the transport layer link trigger input signal *n* (from the camera) to the output signal, where *n* is the number of the link trigger input signal.												l	m		
	M_TIMER*n*	Specifies to route the output of timer *n*, where *n* is the number of timers available. INFO			e		g					j	k	l	m	n	
	M_TL_TRIGGER	Specifies to use the transport layer trigger signal (input mode only). INFO												l	m		
	M_USER_BIT_CC_IO*n*	Specifies to route the state of bit *n* of the camera control static-user-output register, where *n* is a value from 0 to 1. INFO										j	k				
	M_USER_BIT_TL_TRIGGER0	Specifies to route the state of the bit of the TL trigger static-user-output register.												l	m		
	M_USER_BIT*n*	Specifies to route the state of bit *n* of the main static-user-output register, where *n* is the bit number. INFO			e		g					j	k	l	m	n	
M_IO_STATUS +		Inquires the status of the input signal or bidirectional signal set to input. You must specify a combination value from the following table: • To specify the type and number of the I/O signal to inquire			e		g					j	k	l	m	n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e		g					j	k	l	m	n	
	M_INVALID	Specifies that the input signal is disabled.			e		g					j	k	l	m	n	
	M_OFF	Specifies that the input signal is off.			e		g					j	k	l	m	n	
	M_ON	Specifies that the input signal is on.			e		g					j	k	l	m	n	
	M_UNKNOWN	Specifies that the input signal cannot be inquired with its current configuration.			e		g					j	k	l	m	n	
		System specific															

	Inquiring the status of an auxiliary output signal will return this value, unless the auxiliary signal is outputting a user-bit (set using MdigControl() with M_IO_SOURCE set to M_USER_BIT...).									j	k				
M_IO_STATUS_ALL +	Inquires the status of all available I/O signals. Note that if there are I/O signals that cannot be inquired, the bits representing those signals, in the bit-encoded value returned, are not necessarily valid; these bits should be ignored.			e		g				j	k	l	m	n	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e		g				j	k	l	m	n	
Value	Specifies the bit-encoded value representing the status of all available and inquirable I/O signals.			e		g				j	k	l	m	n	
M_TL_TRIGGER_COUNT	Inquires the total number of TL trigger signals.			e	f	g				j	k	l	m	n	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g				j	k	l	m	n	
Value	Specifies the total number of TL trigger signals.			e	f	g				j	k	l	m	n	
M_TL_TRIGGER_COUNT_IN	Inquires the total number of input and I/O TL trigger signals.			e	f	g				j	k	l	m	n	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g				j	k	l	m	n	
Value	Specifies the total number of input and I/O TL trigger signals.			e	f	g				j	k	l	m	n	
M_TL_TRIGGER_COUNT_OUT	Inquires the total number of output and I/O TL trigger signals.			e	f	g				j	k	l	m	n	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g				j	k	l	m	n	
Value	Specifies the total number of output and I/O TL trigger signals.			e	f	g				j	k	l	m	n	

Combination values for M_IO_DEBOUNCE_TIME; M_IO_FORMAT; M_IO_INTERRUPT_ACTIVATION; M_IO_INTERRUPT_STATE; M_IO_MODE; M_IO_SOURCE; M_IO_STATUS.

You must add one of the following values to the above-mentioned values to specify the type and number of the I/O signal to inquire.

These combination constants have certain restrictions; for more information, refer to the For specifying the type and number of the I/O signal to affect combination table in MdigControl().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type and number of the I/O signal to inquire																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AUX_IO*n*	Specifies to inquire auxiliary signal *n*, where *n* is the signal number. Note that, when using this value with M_IO_SOURCE, the auxiliary signal *n* must be an output signal, or an I/O signal set to output (using M_IO_MODE).			e		g					j	k	l	m	n	
	System specific															
	For a list of the available auxiliary I/O signals, see the Connectors and signal names section of the Aurora Imaging Library Hardware-specific Notes chapter for your Zebra imaging board or Aurora Imaging Library driver.										j	k	l	m		
	With GigE Vision, USB3 Vision, and GenTL, *n* is a number from 0 to 31.			e		g									n	
M_CC_IO*n*	Specifies to inquire Camera Link camera control signal *n*, where *n* is a value from 1 to 4. Note that this combination value is only available for M_IO_SOURCE. For a list of the available camera control signals, see the Connectors and signal names section of the Aurora Imaging Library Hardware-specific Notes chapter for your Zebra imaging board or Aurora Imaging Library driver.										j	k				

M_LINK_TRIGGER_*n*_IN	Specifies to inquire the link trigger input signal *n* (from the camera), where *n* is the number of the link trigger input signal. Note that this combination value is only available for M_IO_FORMAT, M_IO_INTERRUPT_ACTIVATION, M_IO_MODE, and M_IO_STATUS.												l	m		
M_LINK_TRIGGER_*n*_OUT	Specifies to inquire the transport layer link trigger output signal *n* (to the camera), where *n* is the number of the link trigger output signal. Note that this combination value is only available for M_IO_FORMAT, M_IO_SOURCE, M_IO_MODE, and M_IO_STATUS.												l	m		
M_TL_TRIGGER	Specifies to inquire the transport layer trigger signal. The transport layer trigger signal is an embedded bidirectional signal from the physical transport layer connection of your camera. Typically, the TL trigger signal is reserved for trigger information and is sent with other control and data signals along the same cable. Note that this combination value is only available for M_IO_FORMAT, M_IO_INTERRUPT_ACTIVATION, M_IO_SOURCE, M_IO_MODE, and M_IO_STATUS.												l	m		

Combination values for M_IO_STATUS_ALL.

You can add one of the following values to the above-mentioned value to specify the type of I/O signal to inquire.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of I/O signal to inquire																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AUX_IO	Specifies to inquire all auxiliary signals.			e		g					j	k	l	m	n	
M_CC_IO	Specifies to inquire all Camera Link camera control signals.										j	k				

The following inquire types and inquire values allow you to inquire the bits in a static-user-output register. The user-bits are the bits associated with I/O signals set to output or output signals. For the correspondence between your Aurora Imaging Library user-bits and the signal names, see the connectors and signal names section of the Aurora Imaging Library Hardware-specific Notes chapter for your Zebra imaging board.

> Note that for other Zebra imaging boards that have user-bits, but are not supported with the constants below, see MsysInquire().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the state of specified user-bits in a static-user-output register																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_USER_BIT_COUNT	Inquires the total number of bits in the static-user-output register.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e		g					j	k	l	m	n	
Value	Specifies the number of bits in the default static-user-output register.			e		g					j	k	l	m	n	
M_USER_BIT_STATE +	Inquires the state of the specified bit in a static-user-output register. SET You must specify a combination value from the following table: • To determine the bit in the static-user-output register			e		g					j	k	l	m	n	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e		g					j	k	l	m	n	
M_OFF	Specifies that the specified bit is set to off.			e		g					j	k	l	m	n	

	M_ON	Specifies that the specified bit is set to on.			e		g					j	k	l	m	n	
M_USER_BIT_STATE_ALL +		Inquires the state of all the bits in a static-user-output register. To change the type of static-user-output register (that is, from the main static-user-output register to a the camera control static-user-output register), use one of the combination values documented below. SET			e		g					j	k	l	m	n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e		g					j	k	l	m	n	
	Value	Specifies a bit-encoded value that establishes the value of all the bits of the specified static-user-output register. INFO			e		g					j	k	l	m	n	

Combination values for M_USER_BIT_STATE.

You must add one of the following values to the above-mentioned value to determine the bit in the static-user-output register.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the bit in the static-user-output register																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_USER_BIT_CC_IO*n*	Specifies which bit of the camera control static-user-output register to inquire , where *n* is a value from 0 to 1. INFO										j	k				
M_USER_BIT_TL_TRIGGER0	Specifies to inquire the bit of the TL trigger static-user-output register.												l	m		
M_USER_BIT*n*	Specifies to inquire bit *n* of the main static-user-output register. INFO			e		g					j	k	l	m	n	

Combination values for M_USER_BIT_STATE_ALL.

You can add one of the following values to the above-mentioned value to specify the static-user-output register to inquire, if you don't want to inquire the main static-user-output register.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the static-user-output register type to inquire																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_USER_BIT	Specifies to inquire all the bits in the main static-user-output register.										j	k	l	m		
M_USER_BIT_CC_IO	Specifies to inquire all the bits in the static-user-output register associated with Camera Link camera control signals (the camera control static-user-output register).										j	k				
M_USER_BIT_TL_TRIGGER	Specifies to inquire all the bits in the static-user-output register associated with TL trigger signals (the TL trigger static-user-output register).												l	m		

The following inquire types and inquire values specify the settings for inquiring triggers. For more information, see the Grabbing with triggers section of Chapter 27: Grabbing with your digitizer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the settings to grab using a trigger																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_GRAB_CONTINUOUS_END_TRIGGER		Inquires whether an automatic trigger is generated after MdigHalt() is issued when performing a triggered continuous grab. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g			i		j	k	l	m	n	b
	M_DISABLE	Specifies not to generate the trigger automatically. INFO	c		e	f	g			i		j	k	l	m	n	b

M_ENABLE	Specifies to generate the trigger automatically. INFO	c		e	f	g			i		j	k	l	m	n	b
M_GRAB_TRIGGER_ACTIVATION	Inquires the signal transition upon which to generate a grab trigger. SET			e	f	g			i		j	k	l	m	n	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g			i		j	k	l	m	n	
M_ANY_EDGE	Specifies that a trigger will be generated upon both a low-to-high and a high-to-low signal transition.			e	f	g									n	
M_EDGE_FALLING	Specifies that a trigger will be generated upon a high-to-low signal transition.			e	f	g			i		j	k	l	m	n	
M_EDGE_RISING	Specifies that a trigger will be generated upon a low-to-high signal transition.			e	f	g			i		j	k	l	m	n	
M_LEVEL_HIGH	Specifies that a trigger is continuously issued during a high signal polarity.			e	f	g			i		j	k	l	m	n	
M_LEVEL_HIGH_END_WHEN_INACTIVE	Specifies that triggered grabs are initiated by a high trigger signal polarity, and completed when the trigger signal goes low.			e	f	g			i		j	k	l	m	n	
M_LEVEL_LOW	Specifies that a trigger is continuously issued during a low signal polarity.			e	f	g			i		j	k	l	m	n	
M_LEVEL_LOW_END_WHEN_INACTIVE	Specifies that triggered grabs are initiated by a low trigger signal polarity, and completed when the trigger signal goes high.			e	f	g			i		j	k	l	m	n	
M_GRAB_TRIGGER_DELAY	Inquires the delay between the trigger and the grab. SET			e	f	g									n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*			e	f	g									n	b
Value >= 0	Specifies the delay, in nsec.			e	f	g									n	b
M_GRAB_TRIGGER_MISSED	Inquires whether the number of grab triggers missed should be counted. SET								i							
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*								i							
M_DISABLE	Specifies to not count the number of grab triggers missed.								i							
M_ENABLE	Specifies to count the number of grab triggers missed.								i							
M_GRAB_TRIGGER_OVERLAP	Inquires how a new grab trigger will be dealt with when the current grab is in progress. SET			e	f	g			i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g			i		j	k	l	m	n	b
M_DISABLE	Specifies that a triggered grab will not overlap the transfer of the previous image. INFO			e	f	g			i						n	b
M_ENABLE	Specifies that a triggered grab can overlap the transfer of the previous image.			e	f	g			i						n	b
M_OFF	Specifies that a new trigger is ignored.										j	k	l	m		
M_PREVIOUS_FRAME	Specifies that a trigger received, while a grab is in progress, will be latched (stored). INFO										j	k	l	m		
M_RESET	Specifies that the current grab will be immediately stopped and a new grab will be started.										j	k	l	m		

M_GRAB_TRIGGER_SOURCE	Inquires the source for the grab trigger. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that there is no trigger source.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUX_IO*n*	Specifies to use auxiliary input signal *n* as the trigger source, where *n* is the number of the auxiliary signal. INFO			e	f	g			i		j	k	l	m	n	
M_HSYNC	Specifies to use the horizontal synchronization signal of the camera as the trigger source.										j	k				
M_IO_COMMAND_LIST1 +	Specifies to use a bit of the I/O command register of I/O command list 1 as a trigger source. This value is combined with a value from the following table: • To specify which I/O command register bit to use								i							
M_ROTARY_ENCODER	Specifies to use the output of the default rotary decoder (set with M_ROTARY_ENCODER_OUTPUT_MODE) as the trigger source. INFO								i		j	k	l	m		
M_ROTARY_ENCODER*n*	Specifies to use rotary decoder *n* as the trigger source, where *n* is a number between 1 and 4. INFO								i				l	m		
M_SOFTWARE	Specifies to use a software trigger as the trigger source. Use MdigControl() with M_GRAB_TRIGGER_SOFTWARE to issue the trigger.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOFTWARE*n*	Specifies to use the software trigger being used for the grab of another digitizer (on the same board) as the trigger source. INFO												l	m		
M_TIMER*n*	Specifies to use the output signal of the specified timer as the trigger source. INFO			e	f	g			i		j	k	l	m	n	
M_VSYNC	Specifies to use the vertical synchronization signal of the camera as the trigger source.										j	k				
M_GRAB_TRIGGER_STATE	Inquires whether, when a grab command is issued (for example, MdigGrab()), to wait for a trigger before grabbing. SET	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Note that, to use this inquire type on the Host, you must pass a simulated digitizer (allocated using MdigAlloc() with M_EMULATED).						a									
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that, when a grab command is issued, the grab occurs without waiting for a trigger.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that, when a grab command is issued, the grab waits for a trigger before occurring.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for [when InquireType=M_GRAB_TRIGGER_SOURCE: M_IO_COMMAND_LIST1].

The following value is combined with the above-mentioned value to specify which I/O command register bit to use.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying which I/O command register bit to use																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_IO_COMMAND_BIT*n*	Specifies I/O command register bit *n*, where *n* represents the bit number. INFO								i							

The following inquire types allow you to inquire the settings for a grab of sequential frames into a multi-frame image buffer (frame burst).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring frame burst settings																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_GRAB_FRAME_BURST_END_TRIGGER_SOURCE	Inquires which signal causes the end of a grab of sequential frames into a multi-frame image buffer (end-of-frame-burst trigger). SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k	l	m		
M_AUX_IO*n*	Specifies to use auxiliary input signal *n* as the trigger source, where *n* is the number of the auxiliary signal. INFO										j	k	l	m		
M_GRAB_FRAME_BURST_END_TRIGGER_STATE	Inquires whether a grab into a multi-frame image buffer is ended upon an end-of-frame-burst trigger. SET										j	k	l	m		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k	l	m		
M_DISABLE	Specifies that an end-of frame-burst trigger is not used to end the grab.										j	k	l	m		
M_ENABLE	Specifies that an end-of frame-burst trigger is used to end the grab.										j	k	l	m		
M_GRAB_FRAME_BURST_MAX_TIME	Inquires the maximum amount of time to wait for all the frames to be grabbed into the multi-frame buffer. The timer starts when the first frame is grabbed. SET										j	k	l	m		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*										j	k	l	m		
M_INFINITE	Specifies to wait indefinitely.										j	k	l	m		
0.000008 <= *Value* <= 1.000000	Specifies the maximum amount of time to wait, in secs.										j	k	l	m		
M_GRAB_FRAME_BURST_SIZE	Inquires the number of sequential frames to grab into a multi-frame buffer with one grab command (MdigGrab(), or one grab of MdigProcess()). SET										j	k	l	m		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k	l	m		
1 <= *Value* <= 4095	Specifies the number of frames to grab.										j	k	l	m		

The following inquire types and inquire values specify the settings for inquiring timers and the signals generated from a timer (timer output signals). For more information, see the Grabbing with triggers section of Chapter 27: Grabbing with your digitizer.

To specify which timer to inquire, see the For specifying which on-board timer to inquire combination value below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the settings of a timer																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_TIMER_ARM +	Inquires the state of the timer arming mechanism. SET															

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*												l	m		
	M_DISABLE	Specifies that timer arming is disabled.												l	m		
	M_ENABLE	Specifies to enable the timer arming mechanism.												l	m		
M_TIMER_ARM_ACTIVATION +		Inquires the signal transition upon which to arm the timer, if the timer arming mechanism is enabled. SET												l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*												l	m		
	M_EDGE_FALLING	Specifies that the timer will be armed upon a high-to-low signal transition.												l	m		
	M_EDGE_RISING	Specifies that the timer will be armed upon a low-to-high signal transition.												l	m		
	M_LEVEL_HIGH	Specifies that a timer is continuously armed during a high signal polarity.												l	m		
	M_LEVEL_LOW	Specifies that a timer is continuously armed during a low signal polarity.												l	m		
M_TIMER_ARM_SOURCE +		Inquires which input signal will arm the timer, if the timer arming mechanism is enabled. SET												l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*												l	m		
	M_NULL	Specifies to use no trigger source.												l	m		
	M_AUX_IO*n*	Specifies to use auxiliary input signal *n* as the trigger source for the specified timer, where *n* is the number of the auxiliary signal. INFO												l	m		
	M_CONTINUOUS	Specifies to automatically arm the specified timer immediately after the timer's duration expires.												l	m		
	M_EXPOSURE_END	Specifies to use the exposure end signal as the trigger source. INFO												l	m		
	M_EXPOSURE_START	Specifies to use the exposure start signal as the trigger source. INFO												l	m		
	M_GRAB_TRIGGER	Specifies to use the grab trigger source signal as the trigger source. INFO												l	m		
	M_HSYNC	Specifies to use the horizontal synchronization signal as the trigger source.												l	m		
	M_ROTARY_ENCODER	Specifies to use the output of the default rotary decoder (set with M_ROTARY_ENCODER_OUTPUT_MODE) as the trigger source.												l	m		
	M_ROTARY_ENCODER*n*	Specifies to use rotary decoder n, where *n* is a number between 1 and 4. INFO												l	m		
	M_SOFTWARE	Specifies to use a software trigger as the trigger source. INFO												l	m		
	M_TIMER*n*	Specifies to use the output signal of the specified timer as the trigger source, where *n* is the timer number. INFO												l	m		
	M_TL_TRIGGER	Specifies to use the transport layer trigger signal. INFO												l	m		
	M_VSYNC	Specifies to use the vertical synchronization signal as the trigger source.												l	m		
M_TIMER_CLOCK_FREQUENCY +		Inquires the frequency of the clock that drives the specified timer.										j	k	l	m		

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*									j	k	l	m		
	0	Specifies that the value cannot be inquired.									j	k	l	m		
	Value	Specifies the frequency of the clock source, in Hz.									j	k	l	m		
M_TIMER_CLOCK_SOURCE +		Inquires the source of the clock that drives the specified timer. SET									j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*									j	k	l	m		
	M_HSYNC	Specifies to use the horizontal synchronization frequency of your camera.									j	k	l	m		
	M_PIXCLK	Specifies to use the pixel clock frequency of your camera.									j	k				
	M_SYSCLK	Specifies to use the frequency of the allocated system's clock source. INFO									j	k	l	m		
	M_TIMER*n*	Specifies to use the frequency of the output of the specified timer, where *n* is the timer number. INFO									j	k	l	m		
	M_VSYNC	Specifies to use the vertical synchronization frequency of your camera.									j	k	l	m		
M_TIMER_DELAY +		Inquires the delay between the timer trigger and the active portion of the timer output signal. SET			e		g			i		j	k	l	m	n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*			e		g			i		j	k	l	m	n	
	0	Specifies that there is no delay.			e		g			i						n	
	Value > 0	Specifies the delay, in nsec.			e		g			i		j	k	l	m	n	
M_TIMER_DELAY2 +		Inquires the delay between the end of the first active portion of the timer output signal and the start of the second pulse. SET									j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*									j	k	l	m		
	Value > 0	Specifies the delay, in nsec.									j	k	l	m		
M_TIMER_DURATION +		Inquires the duration for the active portion of the timer output signal. SET			e		g			i		j	k	l	m	n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*			e		g			i		j	k	l	m	n	
	Value > 0	Specifies the duration of the active portion of the timer output signal, in nsec.			e		g			i		j	k	l	m	n	
M_TIMER_DURATION2 +		Inquires the duration for the active portion of the second pulse of the timer output signal. SET									j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*									j	k	l	m		
	Value > 0	Specifies the duration of the active portion of the second pulse of the timer output signal, in nsecs.									j	k	l	m		
M_TIMER_OUTPUT_INVERTER +		Inquires whether the output of the timer is inverted. SET								i	j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*								i	j	k	l	m		
	M_DISABLE	Specifies not to invert the output of the timer.								i	j	k	l	m		
	M_ENABLE	Specifies to invert the output of the timer.								i	j	k	l	m		

M_TIMER_RESET_SOURCE +	Inquires the signal source to use to reset the timer to 0. SET												l	m		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*												l	m		
M_NULL	Specifies to use no trigger source.												l	m		
M_AUX_IO*n*	Specifies to use auxiliary input signal *n* as the trigger source, where *n* is the number of the auxiliary signal. INFO												l	m		
M_EXPOSURE_END	Specifies to use the exposure end signal as the trigger source. INFO												l	m		
M_EXPOSURE_START	Specifies to use the exposure start signal as the trigger source. INFO												l	m		
M_GRAB_TRIGGER	Specifies to use the grab trigger source signal as the trigger source. INFO												l	m		
M_HSYNC	Specifies to use the horizontal synchronization signal as the trigger source.												l	m		
M_ROTARY_ENCODER	Specifies to use the output of the default rotary decoder (set with M_ROTARY_ENCODER_OUTPUT_MODE) as the trigger source.												l	m		
M_ROTARY_ENCODER*n*	Specifies to use rotary decoder n, where *n* is a number between 1 and 4. INFO												l	m		
M_SOFTWARE	Specifies to use a software trigger as the trigger source. INFO												l	m		
M_TIMER*n*	Specifies to use the output signal of the specified timer as the trigger source, where *n* is the timer number.												l	m		
M_TL_TRIGGER	Specifies to use the transport layer trigger signal. INFO												l	m		
M_VSYNC	Specifies to use the vertical synchronization signal as the trigger source.												l	m		
M_TIMER_STATE +	Inquires the state of the specified timer. SET			e		g			i		j	k	l	m	n	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e		g			i		j	k	l	m	n	
M_DISABLE	Specifies that the timer is disabled.			e		g			i		j	k	l	m	n	
M_ENABLE	Specifies that the timer is enabled.			e		g			i		j	k	l	m	n	
M_TIMER_TRIGGER_ACTIVATION +	Inquires the signal variation upon which to generate a timer trigger, when the specified timer is enabled. SET			e		g					j	k	l	m	n	
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e		g					j	k	l	m	n	
M_ANY_EDGE	Specifies that a timer trigger will be generated both upon a high-to-low and a low-to-high signal transition.			e		g									n	
M_EDGE_FALLING	Specifies that a timer trigger will be generated upon a high-to-low signal transition.			e		g					j	k	l	m	n	
M_EDGE_RISING	Specifies that a timer trigger will be generated upon a low-to-high signal transition.			e		g					j	k	l	m	n	
M_LEVEL_HIGH	Specifies that a timer trigger is continuously issued during a high signal polarity.			e		g									n	

	M_LEVEL_LOW	Specifies that a timer trigger is continuously issued during a low signal polarity.			e		g									n	
M_TIMER_TRIGGER_MISSED +		Inquires whether to count the number of trigger pulses missed. SET										j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k	l	m		
	M_DISABLE	Specifies not to count the number of trigger pulses missed.										j	k	l	m		
	M_ENABLE	Specifies to count the number of trigger pulses missed.										j	k	l	m		
M_TIMER_TRIGGER_OVERLAP +		Inquires how to deal with a new trigger that occurs while the associated timer has not yet expired (both its delay and duration). This is applied to both the delay and duration. SET										j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k	l	m		
	M_LATCH	Specifies that a trigger received, while the associated timer has not expired, will be latched (stored). INFO										j	k	l	m		
	M_OFF	Specifies that a new trigger is ignored.										j	k	l	m		
	M_RESET	Specifies that a new trigger automatically resets the timer (regardless of whether it is in its delay or active period) and then restarts the timer. INFO										j	k	l	m		
M_TIMER_TRIGGER_RATE_DIVIDER +		Inquires the frequency with which to accept trigger pulses. SET										j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k	l	m		
	1 <= *Value* <= 255	Specifies the frequency with which to accept a trigger out of a series of trigger pulses. INFO										j	k	l	m		
M_TIMER_TRIGGER_SOURCE +		Inquires the trigger source for the specified timer output signal. SET			e		g			i		j	k	l	m	n	
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e		g			i		j	k	l	m	n	
	M_NULL	Specifies that there is no timer trigger source.			e		g			i		j	k	l	m	n	
	M_AUX_IO*n*	Specifies to use auxiliary input signal *n* as the trigger source for the specified timer, where *n* is the number of the auxiliary signal. INFO			e		g					j	k	l	m	n	
	M_CONTINUOUS	Specifies to run the specified timer in periodic mode; no actual trigger signal is used. INFO			e		g					j	k	l	m	n	
	M_EXPOSURE_END	Specifies to use the exposure end signal as the trigger source. INFO			e		g									n	
	M_EXPOSURE_START	Specifies to use the exposure start signal as the trigger source. INFO			e		g			i						n	
	M_GRAB_TRIGGER	Specifies to use the grab trigger source signal as the trigger source. INFO			e		g			i						n	
	M_HSYNC	Specifies to use the horizontal synchronization signal as the trigger source.										j	k	l	m		
	M_ROTARY_ENCODER	Specifies to use the output of the default rotary decoder (set with M_ROTARY_ENCODER_OUTPUT_MODE) as the trigger source. INFO										j	k	l	m		
	M_ROTARY_ENCODER*n*	Specifies to use rotary decoder n, where *n* is a number between 1 and 4. INFO												l	m		
	M_SOFTWARE	Specifies to use a software trigger as the trigger source. INFO			e		g					j	k	l	m	n	

	M_TIMER*n*	Specifies to use the output signal of the specified timer as the trigger source, where *n* is the timer number. INFO			e		g					j	k	l	m	n		
	M_TL_TRIGGER	Specifies to use the transport layer trigger signal. INFO												l	m			
	M_VSYNC	Specifies to use the vertical synchronization signal as the trigger source.										j	k	l	m			

The following inquire types and inquire values specify the settings for inquiring the camera's exposure. For more information, see the Hardware-specific Notes for your Zebra Imaging board.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the camera's exposure																		
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																	
M_AUTO_EXPOSURE		Inquires the automatic exposure mode of the camera. SET																
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g									n	b
	M_AUTOMATIC	Specifies that the camera's exposure duration is constantly adapted by the device to maximize its effect.			e	f	g									n	b
	M_DISABLE	Specifies that the camera will not control the gain, iris, and shutter settings automatically.			e	f	g									n	b
	M_ENABLE	Specifies that the camera controls the gain, iris, and shutter settings automatically.			e	f	g									n	b
M_EXPOSURE_DELAY +		Inquires the required delay between the exposure trigger and the start of your camera's image sensor being exposed. SET								i							
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*								i							
	Value >= 0	Specifies the delay, in nsec.								i							
M_EXPOSURE_MODE		Inquires whether the exposure duration is set using an exposure timer or using the width of the trigger signal. SET								i							
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*								i							
	M_TIMED	Specifies to use M_EXPOSURE_TIME to control the length of the exposure.								i							
	M_TRIGGER_WIDTH	Specifies to use duration of the active portion of the grab trigger's output signal to control the length of the exposure. INFO								i							
M_EXPOSURE_TIME +		Inquires the amount of time to expose the camera's image sensor. SET			e	f	g			i						n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*			e	f	g			i						n	b
	Value >= 0	Specifies the amount of time to expose the camera's image sensor, in nsec. INFO			e	f	g			i						n	b

Combination values for M_FOCUS; M_SIZE_X; M_SIZE_Y; M_BLACK_OFFSET; M_BLACK_REF; M_WHITE_REF; M_GAIN; M_TIMER_DELAY; M_TIMER_DURATION; M_EXPOSURE_DELAY; M_EXPOSURE_TIME.

You can add one of the following values to the above-mentioned values to determine the largest or smallest value supported.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the largest or smallest values supported

Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MAX_VALUE	Inquires the largest value supported.			e	f	g			i						n	b
M_MIN_VALUE	Inquires the smallest value supported.			e	f	g			i						n	b

The following inquire types and inquire values allow you to inquire the settings of a rotary decoder.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the settings of a rotary decoder																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_ROTARY_ENCODER_BIT0_SOURCE +	Inquires the auxiliary input signal on which to receive bit 0 of the 2-bit Gray code. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k	l	m		
M_AUX_IO*n*	Specifies the auxiliary input signal to use. INFO										j	k	l	m		
M_ROTARY_ENCODER_BIT1_SOURCE +	Inquires the auxiliary input signal on which to receive bit 1 of the 2-bit Gray code. SET										j	k	l	m		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k	l	m		
M_AUX_IO*n*	Specifies the auxiliary input signal to use. INFO										j	k	l	m		
M_ROTARY_ENCODER_DIRECTION +	Inquires the direction of movement occurring when the Gray code sequence received by the rotary decoder is 00 - 01 - 11 - 10; this essentially establishes if the counter increments or decrements when receiving this sequence. A forward direction increments; while a backward direction decrements. SET										j	k	l	m		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k	l	m		
M_BACKWARD	Specifies a backward direction of movement and to decrement the counter when the Gray code sequence is 00 - 01 - 11 - 10. INFO										j	k	l	m		
M_FORWARD	Specifies a forward direction of movement and to increment the counter when the Gray code sequence is 00 - 01 - 11 - 10. INFO										j	k	l	m		
M_ROTARY_ENCODER_FORCE_VALUE_SOURCE +	Inquires the signal source to use to set the rotary decoder's counter to 0xFFFFFFFF. SET										j	k	l	m		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k	l	m		
M_NULL	Specifies not to set the counter to 0xFFFFFFFF upon a signal.										j	k	l	m		
M_AUX_IO*n*	Specifies to use an auxiliary input signal or an auxiliary bidirectional signal set to input (using M_IO_MODE set to M_INPUT). INFO										j	k	l	m		
M_COUNTER_OVERFLOW	Specifies to set the counter to 0xFFFFFFFF and keep it at this value after the counter increments past 0x7FFFFFFF. INFO										j	k	l	m		
M_POSITION_TRIGGER	Specifies to use the trigger signal generated by the rotary decoder when the counter reaches the value specified with M_ROTARY_ENCODER_POSITION_TRIGGER.										j	k	l	m		

	M_STEP_BACKWARD_WHILE_POSITIVE	Specifies to set the counter to 0xFFFFFFFF upon a decrement, only if the counter value is in the range of 0x0 to 0x7FFFFFFF before the decrement occurs; when interpreting the counter value as signed, this would be the positive counter value range. INFO										j	k	l	m		
	M_TL_TRIGGER	Specifies to use a transport layer trigger input signal.												l	m		
M_ROTARY_ENCODER_FRAME_END_POSITION +		Inquires the value of the rotary decoder's counter at the end of the last grab. To retrieve this value, you must have enabled the rotary decoder to store this value (before the grab), using MdigControl() with M_ROTARY_ENCODER_FRAME_END_READ. To retrieve the counter value at the end of the last grab from a hooked function, use MdigGetHookInfo() with M_ROTARY_ENCODER_FRAME_END_POSITION.										j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k	l	m		
	0 <= *Value* <= 429496295	Specifies the value of the rotary decoder's counter at the end of the last grab.										j	k	l	m		
M_ROTARY_ENCODER_FRAME_END_READ +		Inquires if the rotary decoder is enabled to store the counter value at the end of the last grab or frame interrupt. If enabled, the value of the counter can be inquired at anytime using M_ROTARY_ENCODER_FRAME_END_POSITION, or retrieved at the end of the last grab from a hooked function, using MdigGetHookInfo() with M_ROTARY_ENCODER_FRAME_END_POSITION. SET										j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k	l	m		
	M_DISABLE	Specifies that the counter value will not be available to be inquired at the end of the last frame grabbed.										j	k	l	m		
	M_ENABLE	Specifies that the rotary decoder is enabled to store the counter value at the end of the last grabbed frame, so that it can be inquired.										j	k	l	m		
M_ROTARY_ENCODER_MULTIPLIER +		Inquires the multiplying factor applied to each increment/decrement of the rotary decoder's counter for every rotary encoder step (change in position); this in turn applies a multiplying factor to the number of pulses that the rotary decoder outputs per step. SET										j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k	l	m		
	0 < *Value* <= 4096	Specifies the multiplying factor to use.										j	k	l	m		
M_ROTARY_ENCODER_OUTPUT_MODE +		Inquires the rotary decoder's counter value and/or the direction of movement upon which the rotary decoder should output a pulse. The pulse can be used to trigger a timer or a grab. SET										j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*										j	k	l	m		
	M_POSITION_TRIGGER	Specifies to output a pulse upon the trigger generated by M_ROTARY_ENCODER_POSITION_TRIGGER.										j	k	l	m		
	M_POSITION_TRIGGER_MULTIPLE	Specifies to start triggering only when the counter value set with M_ROTARY_ENCODER_POSITION_START_TRIGGER is reached and then to generate a trigger at a set interval (change) in counter value. INFO										j	k	l	m		
	M_STEP_ANY	Specifies to output a pulse upon any change in the rotary decoder's counter value (position change in any direction).										j	k	l	m		

	M_STEP_ANY_WHILE_POSITIVE	Specifies to output a pulse upon any change in the rotary decoder's counter value (position change in any direction), only if the counter value is in the range of 0x0 to 0x7FFFFFFF before the increment or decrement occurs; when interpreting the counter value as signed, this would be the positive counter value range. INFO									j	k	l	m		
	M_STEP_FORWARD	Specifies to output a pulse upon a rotary decoder counter increment only.									j	k	l	m		
	M_STEP_FORWARD_WHILE_POSITIVE	Specifies to output a pulse upon a rotary decoder counter increment, only if the counter value is in the range of 0x0 to 0x7FFFFFFF before the increment occurs; when interpreting the counter value as signed, this would be the positive counter value range. INFO									j	k	l	m		
M_ROTARY_ENCODER_POSITION +		Inquires the current value of the rotary decoder's counter.									j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*									j	k	l	m		
	0 <= *Value* <= 429497295	Specifies the current value of the counter.									j	k	l	m		
M_ROTARY_ENCODER_POSITION_START_TRIGGER +		Inquires the rotary decoder's counter value upon which to start generating triggers, when M_ROTARY_ENCODER_OUTPUT_MODE is set to M_POSITION_TRIGGER_MULTIPLE. If it is not set to this value, then M_ROTARY_ENCODER_POSITION_START_TRIGGER is ignored. SET									j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*									j	k	l	m		
	0 <= *Value* <= 0*xFFFFFFFF*	Specified the value of the counter upon which to start generating triggers.									j	k	l	m		
M_ROTARY_ENCODER_POSITION_TRIGGER +		Inquires the rotary decoder's counter value upon which a trigger is generated. SET									j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*									j	k	l	m		
	0 <= *Value* <= 0*xFFFFFFFF*	Specifies the value of the counter upon which a trigger is generated. If a value beyond the supported range is specified, an error is generated. If you are treating the counter values as a signed range of values (for example, forcing the counter to reset to 0 at 0x80000000) and you want to generate a trigger upon a negative value, specify the equivalent value in the range of 0x80000000 to 0xFFFFFFFF. If M_ROTARY_ENCODER_OUTPUT_MODE is set to M_POSITION_TRIGGER_MULTIPLE, this will specify the change in counter value at which to generate a trigger, beginning at the counter value specified by M_ROTARY_ENCODER_POSITION_START_TRIGGER. If M_ROTARY_ENCODER_OUTPUT_MODE is set to M_POSITION_TRIGGER, this will specify the counter value at which to generate a trigger each time this value is reached. If M_POSITION_TRIGGER is set to any other control value, this value will be ignored.									j	k	l	m		
M_ROTARY_ENCODER_RESET_SOURCE +		Inquires the signal source to use to reset the rotary decoder's counter to 0. SET									j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*									j	k	l	m		
	M_NULL	Specifies not to reset using a hardware signal source.									j	k	l	m		

	M_AUX_IO*n*	Specifies to use an auxiliary input signal or an auxiliary bidirectional signal set to input (using M_IO_MODE set to M_INPUT). INFO										j	k	l	m		
	M_POSITION_TRIGGER	Specifies to use the trigger signal generated by the rotary decoder when the counter reaches the value specified with M_ROTARY_ENCODER_POSITION_TRIGGER.										j	k	l	m		
	M_TL_TRIGGER	Specifies to use a transport layer trigger input signal.												l	m		

Combination value for the values listed in For inquiring the settings of a rotary decoder.

You can add the following value to the above-mentioned values to specify which rotary decoder to inquire.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying which rotary decoder to inquire																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ROTARY_ENCODER*n*	Specifies to inquire rotary decoder n, where *n* is a number between 1 and 4. INFO										j		l	m		

The following inquire types allow you to inquire the settings for one of the data latches of your installed Aurora Imaging Library board. Data latches store information specific to a grabbed frame when the latch is triggered. To retrieve information from a data latch, use MdigGetHookInfo() with M_DATA_LATCH.... Note that data latch information is only available when MdigGetHookInfo() is called from a function hooked to an end-of-frame event using MdigHookFunction() with M_GRAB_FRAME_END or from the callback function (hooked using MdigProcess()).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring a data latch																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_DATA_LATCH_CLOCK_FREQUENCY		Inquires the frequency of the clock when retrieving a timestamp in a data latch.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*										j	k	l	m		
	Value	Specifies the frequency of the clock when retrieving a timestamp in a data latch, in Hz.										j	k	l	m		
M_DATA_LATCH_MODE +		Inquires whether to latch data from the point when the first grab is queued until the start of the first frame of a series of queued grabs or grab sequence. SET You must specify a combination value from the following table: • To specify which data latch to inquire about										j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*										j	k	l	m		
	M_NORMAL	Specifies not to latch before the start of the first grabbed frame.										j	k	l	m		
	M_PREFETCH	Specifies to latch data before the start of the first grabbed frame.										j	k	l	m		
M_DATA_LATCH_STATE +		Inquires the state of the specified data latch. SET You must specify a combination value from the following table: • To specify which data latch to inquire about										j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*										j	k	l	m		

	M_DISABLE	Specifies that the data latch is disabled.										j	k	l	m		
	M_ENABLE	Specifies that the data latch is enabled.										j	k	l	m		
M_DATA_LATCH_TRIGGER_ACTIVATION +		Inquires the trigger signal transition upon which to store the specified information to the specified data latch. SET You must specify a combination value from the following table: • To specify which data latch to inquire about										j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*										j	k	l	m		
	M_EDGE_FALLING	Specifies to store the specified information to the data latch upon a high-to-low signal transition.										j	k	l	m		
	M_EDGE_RISING	Specifies to store the specified information to the data latch upon a low-to-high signal transition.										j	k	l	m		
M_DATA_LATCH_TRIGGER_SOURCE +		Inquires what triggers storing the specified information to the specified data latch. SET You must specify a combination value from the following table: • To specify which data latch to inquire about										j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*										j	k	l	m		
	M_AUX_IO*n*	Specifies to use auxiliary signal *n* as the trigger source, where *n* is the auxiliary signal number. INFO										j	k	l	m		
	M_GRAB_FRAME_END	Specifies to trigger the data latch on the event that occurs at the end of each grabbed frame; this event occurs when the frame grabber receives the last pixels (or packet) of a frame from the camera. INFO										j	k	l	m		
	M_GRAB_FRAME_START	Specifies to trigger the data latch on the event that occurs at the start of each grabbed frame.										j	k	l	m		
	M_GRAB_LINE	Specifies to trigger the data latch on the event that occurs at the end of each grabbed line; this event occurs when the frame grabber receives the last pixels (or packet) of a line from the camera. INFO										j	k	l	m		
	M_TIMER_ACTIVE +	Specifies to trigger the data latch on the event that occurs when the specified timer is active. INFO										j	k	l	m		
M_DATA_LATCH_TYPE +		Inquires the type of information to store to the specified data latch. SET You must specify a combination value from the following table: • To specify which data latch to inquire about										j	k	l	m		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*										j	k	l	m		
	M_IO_STATUS_ALL	Retrieves the status of all the auxiliary I/O signals.										j	k	l	m		
	M_ROTARY_ENCODER*n*	Retrieves the value of the counter of rotary decoder *n*, where *n* is a number between 1 and 4. INFO										j	k	l	m		
	M_TIME_STAMP	Retrieves the timestamp upon which the data latch is triggered, in ticks. INFO										j	k	l	m		

Combination value for M_DATA_LATCH_MODE; M_DATA_LATCH_STATE; M_DATA_LATCH_TRIGGER_ACTIVATION; M_DATA_LATCH_TRIGGER_SOURCE; M_DATA_LATCH_TYPE.

You must add the following value to the above-mentioned values to specify which data latch to inquire about.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data latch to inquire																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_LATCH*n*	Inquires which data latch to affect, where n is a value from 0 to 15.										j	k	l	m		

Combination values for the following values of in-parameters: the values listed in For inquiring the settings of a timer.
Combination values for the following values of out-parameters: [when InquireType=M_DATA_LATCH_TRIGGER_SOURCE: M_TIMER_ACTIVE].

The above-mentioned in-parameter values can be added to the following values. The above-mentioned out-parameter value might be returned combined with the following values. In either case, the following values are used to specify which on-board timer to inquire.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying which on-board timer to inquire																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TIMER_STROBE	Inquires the strobe timer.								i							
M_TIMER*n*	Inquires on-board timer *n*, where *n* is one of the timers available on your Zebra imaging board or the hardware associated with your Aurora Imaging Library driver. INFO			e		g					j	k	l	m	n	

The following inquire types allow you to inquire the settings of your GigE Vision camera. These settings are part of the GigE Vision Control Protocol (GVCP), described in the GigE Vision Specification version 2.0. The GVCP details GigE Vision specific settings to send and receive information across your network, as well as store information on your GigE Vision camera. Each inquire type specifies a family of capabilities within your GigE Vision device, where your camera is capable of using one or more features (although, each of these features is not necessarily enabled or configured). Note that, for these inquire types to be available, your GigE Vision camera must support one or more GigE Vision Control Protocol (GVCP) commands. Each of these inquire types returns a bitwise string, whose potential values are presented as inquire values. The inquire values are listed alphabetically; the bitwise order might differ.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring your GigE Vision camera's optional settings																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_GC_CONTROL_PROTOCOL_CAPABILITY	Inquires the presence of one or more optional GigE Vision features on your camera. Note that a combination of the values below can be returned. Bitwise operators must be used to verify the presence of a specific capability.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g										
M_GC_ACTION_SUPPORT	Specifies that your camera supports action commands. To configure an action command, use MsysControl() with M_GC_ACTION... + M_GC_ACTIONn, where *n* is the action number. Configure your camera, using MdigControlFeature() or Feature Browser (accessible through Aurora Imaging Intellicam or MdigControl() with M_GC_FEATURE_BROWSER) with the appropriate device features.				f	g										
M_GC_CONCATENATION_SUPPORT	Specifies that your camera supports concatenating multiple messages of a certain type into a single packet.				f	g										

	M_GC_DISCOVERY_ACK_DELAY_SUPPORT	Specifies that your camera supports reading the discovery acknowledgment delay field. The discovery acknowledgment delay is the maximum delay permitted before the GenICam-compliant device sends a discovery acknowledgment. On some GenICam-compliant devices, this period is static; on others, it is an automatically established amount of time when the device boots. To inquire whether the discovery acknowledgment delay field on your GenICam-compliant device is a static amount, or changes when the camera boots, use M_GC_WRITABLE_DISCOVERY_ACK_DELAY_SUPPORT.				f	g									
	M_GC_EVENT_DATA_SUPPORT	Specifies that your camera supports event-data messages. This means that when an event occurs on the camera, the camera can notify the application of the events and attach camera-specific data to the message.				f	g									
	M_GC_EVENT_SUPPORT	Specifies that your camera supports sending event messages. Event messages notify the application of events occurring on the camera.				f	g									
	M_GC_EXTENDED_STATUS_CODES_1_SUPPORT	Specifies that your camera supports the extended status codes introduced in the GigE Vision specification, version 1.1.				f	g									
	M_GC_EXTENDED_STATUS_CODES_2_SUPPORT	Specifies that your camera supports the extended status codes introduced in the GigE Vision specification, version 2.0.				f	g									
	M_GC_HEARTBEAT_DISABLE_SUPPORT	Specifies that your camera supports disabling the heartbeat mechanism. The heartbeat mechanism is used to inform your GigE Vision camera that communication between your Aurora Imaging Library application and the camera is still likely even though the camera has not contacted your application recently.				f	g									
	M_GC_IEEE_1588_SUPPORT	Specifies that your camera supports the IEEE 1588 standard for the precision time protocol. To configure an action command to use a scheduled time, use MsysControl() with M_GC_ACTION_TIME + M_GC_ACTIONn, where *n* is the action number. Enable the 1588 feature(s) on your camera, using MdigControlFeature() or Feature Browser (accessible through Aurora Imaging Intellicam or MdigControl() with M_GC_FEATURE_BROWSER) with the appropriate device features (for example, using the `GevIEEE1588` feature).				f	g									
	M_GC_LINK_SPEED_REGISTER_SUPPORT	Specifies that your camera supports storing the Ethernet link speed of the camera's network interface, in Mbits/sec.				f	g									
	M_GC_MANIFEST_TABLE_SUPPORT	Specifies that your camera supports a manifest table, which stores the camera's device description file (XML).				f	g									
	M_GC_PACKET_RESEND_SUPPORT	Specifies that your camera supports resending lost packets.				f	g									
	M_GC_PENDING_ACK_SUPPORT	Specifies that your camera supports a pending acknowledgment packet. A pending acknowledged packet indicates that the processing of a command will take a longer period than expected to execute. If your camera is capable of using such a feature, it will then increase the amount of time your application will wait before causing a timeout.				f	g									
	M_GC_PORT_AND_IP_REGISTER_SUPPORT	Specifies that your camera supports the control channel's primary port and IP address.				f	g									

	M_GC_PRIMARY_APP_SWITCHOVER_SUPPORT	Specifies that your camera supports primary application switchover. A primary application is an application granted full access to your camera. A primary application switchover occurs when two applications, both with the same level of access, trade control over your camera.				f	g									
	M_GC_SCHEDULED_ACTION_SUPPORT	Specifies that your camera supports scheduling commands at a specific time in the future. To configure an action command to use a scheduled time, use MsysControl() with M_GC_ACTION_TIME + M_GC_ACTIONn, where *n* is the action command number. Enable the IEEE 1588 features on your camera, using MdigControlFeature() or Feature Browser (accessible through Aurora Imaging Intellicam or MdigControl() with M_GC_FEATURE_BROWSER) with the appropriate device feature (for example, using the `GevIEEE1588` feature).				f	g									
	M_GC_SERIAL_NUMBER_SUPPORT	Specifies that your camera supports storing the camera's serial number.				f	g									
	M_GC_TEST_DATA_SUPPORT	Specifies that your camera supports creating and using a test packet to determine the size of the largest packet that your camera can transmit without error.				f	g									
	M_GC_UNCONDITIONAL_ACTION_SUPPORT	Specifies that your camera supports action messages, even when the primary control channel is closed.				f	g									
	M_GC_USER_DEFINED_NAME_SUPPORT	Specifies that your camera supports storing a user-defined name.				f	g									
	M_GC_WRITABLE_DISCOVERY_ACK_DELAY_SUPPORT	Specifies that your camera supports changing the discovery acknowledgment delay value when the camera boots. The discovery acknowledgment delay is the maximum delay permitted before the GenICam sends a discovery acknowledgment. On some GenICam devices, this period is static, while on others it is a random amount of time. To inquire whether the discovery acknowledgment delay field is supported on your GenICam device, use M_GC_DISCOVERY_ACK_DELAY_SUPPORT.				f	g									
	M_GC_WRITE_MEM_SUPPORT	Specifies that your camera supports having the primary application write to the camera's memory.				f	g									
M_GC_MESSAGE_PROTOCOL_CAPABILITY		Inquires whether your camera has one or more message channels available and their settings.				f	g									
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g									
	M_NULL	Specifies that no options are supported for the message channel.				f	g									
	M_GC_FIREWALL_TRAVERSAL_SUPPORT	Specifies that the firewall will not block message packets.				f	g									
M_GC_NETWORK_INTERFACE_CAPABILITY		Inquires whether your camera supports a series of networking interface options. Note that a combination of the values below can be returned. Bitwise operators must be used to verify the presence of a specific capability.				f	g									
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g									
	M_GC_DHCP_SUPPORT	Specifies that your camera supports DHCP.				f	g									
	M_GC_LINK_LOCAL_ADDRESS_SUPPORT	Specifies that your camera supports link-local addresses (LLA). A LLA is an IPv4 address in the 169.254.0.0/16 range.				f	g									

	M_GC_PAUSE_GENERATION_SUPPORT	Specifies that your camera supports generating Ethernet pause frames that halt the transmissions from the Host to the camera for a period of time. Pause frames are a common way of controlling Ethernet flow control.				f	g									
	M_GC_PAUSE_RECEPTION_SUPPORT	Specifies that your camera supports receiving Ethernet pause frames that halt the transmissions from the camera for a period of time. Pause frames are a common method of Ethernet flow control.				f	g									
	M_GC_PERSISTENT_IP_SUPPORT	Specifies that your camera supports storing a persistent (static) IP address.				f	g									
M_GC_NETWORK_INTERFACE_CONFIGURATION		Inquires which network and IP configuration schemes are enabled and in use on your current network. Note that a combination of the values below can be returned (for example, M_GC_PAUSE_RECEPTION_SUPPORT + M_GC_PAUSE_GENERATION_SUPPORT + M_GC_LINK_LOCAL_ADDRESS_SUPPORT). Bitwise operators must be used to verify the presence of a specific capability.				f	g									
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g									
	M_GC_DHCP_SUPPORT	Specifies that your camera is using DHCP.				f	g									
	M_GC_LINK_LOCAL_ADDRESS_SUPPORT	Specifies that your camera is using link-local addresses (LLA). A LLA is an IPv4 address in the 169.254.0.0/16 range.				f	g									
	M_GC_PAUSE_GENERATION_SUPPORT	Specifies that your camera is using generating Ethernet pause frames that halt the transmissions from the Host to the camera for a period of time. Pause frames are a common way of controlling Ethernet flow control.				f	g									
	M_GC_PAUSE_RECEPTION_SUPPORT	Specifies that your camera is using receiving Ethernet pause frames that halt the transmissions from the camera for a period of time. Pause frames are a common method of Ethernet flow control.				f	g									
	M_GC_PERSISTENT_IP_SUPPORT	Specifies that your camera is using storing a persistent (static) IP address.				f	g									
M_GC_PHYSICAL_LINK_CONFIGURATION_CAPABILITY		Inquires whether your camera supports a dynamic or static link aggregation group (LAG) in either a multiple or single link configuration. Note that a combination of the values below can be returned. Bitwise operators must be used to verify the presence of a specific capability.				f	g									
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g									
	M_GC_DYNAMIC_LINK_AGGREGATION_SUPPORT	Specifies that your camera supports a dynamically configured LAG.				f	g									
	M_GC_MULTIPLE_LINK_SUPPORT	Specifies that your camera supports multiple links.				f	g									
	M_GC_SINGLE_LINK_SUPPORT	Specifies that your camera supports a single link.				f	g									
	M_GC_STATIC_LINK_AGGREGATION_SUPPORT	Specifies that your camera supports a static LAG.				f	g									
M_GC_STREAM_CHANNEL_CAPABILITY		Inquires the list of capabilities specific to the stream channels of your camera. Note that a combination of the values below can be returned. Bitwise operators must be used to verify the presence of a specific capability.				f	g									
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g									

	M_GC_ALL_IN_SUPPORT	Specifies that the stream channel supports the All-in transmission mode when transmitting data.				f	g										
	M_GC_BIG_AND_LITTLE_ENDIAN_SUPPORT	Specifies that your camera supports both big and little-endian on the stream channel. Note that all stream channels must support little-endian.				f	g										
	M_GC_EXTENDED_CHUNK_DATA_SUPPORT	Specifies that the stream channel supports an older formatting for chunk data.				f	g										
	M_GC_IP_REASSEMBLY_SUPPORT	Specifies that the stream channel(s) supports the reassembly of fragmented IP packets when receiving data.				f	g										
	M_GC_MULTI_ZONE_SUPPORT	Specifies that the stream channel supports transmission of uncompressed image data that is sliced into horizontal strips.				f	g										
	M_GC_PACKET_RESEND_OPTION_SUPPORT	Specifies that the stream channel supports unconditional streaming capabilities when transmitting.				f	g										
	M_GC_UNCONDITIONAL_STREAMING_SUPPORT	Specifies that the stream channel supports alternate destinations for packet resend requests.				f	g										
M_GC_STREAM_PROTOCOL_CAPABILITY		Inquires how GVSP traffic (typically, unidirectional in nature) supports bypassing the firewall or other network restrictions. Note that a combination of the values below can be returned. Bitwise operators must be used to verify the presence of a specific capability.				f	g										
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*				f	g										
	M_GC_FIREWALL_TRAVERSAL_SUPPORT	Specifies that the firewall will not block message packets.				f	g										
	M_GC_LEGACY_16BIT_BLOCK_SUPPORT	Specifies that your camera supports legacy GEV 1.x stream mode.				f	g										

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MdigInquireFeature

Synopsis

Inquires a feature of the camera.

Syntax

```
void MdigInquireFeature(
    AIL_ID DigId,                  //in
    AIL_INT64 InquireType,         //in
    AIL_CONST_TEXT_PTR FeatureName, //in
    AIL_INT64 UserVarType,         //in
    void *UserVarPtr               //out
)
```

Description

This function allows you to directly inquire various GenICam standard feature naming convention (SFNC) features and manufacturer-specific features of the camera. When used with a GenICam SFNC-compliant camera, it allows you to directly inquire various manufacture-specific features specified with the camera's device description file (XML). Note that for the purpose of this function, a feature can also be of type category or command. Although category and command features lack a feature value, all other attributes of the feature can be inquired. For a list of standard category feature names, see the Standard Feature Naming Convention or your GenICam SFNC-compliant camera's documentation.

[Zebra Radient eV-CL; Zebra Rapixo CL]

For Camera Link boards to work with this function, you must first configure and enable the GenICam CLProtocol module, using MdigControl() with M_GC_CLPROTOCOL. Note that this control type is only available after the required third-party vendor-supplied, standard compliant, CLProtocol library for your Camera Link camera is installed on your computer and M_GC_CLPROTOCOL_DEVICE_ID is set. For more information, refer to the Using Aurora Imaging Library with GenICam section of Chapter 27: Grabbing with your digitizer.

The features described in this function are primarily available to check existing settings using Aurora Imaging Library code or to retrieve information from which you can build your own interface for the camera device's description file (XML). Aurora Imaging Library provides two versions of an interface that you can use to interactively inquire the camera's features. At design-time, you can use Aurora Imaging Intellicam's Feature Browser. At runtime, you can launch Feature Browser, using MdigControl() with M_GC_FEATURE_BROWSER. Both versions of Feature Browser provide a list of available feature names and associated possible values, as well as code snippets with the Aurora Imaging Library functions and Aurora Imaging Library constants associated with the settings selected; you can copy the code snippets to your Aurora Imaging Library application code.

Occasionally, the returned information is a string whose length can change between when you inquires its size and when you inquire its value (for example, the temperature can change from 99 to 100 between calls). In this case, when you inquire the information's length to allocate an array of an appropriate size, you might allocate an array that is too small to retrieve the data and experience an application crash. To avoid an application crash, specify the length of the array passed to the UserVarPtr parameter using the M_FEATURE_USER_ARRAY_SIZE() macro; the data will be truncated to fit in the array and an error will be generated.

To control a camera's manufacturer-specific feature, use MdigControlFeature().

Parameters

DigId

Specifies the identifier of the digitizer accessing the camera to inquire. This parameter must be given a valid digitizer identifier, previously allocated using MdigAlloc().

InquireType

Specifies the type of information to inquire about the feature.

See the Parameter associations section for possible values that can be specified.

FeatureName

Specifies the name of the camera feature to inquire. Note that this parameter expects a case-sensitive string.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the name of the feature																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

AIL_TEXT("*FeatureName*") 1	Specifies the name of the feature. Note that the feature name is case-sensitive. Refer to your camera's documentation for a list of the features available.			e	f	g					j	k	l	m	n	b
AIL_TEXT("*Root*") 1	Specifies to inquire the highest-level feature of the XML structure; this feature is of type category and typically, its subfeatures are also of type category. Note that this value is only available when using M_SUBFEATURE_....			e	f	g					j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

UserVarType

Specifies the data type of the address pointed to by the UserVarPtr parameter. If not inquiring the feature's value (using M_FEATURE_VALUE), set this parameter to **M_DEFAULT**.

Note that, in the case of a string (or a multi-byte register), the size of the array passed to the UserVarPtr parameter must be equal to or larger than the returned information; otherwise an application crash will occur.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the UserVarPtr's data type																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TYPE_BOOLEAN	Specifies that UserVarPtr is passed an address of type *AIL_BOOL*.			e	f	g					j	k	l	m	n	b
M_TYPE_DOUBLE	Specifies that UserVarPtr is passed an address of type *AIL_DOUBLE*.			e	f	g					j	k	l	m	n	b
M_TYPE_INT64	Specifies that UserVarPtr is passed an address of type *AIL_INT64*.			e	f	g					j	k	l	m	n	b
M_TYPE_STRING +	Specifies that UserVarPtr is passed an address of type *AIL_TEXT_CHAR*. You must specify a combination value from the following table: • To specify the length of the array			e	f	g					j	k	l	m	n	b
M_TYPE_UINT8 +	Specifies that UserVarPtr is passed an address of type *AIL_UINT8*. You must specify a combination value from the following table: • To specify the length of the array			e	f	g					j	k	l	m	n	b

Combination value for M_TYPE_STRING; M_TYPE_UINT8.

You must add the following value to the above-mentioned values to specify the length of the array.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the length of the array																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FEATURE_USER_ARRAY_SIZE(AIL_INT Length)	Specifies the length of the array.			e	f	g					j	k	l	m	n	b
	Parameters															
	Length Sets the length, in bytes.															

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_BOOL [optionally, in C++: a reference to a std::vector<AIL_BOOL>]
- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]

- array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>]
- AIL_BOOL
- AIL_DOUBLE
- AIL_INT
- AIL_INT64

Specifies the address of the variable in which to return the requested information.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For specifying the type of information to inquire about the feature and the data type returned
- For inquiring about the enumeration entry of a feature's value
- For inquiring about the subfeatures of a feature

The following inquire types are used to specify the type of information about the feature to inquire and the data type returned.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of information to inquire about the feature and the data type returned																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_FEATURE_ACCESS_MODE +	Inquires whether the specified feature is implemented, available, readable, and/or writable. Note that an error is generated if the feature does not exist in the camera's device description file (XML). To learn if the specified feature exists in the file and is implemented on your camera, use M_FEATURE_PRESENT.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e	f	g					j	k	l	m	n	b
Bit-encoded access mode value	Specifies a bit-encoded value that details the access mode. To establish the access mode from the encoded value, use the following macros: • To learn whether the specified feature is available, use the **M_FEATURE_IS_AVAILABLE** macro. Note that if a feature is not available, use **M_FEATURE_IS_IMPLEMENTED** to learn whether the specified feature is only temporarily unavailable or actually not implemented on your camera. A feature that is temporarily unavailable typically relies on the setting of another feature (for example, when a trigger feature is set to off, its associated trigger polarity feature would be unavailable). • To learn whether the specified feature is implemented, use the **M_FEATURE_IS_IMPLEMENTED** macro. For example, a Bayer color feature would not be implemented on a monochrome camera because the hardware cannot support it. • To learn whether the specified feature can be read, use the **M_FEATURE_IS_READABLE** macro. • To learn whether the specified feature can be written to, use the **M_FEATURE_IS_WRITABLE** macro.			e	f	g					j	k	l	m	n	b

M_FEATURE_CACHING_MODE +	Inquires whether a copy of the specified feature's value is stored in Host memory for faster access and retrieval. If so, the feature is known as a cachable feature. A cachable feature is typically only retrieved from the camera whenever the cache is invalidated (such as, before the first read and before the first read to occur after a write operation). A cachable feature is typically a value that only changes based on user-interaction (setting a control value). For example, a temperature feature is not cachable. To return a true/false value for this inquire, use the **M_FEATURE_IS_CACHABLE** macro on the returned result.			e	f	g					j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e	f	g					j	k	l	m	n	b
M_FEATURE_CACHING_MODE_NONE	Specifies that the feature is not cachable.			e	f	g					j	k	l	m	n	b
M_FEATURE_CACHING_MODE_WRITE_AROUND	Specifies that feature's values are written only to the camera, and the cache is updated the next time the feature's value is inquired.			e	f	g					j	k	l	m	n	b
M_FEATURE_CACHING_MODE_WRITE_THROUGH	Specifies that when information is written to the camera, the cache is updated simultaneously.			e	f	g					j	k	l	m	n	b
M_FEATURE_CHANGE_HOOK +	Inquires whether an event occurs when the value of the specified feature changes. If enabled, MdigHookFunction() with M_FEATURE_CHANGE hooks a specified function to the feature change event. SET			e	f	g					j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g					j	k	l	m	n	b
M_DISABLE	Specifies that no event is generated when the value of the specified feature changes. This is the default value.			e	f	g					j	k	l	m	n	b
M_ENABLE	Specifies that an event is generated when the value of the specified feature changes.			e	f	g					j	k	l	m	n	b
M_FEATURE_DEPRECATED +	Inquires whether the specified feature is deprecated and should no longer be used.			e	f	g					j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e	f	g					j	k	l	m	n	b
M_FALSE	Specifies that the feature is not depreciated.			e	f	g					j	k	l	m	n	b
M_TRUE	Specifies that the feature is depreciated.			e	f	g					j	k	l	m	n	b
M_FEATURE_DESCRIPTION +	Inquires the description of the specified feature.			e	f	g					j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g					j	k	l	m	n	b
Value	Specifies the description of the feature.			e	f	g					j	k	l	m	n	b
M_FEATURE_DISPLAY_NAME +	Inquires the specified feature's display name. Note that this can differ slightly from the internally-used feature name (M_FEATURE_NAME).			e	f	g					j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g					j	k	l	m	n	b
Value	Specifies the display name of the feature.			e	f	g					j	k	l	m	n	b

M_FEATURE_EXECUTE_COMPLETED +	Inquires whether the specified executable camera feature (command) has finished executing on your camera. Note that to enable command execution-completion polling, set MsysControl() with M_GC_FEATURE_EXECUTE_POLLING_MODE to M_AUTOMATIC.			e	f	g										b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_BOOL*			e	f	g										b
M_FALSE	Specifies that the executable camera feature has not finished executing.			e	f	g										b
M_TRUE	Specifies that the executable camera feature has finished executing.			e	f	g										b
M_FEATURE_INCREMENT +	Inquires the value by which the feature's value increments. For example, if a feature's possible values are 2, 4, 6, ..., then the feature's value increments by 2.			e	f	g				j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e	f	g				j	k	l	m	n	b
Value	Specifies the feature's incremental value.			e	f	g				j	k	l	m	n	b
M_FEATURE_MAX +	Inquires the maximum possible value for the feature.			e	f	g				j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *address of an AIL_INT64*			e	f	g				j	k	l	m	n	b
Value	Specifies the maximum possible value for the feature.			e	f	g				j	k	l	m	n	b
M_FEATURE_MIN +	Inquires the minimum possible value for the feature.			e	f	g				j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *address of an AIL_INT64*			e	f	g				j	k	l	m	n	b
Value	Specifies the maximum possible value for the feature.			e	f	g				j	k	l	m	n	b
M_FEATURE_NAME +	Inquires the specified internally-used feature name. Note that this can differ slightly from the displayed name of the feature (M_FEATURE_DISPLAY_NAME).			e	f	g				j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g				j	k	l	m	n	b
Value	Specifies the feature's name.			e	f	g				j	k	l	m	n	b
M_FEATURE_POLLING_INTERVAL +	Inquires the interval between inquiring an executable feature's completion status, when automatically polling (using MsysControl() with M_GC_FEATURE_EXECUTE_POLLING_MODE).			e	f	g						l	m		b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e	f	g						l	m		b
Value >= -1	Specifies the polling interval, in msecs.			e	f	g						l	m		b
M_FEATURE_PRESENT +	Inquires whether the specified feature is present in the camera's device description file. Note that, if the feature is listed in the camera's device description file (XML), use M_FEATURE_ACCESS_MODE and the **M_FEATURE_IS_IMPLEMENTED** macro to determine if the feature is supported. If the feature is not present in the file, M_FEATURE_PRESENT will return M_NO, whereas M_FEATURE_ACCESS_MODE will generate an error.			e	f	g				j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_BOOL*			e	f	g				j	k	l	m	n	b

	M_NO	Specifies that the specified feature is not present.			e	f	g					j	k	l	m	n	b
	M_YES	Specifies that the specified feature is present.			e	f	g					j	k	l	m	n	b
M_FEATURE_REPRESENTATION +		Inquires the display format for the feature's value. The display format dictates the size of the field and its form (for example, check box, slider, text-box) used in a user interface for the camera's device description file (XML), such as Feature Browser (accessible through Aurora Imaging Intellicam or MdigControl() with M_GC_FEATURE_BROWSER). Note that this inquire type can only be used with a feature that has a data type of *AIL_INT64* or *AIL_DOUBLE*.			e	f	g					j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e	f	g					j	k	l	m	n	b
	M_FEATURE_REPRESENTATION_BOOLEAN	Specifies that the feature should be displayed as a checkbox.			e	f	g					j	k	l	m	n	b
	M_FEATURE_REPRESENTATION_HEX_NUMBER	Specifies that the feature should be displayed as a hexadecimal value.			e	f	g					j	k	l	m	n	b
	M_FEATURE_REPRESENTATION_IPV4_ADDRESS	Specifies that the feature should be displayed as an IP address in dotted decimal notation.			e	f	g					j	k	l	m	n	b
	M_FEATURE_REPRESENTATION_LINEAR	Specifies that the feature should be displayed as a slider with the appropriate range shown.			e	f	g					j	k	l	m	n	b
	M_FEATURE_REPRESENTATION_LOGARITHMIC	Specifies that the feature should be displayed as a slider with the appropriate logarithmic range shown.			e	f	g					j	k	l	m	n	b
	M_FEATURE_REPRESENTATION_MAC_ADDRESS	Specifies that the feature should be displayed as a hexadecimal MAC address.			e	f	g					j	k	l	m	n	b
	M_FEATURE_REPRESENTATION_PURE_NUMBER	Specifies that the feature should be displayed as edit text-box with a decimal display.			e	f	g					j	k	l	m	n	b
M_FEATURE_SELECTOR_COUNT +		Inquires the number of selectors that are ancestors of the feature. Selectors take an enumerator or integer to index which instance of a subfeature should be accessed, when multiple instances of a subfeature exist. For example, the GenICam standard `TriggerSelector` feature is used to select which type of trigger to configure (for example, `FrameStart` and `AcquisitionStart` are both enumerators that can be passed to `TriggerSelector`). All subfeatures of the selector (such as its source) are referenced from the trigger selector (`TriggerSource[AcquisitionStart]`). This inquire type only returns a value greater than 0 if the feature is a subfeature of a selector. It returns 1 if the feature is a subfeature of a selector. It returns 2 if its selector is also referenced by another selector. The M_FEATURE_SELECTOR_COUNT for `TriggerMode`, in this example, would be 1. To inquires the name of the selector associated with the specified feature, use M_FEATURE_SELECTOR_NAME + n.			e	f	g									n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g									n	b
	Value	Specifies the number of ancestor selectors of the feature.			e	f	g									n	b

M_FEATURE_SELECTOR_NAME + *n* +	Inquires the name of the ancestor selector of the specified subfeature, where *n* is the number of generations. So, if inquiring the feature selector name of the 1st selector of `TriggerSource`, the returned name would be `TriggerSelector`. To inquires the number of selectors that are associated with the specified feature, use M_FEATURE_SELECTOR_COUNT.			e	f	g									n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g									n	b
Value	Specifies the name associated with the selector index.			e	f	g									n	b
M_FEATURE_SIZE +	Inquires the size of the feature's value.			e	f	g				j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g				j	k	l	m	n	b
Value	Specifies the feature's size, in bytes.			e	f	g				j	k	l	m	n	b
M_FEATURE_STREAMABLE +	Inquires whether the specified feature is streamable. A streamable feature can be stored in the local camera's description file (an XML file stored on your computer). Streamable features are typically not persistent values (such as pixel format and image size). In most cases, a streamable feature is capable of being saved and loaded from file at a later date, possibly on a different camera.			e	f	g				j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e	f	g				j	k	l	m	n	b
M_FALSE	Specifies that the feature is not streamable.			e	f	g				j	k	l	m	n	b
M_TRUE	Specifies that the feature is streamable.			e	f	g				j	k	l	m	n	b
M_FEATURE_TOOLTIP +	Inquires the tooltip of the specified feature.			e	f	g				j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g				j	k	l	m	n	b
Value	Specifies the feature's tooltip.			e	f	g				j	k	l	m	n	b
M_FEATURE_TYPE +	Inquires the specified feature's type.			e	f	g				j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e	f	g				j	k	l	m	n	b
M_TYPE_BOOLEAN	Specifies that the feature's value is a boolean.			e	f	g				j	k	l	m	n	b
M_TYPE_CATEGORY	Specifies that the feature is a category feature, and its value cannot be inquired.			e	f	g				j	k	l	m	n	b
M_TYPE_COMMAND	Specifies that the feature is a command and its value cannot be inquired.			e	f	g				j	k	l	m	n	b
M_TYPE_DOUBLE	Specifies that the feature's value is a floating-point.			e	f	g				j	k	l	m	n	b
M_TYPE_ENUMERATION	Specifies that the feature's value is an enumeration.			e	f	g				j	k	l	m	n	b
M_TYPE_INT64	Specifies that the feature's value is a 64-bit integer.			e	f	g				j	k	l	m	n	b
M_TYPE_REGISTER	Specifies that the feature's value is mapped to a multi-byte register.			e	f	g				j	k	l	m	n	b

	M_TYPE_STRING	Specifies that the feature's value is a string.			e	f	g					j	k	l	m	n	b
M_FEATURE_VALID_VALUE_COUNT		Inquires the number of valid values for the specified feature. Integer and floating-point type features can support a fixed list of valid values instead of the traditional minimum, maximum and increment values.			e	f	g									n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g									n	b
	Value	Specifies the number of valid values. > A value of 0 returned indicates the feature does not support a list of valid values. Minimum, maximum and increment values must be used instead.			e	f	g									n	b
M_FEATURE_VALID_VALUE + *n* +		Inquires the nth valid value of the feature, where *n* is a value between 0 and M_FEATURE_VALID_VALUE_COUNT.			e	f	g									n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *address of an AIL_INT64*			e	f	g									n	b
	Value	Specifies the valid value.			e	f	g									n	b
M_FEATURE_VALUE +		Inquires the current value of the feature.			e	f	g					j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_BOOL* **|** *address of an AIL_DOUBLE* **|** *address of an AIL_INT64* **|** *array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>]* **|** *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g					j	k	l	m	n	b
	Value	Specifies the feature's value.			e	f	g					j	k	l	m	n	b
M_FEATURE_VALUE_ARRAY		Inquires the array of values of the specified feature. The array of values will be inquired without needing to be indexed by a selector. This is internally handled by Aurora Imaging Library and simplifies implementation.			e	f	g					j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_BOOL [optionally, in C++: a reference to a std::vector<AIL_BOOL>]* **|** *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*			e	f	g					j	k	l	m	n	b
M_FEATURE_VALUE_ARRAY_SIZE		Inquires the number of elements in the specified feature's array of values. For use with M_FEATURE_VALUE_ARRAY.			e	f	g					j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e	f	g					j	k	l	m	n	b
	Value >= 0	Specifies the number of elements in the feature array.			e	f	g					j	k	l	m	n	b
M_FEATURE_VISIBILITY +		Inquires the visibility level of the specified feature. The visibility level indicates the level of expertise required to view or select the feature. When creating a custom user interface for the camera's device description file (XML), the visibility level of the specified feature can be used to potentially hide the specified feature. The current level of visibility should be set within your custom user interface, and the enumeration entry for the visibility level of the specified feature must be compared against it. If the specified enumeration entry has a level of visibility higher than the current level of visibility, it is hidden.			e	f	g					j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e	f	g					j	k	l	m	n	b
	M_FEATURE_VISIBILITY_BEGINNER	Specifies that the feature is suggested for beginners.			e	f	g					j	k	l	m	n	b

	M_FEATURE_VISIBILITY_EXPERT	Specifies that the feature is suggested for experts.			e	f	g					j	k	l	m	n	b
	M_FEATURE_VISIBILITY_GURU	Specifies that the feature is suggested for advanced experts.			e	f	g					j	k	l	m	n	b
	M_FEATURE_VISIBILITY_INVISIBLE	Specifies that the feature should not be shown in the user interface for the camera's device description file (XML). Note that an invisible feature can still be set or inquired using MdigControlFeature() and **MdigInquireFeature**(), respectively.			e	f	g					j	k	l	m	n	b

When the specified feature is of type enumeration, you can use one of the following to inquire about the specified enumeration entry in the supported enumeration list of the feature. When the information pertains to a specific enumeration entry (for example, M_FEATURE_ENUM_ENTRY_ACCESS_MODE + n), the value is described in relation to its position in the enumerated list, where *n* is the index into the enumerated list.

> Note that these values can only be used when the feature type to inquire is an enumeration.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the enumeration entry of a feature's value																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																	
M_FEATURE_ENUM_ENTRY_ACCESS_MODE + *n* +	Inquires whether the specified enumeration entry is implemented, available, readable, and/or writable, where *n* is the index into the enumerated list. Note that an error is generated if the enumeration entry does not exist in the camera's device description file (XML). To learn if the specified enumeration entry exists in the file and is implemented on your camera, use the **M_FEATURE_IS_IMPLEMENTED** macro.																
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e	f	g									n	b
Bit-encoded access mode value	Specifies a bit-encoded value that details the access mode. To establish the access mode from the encoded value, use the following macros: • To learn whether the specified enumeration entry is available, use the **M_FEATURE_IS_AVAILABLE** macro. Note that if an enumeration entry is not available, use **M_FEATURE_IS_IMPLEMENTED** to learn whether the specified enumeration entry is only temporarily unavailable or actually not implemented on your camera. An enumeration entry that is temporarily unavailable typically relies on the setting of another feature or enumeration entry. • To learn whether the specified enumeration entry is implemented, use the **M_FEATURE_IS_IMPLEMENTED** macro. • To learn whether the specified enumeration entry can be read, use the **M_FEATURE_IS_READABLE** macro.			e	f	g									n	b
M_FEATURE_ENUM_ENTRY_CACHING_MODE + *n* +	Inquires the caching mode of the specified enumeration entry, where *n* is the index into the enumerated list. The caching mode indicates whether a copy of the specified enumeration entry's value is stored in Host memory for the enumeration entry. If so, the enumeration entry is known as cachable. A cachable enumeration entry is typically only retrieved from the camera whenever the cache is invalidated (such as, before the first read and before the first read to occur after a write operation). A cachable enumeration entry is typically a value that only changes based on user-interaction (setting a control value). For example, a temperature feature is not cachable. To return a true/false value for this inquire, use the **M_FEATURE_IS_CACHABLE** macro on the returned result.			e	f	g									n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e	f	g									n	b
	M_FEATURE_CACHING_MODE_NONE	Specifies that the enumeration entry is not cachable.			e	f	g									n	b
	M_FEATURE_CACHING_MODE_WRITE_AROUND	Specifies that enumeration entry's values are written only to the camera, and the cache is updated the next time the enumeration entry's value is inquired.			e	f	g									n	b
	M_FEATURE_CACHING_MODE_WRITE_THROUGH	Specifies that when the enumeration entry's values are written to the camera, the cache is updated simultaneously.			e	f	g									n	b
M_FEATURE_ENUM_ENTRY_COUNT +		Inquires the total number of enumeration entries in the supported enumerated list of the feature. Note that this inquire type can only be used when the feature to inquire is of type enumeration.			e	f	g				j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g				j	k	l	m	n	b
	Value	Specifies the total number of enumeration entries.			e	f	g				j	k	l	m	n	b
M_FEATURE_ENUM_ENTRY_DESCRIPTION + *n* +		Inquires the description of the specified enumeration entry, where *n* is the index into the enumerated list. Note that this inquire type can only be used when the feature type to inquire is an enumeration.			e	f	g								n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g								n	b
	Value	Specifies the enumeration entry's description.			e	f	g								n	b
M_FEATURE_ENUM_ENTRY_DISPLAY_NAME + *n* +		Inquires the display name of the specified enumeration entry, where *n* is the index into the enumerated list. Note that this can differ slightly from the internally-used name of the entry (M_FEATURE_ENUM_ENTRY_NAME + n).			e										n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e										n	b
	Value	Specifies the enumeration entry's display name.			e										n	b
M_FEATURE_ENUM_ENTRY_NAME + *n* +		Inquires the internally-used name of the specified enumeration entry of the feature, where *n* is the index into the enumerated list. Note that this can differ slightly from the display name of the entry (M_FEATURE_ENUM_ENTRY_DISPLAY_NAME + n).			e	f	g				j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g				j	k	l	m	n	b
	Value	Specifies the enumeration entry's name.			e	f	g				j	k	l	m	n	b
M_FEATURE_ENUM_ENTRY_STREAMABLE + *n* +		Inquires whether the specified enumeration entry, where *n* is the index into the enumerated list is streamable. A streamable enumeration entry can be stored in the local camera's description file (an XML file stored on your computer). Streamable enumeration entries are typically not persistent values. In most cases, a streamable enumeration entry is capable of being saved and loaded from file at a later date, possibly on a different camera.			e	f	g								n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e	f	g								n	b
	M_FALSE	Specifies that enumeration entry is not streamable.			e	f	g								n	b
	M_TRUE	Specifies that when the enumeration entry is streamable.			e	f	g								n	b

M_FEATURE_ENUM_ENTRY_TOOLTIP + *n* +	Inquires the tool-tip for the specified enumeration entry, where *n* is the index into the enumerated list.			e	f	g									n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g									n	b
Value	Specifies the enumeration entry's tooltip.			e	f	g									n	b
M_FEATURE_ENUM_ENTRY_VALUE + *n* +	Inquires the enumeration integer value of the specified enumeration entry, where *n* is the index into the enumerated list.			e	f	g				j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e	f	g				j	k	l	m	n	b
Value	Specifies the enumeration entry's integer value.			e	f	g				j	k	l	m	n	b
M_FEATURE_ENUM_ENTRY_VISIBILITY + *n* +	Inquires the visibility level of the specified enumeration entry, where *n* is the index into the enumerated list. The visibility level indicates the level of expertise required to view or select the enumeration entry. When creating a custom user interface for the camera's device description file (XML), the visibility level of the specified enumeration entry can be used to potentially hide the specified enumeration entry. The current level of visibility should be set within your custom user interface, and the visibility level of the specified enumeration entry must be compared against it. If the specified enumeration entry has a level of visibility higher than the current level of visibility, it is hidden.			e	f	g								n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e	f	g								n	b
M_FEATURE_VISIBILITY_BEGINNER	Specifies that the enumeration entry is suggested for beginners.			e	f	g								n	b
M_FEATURE_VISIBILITY_EXPERT	Specifies that the enumeration entry is suggested for experts.			e	f	g								n	b
M_FEATURE_VISIBILITY_GURU	Specifies that the enumeration entry is suggested for advanced experts.			e	f	g								n	b
M_FEATURE_VISIBILITY_INVISIBLE	Specifies that the enumeration entry should not be shown in the user interface for the camera's device description file (XML). Note that an invisible enumeration entry can still be set or inquired using MdigControlFeature() and **MdigInquireFeature**(), respectively.			e	f	g								n	b

When the specified feature is of type category or "Root", you can use one of the following to inquire about its subfeatures. This is useful for enumerating the features of your camera.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the subfeatures of a feature																
InquireType UserVarPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SUBFEATURE_COUNT +	Inquires the number of subfeatures or subcategories that the specified category feature (node) has.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g					j	k	l	m	n	b
Value	Specifies the number of subfeatures.			e	f	g					j	k	l	m	n	b
M_SUBFEATURE_NAME + *n* +	Inquires the name of the specified subfeature, where *n* is the index of subfeature.			e	f	g					j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e	f	g					j	k	l	m	n	b

Value	Specifies the name of the subfeature.			e	f	g					j	k	l	m	n	b
M_SUBFEATURE_TYPE + *n* +	Inquires the type of the specified subfeature, where *n* is the index of subfeature.			e	f	g					j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e	f	g					j	k	l	m	n	b
M_TYPE_BOOLEAN	Specifies that the feature's value is a boolean.			e	f	g					j	k	l	m	n	b
M_TYPE_CATEGORY	Specifies that the feature is a category.			e	f	g					j	k	l	m	n	b
M_TYPE_COMMAND	Specifies that the feature is a command to be executed.			e	f	g					j	k	l	m	n	b
M_TYPE_DOUBLE	Specifies that the feature's value is a floating-point.			e	f	g					j	k	l	m	n	b
M_TYPE_ENUMERATION	Specifies that the feature's value is an enumeration.			e	f	g					j	k	l	m	n	b
M_TYPE_INT64	Specifies that the feature's value is a 64-bit integer.			e	f	g					j	k	l	m	n	b
M_TYPE_REGISTER	Specifies that the feature is mapped to a multi-byte register.			e	f	g					j	k	l	m	n	b
M_TYPE_STRING	Specifies that the feature's value is a string.			e	f	g					j	k	l	m	n	b

Combination values for the values listed in For inquiring about the enumeration entry of a feature's value, For inquiring about the subfeatures of a feature; and for the following values: M_FEATURE_ACCESS_MODE; M_FEATURE_CACHING_MODE; M_FEATURE_CHANGE_HOOK; M_FEATURE_DEPRECATED; M_FEATURE_DESCRIPTION; M_FEATURE_DISPLAY_NAME; M_FEATURE_EXECUTE_COMPLETED; M_FEATURE_INCREMENT; M_FEATURE_MAX; M_FEATURE_MIN; M_FEATURE_NAME; M_FEATURE_POLLING_INTERVAL; M_FEATURE_PRESENT; M_FEATURE_REPRESENTATION; M_FEATURE_SELECTOR_COUNT; M_FEATURE_SELECTOR_NAME + n; M_FEATURE_SIZE; M_FEATURE_STREAMABLE; M_FEATURE_TOOLTIP; M_FEATURE_TYPE; M_FEATURE_VALID_VALUE + n; M_FEATURE_VALUE; M_FEATURE_VISIBILITY.

You can add one of the following values to the above-mentioned values to specify the GenTL configuration file (XML file) that is associated with the feature.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the configuration file associated with the feature																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_GENTL_STREAM_NUMBER(AIL_INT IndexValue)	Specifies that the feature is associated with an instance of the GenTL stream configuration information.			e												
	Parameters															
	IndexValue Specifies the instance (index number) of the GenTL stream library associated with the feature.			e												
	0 <= *Value* <= M_GENTL_STREAM_COUNT — Specifies the index.			e												
M_GENTL_DEVICE	Specifies that the feature is associated with the GenTL device configuration information.			e												
M_GENTL_REMOTE_DEVICE	Specifies that the feature is associated with the GenTL remote device configuration information. This is the default value.			e												

Combination value for M_FEATURE_DESCRIPTION; M_FEATURE_DISPLAY_NAME; M_FEATURE_NAME; M_FEATURE_SELECTOR_NAME + n; M_FEATURE_TOOLTIP; M_FEATURE_VALUE; M_FEATURE_ENUM_ENTRY_DESCRIPTION + n; M_FEATURE_ENUM_ENTRY_DISPLAY_NAME + n; M_FEATURE_ENUM_ENTRY_NAME + n; M_FEATURE_ENUM_ENTRY_TOOLTIP + n; M_SUBFEATURE_NAME + n.

You can add the following value to the above-mentioned values to get the string's length.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To inquire the size of a string			(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
InquireType combination value		Description															
	UserVarPtr - Possible values returned																
M_STRING_SIZE		Retrieves the length of the string, including the terminating null character ("\0"). Note that, when used with M_FEATURE_VALUE, you can only use this combination value if the feature is of type string.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*			e	f	g					j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MdigProcess

Synopsis

Grab frames of data sequentially into a list of manually or automatically allocated image buffers or containers, and process them with a user-defined function as they are grabbed.

Syntax

```
void MdigProcess(
    AIL_ID DigId,                                  //in
    const AIL_ID *DestContainerOrImageBufArrayPtr, //in
    AIL_INT ImageCount,                            //in
    AIL_INT64 Operation,                           //in
    AIL_INT64 OperationFlag,                       //in
    AIL_DIG_HOOK_FUNCTION_PTR HookHandlerPtr,      //in
    void *UserDataPtr                              //in-out
)
```

Description

This function uses the specified digitizer to acquire frames of data and store them sequentially in a list of image buffers or a list of containers. It also hooks a user-defined function to the modification of any buffers/containers in the specified list. The user-defined function is therefore called every time a new frame is grabbed.

Optionally, instead of grabbing data into a list of previously allocated buffers or containers, this function can automatically allocate an appropriate image buffer or container to store each grabbed frame of data. This is particularly useful when grabbing from a camera that is configured to transmit images with different dimensions for each grab. To specify that the buffers/containers should be automatically allocated, set DestContainerOrImageBufArrayPtr to M_NULL.

> When explicitly passing an array of image buffer or containers, note the following. When grabbing an image, you should use an image buffer as a destination. When grabbing any other type of data (such as 3D data), you should use a container as a destination. You can determine what type of Aurora Imaging Library object the digitizer is designed to grab into using MdigInquire() with M_TARGET_BUFFER_OBJECT.

If set synchronously, **MdigProcess**() will block the thread until the list of sequential grabs has completed. If set asynchronously, the buffers/containers are prepared, and then control is returned to the calling thread while, in the background, the grabs will begin.

The grabs can stop at the end of the list, after a predefined number of grabs occur, or when a stop instruction is explicitly given. In the latter two cases, grabbing occurs in a round robin fashion; buffers/containers are filled with the grabbed data typically in the order they are stored in the list, wrapping around to the first buffer/container in the list once the last buffer/container in the list is filled.

The index of the modified buffer/container can be inquired using MdigGetHookInfo() with M_MODIFIED_BUFFER + M_BUFFER_INDEX. The Aurora Imaging Library identifier of the modified buffer/container can be inquired using MdigGetHookInfo() with M_MODIFIED_BUFFER + M_BUFFER_ID. To identify whether the modified buffer/container (that is, the current frame) is corrupt (for example, because a packet was not received successfully), use MdigGetHookInfo() with M_CORRUPTED_FRAME.

Once a frame of data is grabbed into a buffer/container, Aurora Imaging Library will not grab into that buffer/container again until the hook function has finished processing it. Once the hook function finishes executing, the buffer/container becomes available for Aurora Imaging Library to grab into it. If all buffers/containers are filled with data and are waiting to be processed by the hook function, Aurora Imaging Library waits for the next available buffer/container (typically the next buffer/container in the list) before grabbing another frame of data.

> You can use MdigInquire() with M_PROCESS_... to inquire the status of process operation. For example, if the average time it takes to process a frame of data is greater than the frame rate of a camera, frames will eventually be missed. To check if this is happening, you can use MdigInquire() with M_PROCESS_PENDING_GRAB_NUM in the hook function to determine if the expected number of processing buffers are currently available.

To grab until a stop instruction, you must call **MdigProcess**() with M_START; when you want to stop grabbing, you must call **MdigProcess**() with M_STOP. Once the stop instruction is issued, no more frames will be grabbed. Buffers/containers that store unprocessed frames are processed before the original calling thread continues, unless the stop instruction is issued by calling **MdigProcess**() from within the hook-handler function itself (in which case, processing is completed while the original calling thread continues).

By default, buffers/containers are processed sequentially, in the order into which grabbed; a single instance of the hooked user-defined function will run on a single thread, processing one buffer/container at a time. When running on a computer with multiple CPU cores, you can have the grabbed buffers/containers handled by different instances of the user-defined function running on different threads. To do so, use MsysControl() with M_MODIFIED_BUFFER_HOOK_MODE set to M_MULTI_THREAD.

> If you are grabbing from multiple cameras simultaneously with the hook function executing in single-threaded mode, each instance of **MdigProcess**() will use its own thread to execute an instance of the hooked function. Additionally, many Aurora Imaging Library functions

> are transparently multi-threaded. These factors can combine to generate a large number of threads, potentially slowing down your application. Since having the hook function execute in multi-threaded mode can multiply the number of threads generated by the number of physical CPU cores in your computer (or, optionally, the number of threads that you specify with M_MULTI_THREAD), it is often more efficient to have the hook function execute in single-threaded mode. If you are considering multi-threaded mode, you should benchmark your application under real-world conditions before making a final decision.

When processing with the hook function executing in multi-threaded mode, it is possible for buffers/containers to be grabbed and processed in a different order than they are stored in the list because they are processed at different rates. For example, if the list contains 3 buffers and all of them are being processed, it is possible that processing will finish on the second buffer in the list before the first buffer in the list. In this case, Aurora Imaging Library will grab into the second buffer and cue it for processing instead of waiting for the first buffer to become available.

When the specified digitizer is set to perform triggered grabs, the default behavior of the function is to wait for a trigger event before grabbing each frame. To grab all or a specific number of frames per trigger, set the OperationFlag parameter to M_TRIGGER_FOR_FIRST_GRAB and use the Operation parameter to specify how many frames to grab per trigger. To grab continuously after just one trigger, use + M_FRAMES_PER_TRIGGER() set to M_INFINITE.

> Note that if the camera is in triggered mode, you should not also use the digitizer in triggered mode.

It is not recommended to change the digitizer settings of the digitizer currently being used when grabs are still pending.

[Host system]

When grabbing from a video or directory of files, once the last image is grabbed, grabbing will restart from the beginning of the video or from the first file in the directory.

Parameters

DigId

Specifies the identifier of the digitizer to be used to perform the grabs.

DestContainerOrImageBufArrayPtr
Accepts the address of one of the following:

- array of type AIL_ID [optionally, in C++: a reference to a constant std::vector<AIL_ID>] (and a maximum array rank for C# of 1)
  Required array size:(ImageCount)

Specifies the address of the array containing the identifiers of the image buffers or containers in which to place the grabbed frames of data, or specifies to automatically allocate image buffers or containers appropriate for the grabbed data.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Aurora Imaging Library identifiers to grab into																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies to automatically allocate appropriate image buffers or containers with M_PROC to store grabbed frames of data. Automatically allocated image buffers always have the correct size, type, and number of bands to store the grabbed data. > You can specify the maximum number of buffers or containers to internally allocate using the ImageCount. You should specify a sufficient number of internal buffers or containers to allow grabbing new frames of data while previously grabbed frames are being processed. Otherwise, frames of data could be missed. To learn the Aurora Imaging Library identifiers of the image buffers or containers that were grabbed into, use MdigGetHookInfo() with M_MODIFIED_BUFFER + M_BUFFER_ID within the hook-handler function. > You cannot use the Aurora Imaging Library identifiers of automatically allocated buffers outside of the hook function, and you should assume that the memory allocated for each buffer or container will be freed immediately after the hook-handler function that processes it returns.	c		e	f	g	a		i		j	k	l	m	n	b

Color image buffer identifier	Specifies an array containing the identifiers of color destination image buffers with an M_IMAGE + M_GRAB attribute. The following pertains to each image buffer. This image buffer must have been previously allocated, typically using MbufAllocColor(). This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. If the grab buffer is smaller than the digitizer's frame size, the image buffer will be filled and all other data will be lost. If the grab buffer is larger than the digitizer's frame size, the buffer will be filled up to the digitizer's frame size. The rest of the buffer will remain untouched. > If you grab into a buffer from a digitizer that outputs data in a format suitable for a container, the first intensity component in the frame of data will be grabbed. If there is no intensity component, the first component that is not metadata will be grabbed.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific														
	The destination buffer's depth can only be 8 bits per band.	c													
	The destination buffer's depth can be 8 or 16 bits per band.			e	f	g								n	b
	When dealing with color smart-cameras, the destination image buffer can only be a BGR packed buffer with a 32-bit color depth (allocated using MbufAllocColor() with M_IMAGE + M_GRAB + M_BGR32). When dealing with monochrome smart-cameras, you cannot grab into an Aurora Imaging Library color image buffer.							i							
Container identifier	Specifies an array containing the identifiers of containers with an M_GRAB attribute. The following pertains to each container. This container must have been previously allocated, typically using MbufAllocContainer(). When grabbing into a container for the first time, Aurora Imaging Library will free all existing components and allocate the components required to store the current frame of data. Aurora Imaging Library will typically reuse these automatically allocated components during subsequent grabs, unless you add or remove components from the container (in which case all components will be freed and new components will again be allocated for the grabbed data). > Aurora Imaging Library will not grab into components of the container that you have allocated manually using MbufAllocComponent(), MbufCopyComponent(), or MbufCreateComponent(). In rare applications, you might configure your camera/3D sensor to transmit components of a different size or format with each grab. If there is any mismatch between the components from the previous grab and those required to store the current frame of data (for example, if the ordering, component type, or size has changed), all components will be automatically freed and new components will be allocated; the components will be assigned new identifiers. You can inquire the new buffer identifiers using MbufInquireContainer() with M_COMPONENT_ID. > If you need the buffer identifier of a component, and it is possible that the components might be reallocated, you should inquire the current buffer identifier after each grab.				f	g	a							n	b

Monochrome image buffer identifier	Specifies an array containing the identifiers of monochrome destination image buffers with an M_IMAGE + M_GRAB attribute. The following pertains to each image buffer. This image buffer must have been previously allocated, typically using MbufAlloc2d(). This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. It is not possible to grab into a color-band child buffer (that is, a child buffer created from one band of a color parent buffer) when the parent buffer is packed. When grabbing into an image buffer, if the grab destination buffer is smaller than the digitizer's frame size, the image buffer will be filled and all other data will be lost. If the grab destination buffer is larger than the digitizer's frame size, the buffer will be filled up to the digitizer's frame size. The rest of the buffer will remain untouched. > If you grab into a buffer from a digitizer that outputs data in a format suitable for a container, the first intensity component in the frame of data will be grabbed. If there is no intensity component, the first component that is not metadata will be grabbed.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific														
	When using a color camera, the color data is first converted into YUV, and then the Y (luminance) component is stored in the buffer. The color space conversion is done in hardware.	c						i				l	m		
	The destination buffer's depth can only be 8 bits per band.	c													
	The destination buffer's depth can be 8 or 16 bits per band.			e	f	g				j	k	l	m	n	b
	The destination buffer can only be an 8-bit monochrome buffer.							i							

ImageCount INQ

Specifies the number of buffers or containers in the array, or the number of internal buffers or containers to automatically allocate.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the DestContainerOrImageBufArrayPtr parameter.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the number of buffers or containers in the array																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies to automatically allocate the default number of internal buffers or containers (5); alternatively, in C++, if you passed a *std::vector<AIL_ID>* to the DestContainerOrImageBufArrayPtr, you can use this setting to have Aurora Imaging Library automatically determine the size. In the former case, this setting is only available when the DestContainerOrImageBufArrayPtr parameter is set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of buffers or containers in the array, or the number of internal buffers or containers to automatically allocate.	c		e	f	g	a		i		j	k	l	m	n	b

Operation

Specifies the type of operation to perform.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of operation to perform																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_SEQUENCE +	Grabs a specific number of frames, storing them sequentially in a list of buffers or containers. Grabbing ends once the specified number of frames have been captured. When using a digitizer set to perform triggered grabs and the OperationFlag parameter is set to M_TRIGGER_FOR_FIRST_GRAB, the function will grab the specified number of frames when the first trigger event occurs.	c		e	f	g	a		i		j	k	l	m	n	b
M_START +	Starts grabbing round-robin into the list of buffers or containers; the grabs will continue until stopped (using **MdigProcess**() with M_STOP). When using a digitizer set to perform triggered grabs and the OperationFlag parameter is set to M_TRIGGER_FOR_FIRST_GRAB, the function will, by default, start grabbing all frames when the first trigger event occurs.	c		e	f	g	a		i		j	k	l	m	n	b
M_STOP +	Stops the grab. By default, it also cancels all grabs that might have been queued. Note that cancelled grabs will not trigger the hooked function, but the events for the previous grabs are still processed. To stop a grab started in synchronous mode, the Operation parameter must be set to M_STOP and the **MdigProcess**() function called from a separate thread.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_STOP.

You can add the following value to the above-mentioned value to specify that previously queued grabs should not be cancelled when stopping.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_STOP																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_WAIT	Stops queuing new grabs and waits for all previously queued grabs to finish before returning control to the Host.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_SEQUENCE.

You can add the following value to the above-mentioned value to specify the number of frames to grab.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_SEQUENCE																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COUNT (AIL_INT n)	Specifies the number of frames to grab in the sequence. This combination constant will cause **MdigProcess**() to stop grabbing when the grab count (*n*) is reached. If the grab count is greater than the buffer count, the grab will be done in round-robin style. By default, the grab count is equal to the number of buffers or containers in the destination image array.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	n Specifies the number of frames to grab.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL — Specifies for a simulated digitizer to grab all frames in the specified source (AVI, directory, list of files) once.						a									
	1 <= *Value* <= 268435456 — Sets to grab a specific number of frames.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_START.

You can add the following value to the above-mentioned value to specify the number of frames to grab sequentially when a digitizer trigger event occurs.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_START																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FRAMES_PER_TRIGGER (AIL_INT NumberOfFrames)	Specifies the number of frames to grab sequentially when a digitizer trigger event occurs. Note that to use this setting, the OperationFlag parameter must be set to M_TRIGGER_FOR_FIRST_GRAB and the digitizer used must be configured to grab upon a trigger.	c							i		j	k	l	m		
	Parameters															
	NumberOfFrames Specifies the number of frames to grab sequentially for each digitizer trigger event.	c							i		j	k	l	m		
	M_INFINITE — Sets to grab an infinite number of frames. No other trigger is necessary.	c							i		j	k	l	m		
	1 <= *Value* <= 268435456 — Sets to grab a specific number of frames.	c							i		j	k	l	m		

OperationFlag

Specifies more information about the synchronization and triggering of the grab.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling the order in which processing occurs																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default execution mode. If the Operation parameter is set to M_START, the default is the same as M_ASYNCHRONOUS. If the Operation parameter is set to M_SEQUENCE, the default is the same as M_SYNCHRONOUS. If the Operation parameter is set to M_STOP, the default is the same as M_SYNCHRONOUS unless **MdigProcess**() is called from the user-defined function, in which case the default is the same as M_ASYNCHRONOUS. Note that when the Operation parameter is set to M_STOP, M_DEFAULT is the only possible setting for the OperationFlag parameter.	c		e	f	g	a		i		j	k	l	m	n	b
M_ASYNCHRONOUS +	Allows your thread to continue after initiating the start of the grabs, rather than waiting for **MdigProcess**() to finish. Note that when you are grabbing and processing a sequence of images asynchronously, you must make an additional call to MdigProcess using M_STOP to free internally allocated resources.	c		e	f	g	a		i		j	k	l	m	n	b
M_SYNCHRONOUS +	Synchronizes your thread with the end of **MdigProcess**().	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_ASYNCHRONOUS; M_SYNCHRONOUS.

You can add one of the following values to the above-mentioned values to specify how the camera calibration context associated with the digitizer is propagated to each modified image buffer in the list.

> Note that these settings are not available when grabbing into a container.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For how the camera calibration context is propagated																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_CALIBRATION_PROPAGATE_AT_EACH_FRAME	Specifies that the camera calibration context associated with the digitizer is re-associated with the buffer each time the buffer is modified and just before calling the hook-handler function. This is less error-prone than M_CALIBRATION_PROPAGATE_ONCE, but it might slow down the hook-handler function.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATION_PROPAGATE_OFF	Specifies to not affect the camera calibration context associated with the modified image buffer. You must associate the required camera calibration context with the image buffers before calling **MdigProcess**(), if necessary.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATION_PROPAGATE_ONCE	Specifies that before starting image acquisition, the camera calibration context associated with the digitizer is also associated with each image buffer. > Since the camera calibration context associated with the digitizer is only propagated at the beginning of the grab sequence, be cautious when using the hook-handler function to change the camera calibration context settings; the camera calibration context associated with the digitizer will not be re-associated with the modified buffer afterwards. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_ASYNCHRONOUS; M_SYNCHRONOUS.

You can add one of the following values to the above-mentioned values to specify whether to keep or remove the ROI.

> Note that these settings are not available when grabbing into a container.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to clear the ROI of the previous grab																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_REGION_DELETE_AT_EACH_FRAME	Specifies that any ROI associated with the modified buffer is deleted immediately before calling the user-defined hook function. This is equivalent to calling MbufSetRegion() to clear the ROI of the modified buffer after every grab. This is less error-prone, but might slow down the user-defined hook function's execution slightly.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_KEEP	Specifies that any ROI associated with the modified buffer is left as is. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_ASYNCHRONOUS; M_SYNCHRONOUS.

You can add the following value to the above-mentioned values to set how the function will behave when the digitizer is configured for triggered grabs.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For only controlling the grab of the first frame with a trigger																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TRIGGER_FOR_FIRST_GRAB	Specifies that a trigger will only be used to control the grab of the first frame. When there is no preset end to the list of grabs (M_START), you can also force every *n* th grab to wait for a trigger. To do so, set the Operation parameter to M_START + M_FRAMES_PER_TRIGGER(). > Note that if the digitizer is not configured for triggered grabs, M_TRIGGER_FOR_FIRST_GRAB is ignored. Also, when the digitizer is configured for triggered grabs, but M_TRIGGER_FOR_FIRST_GRAB is not set (the default behavior), the function will only grab each frame when triggered.	c		e	f	g	a		i		j	k	l	m	n	b

HookHandlerPtr

Specifies the address of the function that is called when a buffer/container is modified in the array of destination image buffers/containers.

The hook-handler function, pointed to by HookHandlerPtr, must be declared as follows:

```
AIL_INT MFTYPE HookHandler(
    AIL_INT HookType,
    AIL_ID EventId,
    void *UserDataPtr
)
```

Parameters

HookType

Type of event that generated the call (M_MODIFIED_BUFFER).

EventId

Event identifier to pass to MdigGetHookInfo() when inquiring about the hooked event.

UserDataPtr

User data pointer that was passed (as UserDataPtr) to **MdigProcess**().

Upon successful completion, the hook-handler function should return **M_NULL**. Note *MFTYPE* and *AIL_DIG_HOOK_FUNCTION_PTR* are reserved Aurora Imaging Library predefined types for functions and data pointers.

If the hook-handler function modifies the buffer or container which triggered the hook-handler function, it will not be called again for that buffer/container.

UserDataPtr

Specifies the address of the user data that you want to make available to the hook-handler function. This address is passed to the hook-handler function, through its UserDataPtr parameter, when the specified event occurs. Set this parameter to **M_NULL** if not used.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.